

UNIVERSIDADE
POLITÉCNICA
DO CÁVADO
E DO AVE

Relatório do Trabalho Prático 1

Ano Letivo de 2026/2027

Licenciatura em Engenharia de Sistemas Informáticos
Integração de Sistemas de Informação

Alunos: a29369 - Simão Martinho Vidal
a29370 - Rúben Salgado

Professor: Óscar Ribeiro

Barcelos | Outubro, 2026

Resumo

Com este trabalho prático da unidade curricular de Integração de Sistemas de Informação (ISI) pretende-se explorar e desenvolver processos de interoperabilidade entre sistemas, assentes no processamento de dados e em serviços web.

O projeto contempla a criação de uma pipeline de ingestão e transformação de dados no KNIME e Pentaho Data Integration (Kettle) , focada na análise de estatísticas do torneio de CS2 (com métricas de Kills, Deaths e cálculo de K/D Ratio para as equipas NaVi, G2 e Vitality). A arquitetura desenvolvida integra a transmissão automatizada de payloads via HTTP POST para o Node-RED, o acionamento de alertas por e-mail e a visualização dinâmica em tempo real através de um Dashboard interativo com manómetros (gauges) e gráficos.

Palavras-Chave: Integração de Sistemas de Informação, KNIME, Pentaho Data Integration (Kettle) , Node-RED, HTTP POST, RESTful, CS2, Dashboard, Streaming de Dados.

Lista de Abreviaturas e de Siglas

- API — Application Programming Interface (Interface de Programação de Aplicações)
- CS2 — Counter-Strike 2
- CSV — Comma-Separated Values (Valores Separados por Vírgulas)
- G2 — G2 Esports (Equipa de eSports)
- HTTP — Hypertext Transfer Protocol (Protocolo de Transferência de Hipertexto)
- ISI — Integração de Sistemas de Informação
- JSON — JavaScript Object Notation (Notação de Objetos JavaScript)
- K/D — Kills/Deaths (Rácio de Baixas/Mortes)
- LESI — Licenciatura em Engenharia de Sistemas Informáticos
- NaVi — Natus Vincere (Equipa de eSports)
- NoSQL — Not Only SQL (Não Apenas SQL)
- REST — Representational State Transfer (Transferência de Estado Representacional)
- SMTP — Simple Mail Transfer Protocol (Protocolo Simples de Transferência de Correio)
- UI — User Interface (Interface de Utilizador)
- XML — Extensible Markup Language (Linguagem de Marcação Extensível)

Índice / Índice de Figuras

Resumo **2**
Lista de Abreviaturas e de Siglas **3**
Índice / Índice de Figuras **4**
1 Introdução **7**
2 Motivação **8**
3 Objetivos **9**
4 Fontes de Dados **10**
Fig 1 - Json Partidas 11
Fig 2 - CSV Jogadores 11
5 Processamento e Limpeza de Dados Estruturados (CSV) **11**
CSV Reader: 11
Fig 3 - CSV Reader 12
Duplicate Row Filter: 12
Fig 4 - Duplicate Row Filter 13
Row Filter: 13
Fig 5 - Row Filter 14
String Manipulation: 14
Fig 6 - String Manipulation 15
6 Ingestão de Dados JSON e Consumo de API REST (HTTP GET) **15**
JSON Reader: 15
Fig 7 - JSON Reader 16
GET Request: 16
Fig 8 - GET Request 17
JSON Path e Ungroup: 17
Fig 9 - JSON Path 18
Fig 10 - JSON Path Config 18
Fig 11 - Ungroup 18
JSON to Table 18
Fig 12 - JSON to Table 19
7 Cruzamento de Dados Relacionais **19**
Joiner 19
Fig 13 - Joiner 20
8 Desduplicação Pós-Junção **20**
Duplicate Row Filter 20
Fig 14 - Duplicate Row Filter 21
9 Agregação e Consolidação de Métricas por Equipa **21**
GroupBy 21
Fig 15 - GroupBy 22
Fig 16 - GroupBy config 22
Cálculo do Indicador de Desempenho **23**

Math Formula..... 23
Fig 17 - Math Formula..... 23
Fig 18 - Math Formula config..... 23
10 Ordenação de Desempenho por K/D Ratio.....24
Sorter.....24
Fig 19 - Sorter..... 24
11 Serialização e Envio de Dados em Tempo Real..... 25
Table to JSON..... 25
Fig 20 - Table to JSON..... 25
Fig 21 - POST Request..... 25
12 Persistência e Exportação Multiformato.....26
(CSV, JSON e XML).....26
Fig 22 - CSV (CSV Writer).....26
Fig 23 - JSON (JSON Writer)..... 27
Fig 24 - XML (JSON to XML e XML Writer).....28
13 Implementação do Servidor REST e Visualização de Partidas.....29
/Partidas..... 29
Fig 25 - HTTP GET (http in)..... 29
Fig 26 - Template.....30
Fig 27 - http response..... 31
Fig 28 - Pontuação em Tempo Real.....32
14 Endpoint de Ingestão de Dados e Deserialização..... 33
/SalvarDados.....33
Fig 29 - HTTP POST..... 33
Fig 30 - Conversão json..... 34
Fig 31 - http response..... 35
Fig 32 - Debug 1.....36
Fig 33 - Alerta / Relatório do Torneio de CS2.....37
Fig 34 - Relatório.....37
Fig 35 - Template.....38
Fig 36 - function 1 (NaVi).....39
Fig 37 - function 2 (G2)..... 39
Fig 38 - function 3 (Vitality).....39
Fig 39 - Manómetro naviKd..... 40
Fig 40 - Manómetro g2Kd.....41
Fig 41 - Manómetro vitalityKd.....42
15 - Interface Gráfica e Visualização Final.....43
Node-RED Dashboard..... 43
Fig 42 - Manómetros K/D Ratio.....43
Fig 43 - Tabela dinâmica..... 44
16 Visão Geral do Fluxo no Pentaho Kettle.....45
1. Fase 1 (Extração e Limpeza):..... 45
2. Fase 2 (Cruzamento e Transformação de Negócio):..... 45
3. Fase 3 (Persistência e Exportação):..... 46

Fig 44 - Visão Geral do Fluxo no Pentaho Kettle..... 46
17 Processamento e Limpeza de Dados Estruturados (CSV)..... 47
CSV file input:..... 47
Fig 45 - CSV file input..... 47
Sort rows e Unique rows:..... 48
Fig 46 - Sort rows e Unique rows..... 48
Regex evaluation:..... 49
Fig 47 - Regex evaluation..... 49
Filter rows:..... 50
Fig 48 - Filter rows..... 50
Replace in string:..... 51
Fig 49 - Replace in string..... 51
18 Ingestão de Dados JSON e Consumo de API REST (HTTP GET)..... 52
Generate rows e REST client:..... 52
Fig 50 - REST client (GET)..... 52
JSON input:..... 53
Fig 51 - JSON input..... 53
19 Cruzamento de Dados Relacionais..... 54
Sort rows e Merge join:..... 54
Fig 52 - Merge join..... 54
20 Limpeza Pós-Junção e Desduplicação..... 55
Select values:..... 55
Fig 53 - Select values..... 55
Sort rows 4 e Unique rows 2:..... 56
Fig 54 - Unique rows 2..... 56
21 Agregação e Consolidação de Métricas por Equipa..... 57
Group by e Calculator:..... 57
Fig 55 -Group by e Calculator e Cálculo de KD_Ratio..... 58
22 Ordenação de Desempenho por K/D Ratio..... 58
Sort rows 5:..... 58
Fig 56 - Sort rows 5..... 58
23 Persistência e Exportação Multiformato (CSV, JSON e XML)..... 59
Text file output (Exportação para CSV):..... 59
Fig 57 - Exportação Text file output..... 59
XML output:..... 60
Fig. 58 - XML Output..... 60
JSON output:..... 61
Fig 59 - Exportação JSON output..... 61
24 Serialização e Envio de Dados em Tempo Real..... 62
REST client 2:..... 62
:Fig 60- REST client 2 (POST)..... 62
25 Endpoint de Ingestão de Dados e Deserialização..... 63
Recepção do Pedido HTTP POST (http in)..... 63
Fig 61 - http in (Endpoint /GuardarDados)..... 63

Confirmação da Resposta HTTP (http):.....63
26 Encaminhamento Lógico, Alertas e Dashboards.....64
Fig 62 - change node.....64
Tabela Dinâmica (change e table):.....65
Fig 63 - table node.....65
Envio Automático de Alertas por E-mail:.....66
Fig 64 - email node (Alertas SMTP).....66
Separação Lógica por Equipas (Functions):.....67
Fig 65 - function node.....67
Manómetros de Desempenho (Gauges):.....68
Fig 66 - gauge node.....68
27 Interface Gráfica e Visualização Final.....69
Node-RED Dashboard:.....69
28 Vídeo com demonstração (QR Code) - Simão Vidal.....70
29 Video com Demonstração (QRCode) - Rúben Salgado.....71
30 Conclusão e Trabalhos futuros.....72
Conclusão.....72
Trabalhos futuros.....72
31 Anexos.....73
Exportação (CSV, JSON e XML).....73

1 Introdução

Este trabalho prático foi realizado no âmbito da Unidade Curricular (UC) de Integração de Sistemas de Informação (ISI) integrada no 1º semestre do 3º ano da Licenciatura em Engenharia de Sistemas Informáticos (LESI).

Os objetivos do desenvolvimento deste trabalho prático são os seguintes:

- Consolidar conceitos de Integração de Sistemas de Informação usando serviços web e arquiteturas orientadas a eventos;
- Desenhar arquiteturas de integração de sistemas, recorrendo a APIs de interoperabilidade (como o protocolo HTTP e payloads JSON);
- Explorar ferramentas de suporte ao desenvolvimento e orquestração de dados (KNIME Analytics Platform e Pentaho Data Integration (Kettle));
- Explorar novas tecnologias, frameworks e paradigmas para implementação de serviços web e visualização em tempo real (Node-RED e Node-RED Dashboard).

Pretendo ainda atingir os seguintes objetivos:

- Potenciar a experiência no desenvolvimento e integração de software;
- Facilitar a assimilação do conteúdo da UC;
- Aplicar e aprofundar os conhecimentos adquiridos durante as aulas práticas (agregação, filtragem, alertas e streaming de dados);

A metodologia utilizada neste trabalho prático de ISI foi o desenvolvimento prático experimental aliado à pesquisa bibliográfica.

2 Motivação

Num processo típico de desenvolvimento em Engenharia de Software e Integração de Sistemas, é muitas vezes necessário o desenvolvimento de uma infraestrutura de processamento de informação para suporte a sistemas de informação de áreas específicas de domínio (como eSports, monitorização em tempo real, logística, etc.).

No atual contexto dos sistemas distribuídos e orientados a eventos, torna-se fulcral modelar uma arquitetura onde pipelines de dados, serviços de notificação automática e ferramentas de visualização estejam devidamente integrados.

Por um lado, são necessários serviços específicos de ingestão, limpeza e agregação de dados para a área identificada (neste caso, o processamento de estatísticas de partidas de CS2 — Kills, Deaths e cálculo de K/D Ratio para as equipas NaVi, G2 e Vitality).
Por outro lado, pensando no potencial de utilização dessas aplicações, é essencial o desenvolvimento de endpoints e fluxos de comunicação para posteriores operações de interoperabilidade, a usar por aplicações terceiras que pretendam consumir estes dados em tempo real.

Assim, qualquer outra aplicação a desenvolver (seja um dashboard web, uma aplicação mobile ou um sistema de alertas) terá ao seu dispor um conjunto de serviços e fluxos que poderá usar de forma agnóstica.

Este trabalho pretende explorar esses dois cenários, no sentido de que:

a) Pretende desenvolver, testar e documentar um fluxo integrado de transformação e orquestração de dados no KNIME Analytics Platform, e Pentaho Data Integration (Kettle), complementado com gatilhos automatizados para envio de alertas por e-mail;

b) Pretende desenvolver uma aplicação cliente / dashboard dinâmico no Node-RED (recorrendo ao Node-RED Dashboard) que demonstre a aplicabilidade dessa integração através da receção de dados via HTTP POST e exibição de métricas em tempo real através de manómetros (gauges) e gráficos.

3 Objetivos

Os objetivos gerais do desenvolvimento deste trabalho são os seguintes:

- Consolidar conceitos de Integração de Sistemas de Informação usando serviços web e arquiteturas orientadas a eventos;
- Desenhar arquiteturas de integração de sistemas, recorrendo a comunicação inter-sistemas via HTTP/REST; Explorar ferramentas de suporte ao processamento e orquestração de dados (KNIME Analytics Platform) e Pentaho Data Integration (Kettle) ;
- Explorar novas tecnologias, frameworks ou paradigmas para visualização dinâmica e streaming de dados em tempo real (Node-RED e Node-RED Dashboard);
- Potenciar a experiência no desenvolvimento de soluções integradas;
- Assimilar conteúdos da Unidade Curricular.

Uma vez que a integração de sistemas moderna assenta em fluxos automatizados de dados e arquiteturas desacopladas, é fundamental explorar a recolha, tratamento, notificação e apresentação gráfica de dados de forma contínua. Desta forma, neste trabalho foi:

- Desenvolvida uma pipeline de Ingestão e Tratamento de Dados no KNIME, e no Pentaho Data Integration (Kettle), capaz de agregar e calcular métricas do torneio de CS2 (Kills, Deaths e K/D Ratio para as equipas NaVi, G2 e Vitality);

- Implementado um mecanismo automatizado de envio de alertas por e-mail a partir do KNIME e no Pentaho Data Integration (Kettle) ;

- Configurada a comunicação via HTTP POST para envio de payloads em formato JSON do KNIME para o Node-RED;

- Desenvolvida uma aplicação de visualização interativa no Node-RED Dashboard, contendo manómetros (gauges) para os rácios K/D e gráficos para acompanhamento temporal; Testada e validada a integração fim-a-fim, garantindo a atualização instantânea das métricas do torneio;

- Identificada a divisão clara de responsabilidades na arquitetura: KNIME e Pentaho Data Integration (Kettle) para data processing e notificações, e Node-RED para streaming e apresentação de resultados.

4 Fontes de Dados

Para dar início ao desenvolvimento no KNIME e no Pentaho Data Integration (Kettle) estruturamos um cenário real de recolha de estatísticas de torneios de eSports (nomeadamente de Counter-Strike 2).

Conforme na preparação dos dados de entrada, foram utilizados os seguintes ficheiros base:

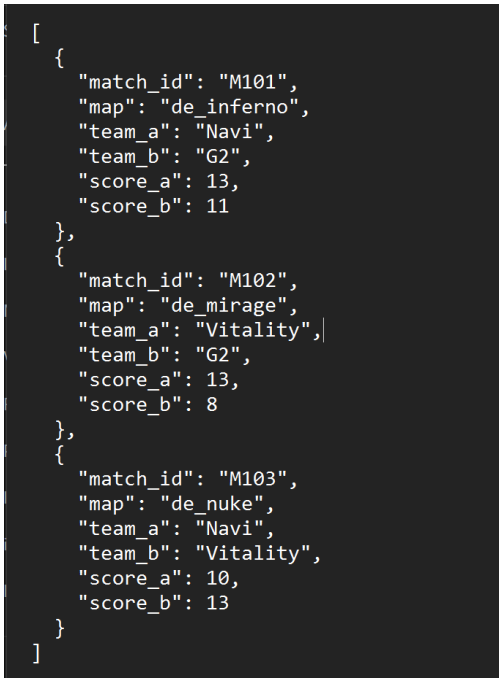
Jogadores (Ficheiro de Valores Separados por Vírgulas / CSV):

- Contém os registos estruturados referentes aos jogadores e equipas (como NaVi, G2 e Vitality), servindo de base para as operações relacionais, cruzamentos e agregações de desempenho.

Partidas.json (Ficheiro JSON):

- Armazena dados semi-estruturados relativos aos eventos e métricas globais das partidas, permitindo explorar a importação, serialização e o tratamento de formatos orientados a documentos.

Esta separação inicial de formatos (CSV e JSON) é fulcral para demonstrar a capacidade do sistema em lidar com múltiplas fontes de dados e realizar a respetiva unificação dentro da arquitetura de integração.



```
[
  {
    "match_id": "M101",
    "map": "de_inferno",
    "team_a": "Navi",
    "team_b": "G2",
    "score_a": 13,
    "score_b": 11
  },
  {
    "match_id": "M102",
    "map": "de_mirage",
    "team_a": "Vitality",
    "team_b": "G2",
    "score_a": 13,
    "score_b": 8
  },
  {
    "match_id": "M103",
    "map": "de_nuke",
    "team_a": "Navi",
    "team_b": "Vitality",
    "score_a": 10,
    "score_b": 13
  }
]
```

Fig 1 - Json Partidas

Player_ID	Nickname	Team	Kills	Deaths	Email	Country
1	s1mple	Navi	145	90	s1mple@navi.gg	UA
2	ZywOo	Vitality	150	85	zywoo@vitality.fr	FR
3	m0NESY	G2	138	92	m0nesy_g2@gmail.com	RU
4	b1t	Navi	110	95	b1t@navi.gg	UA
5	niko	G2	125	100	niko_cs@g2.com	BA
6	apex	Vitality	80	115	apex@vitality.fr	FR
7	INVALID_U	Test	0	0	bad_email_format	XX
8	s1mple	Navi	145	90	s1mple@navi.gg	UA

Fig 2 - CSV Jogadores

5 Processamento e Limpeza de Dados Estruturados (CSV)

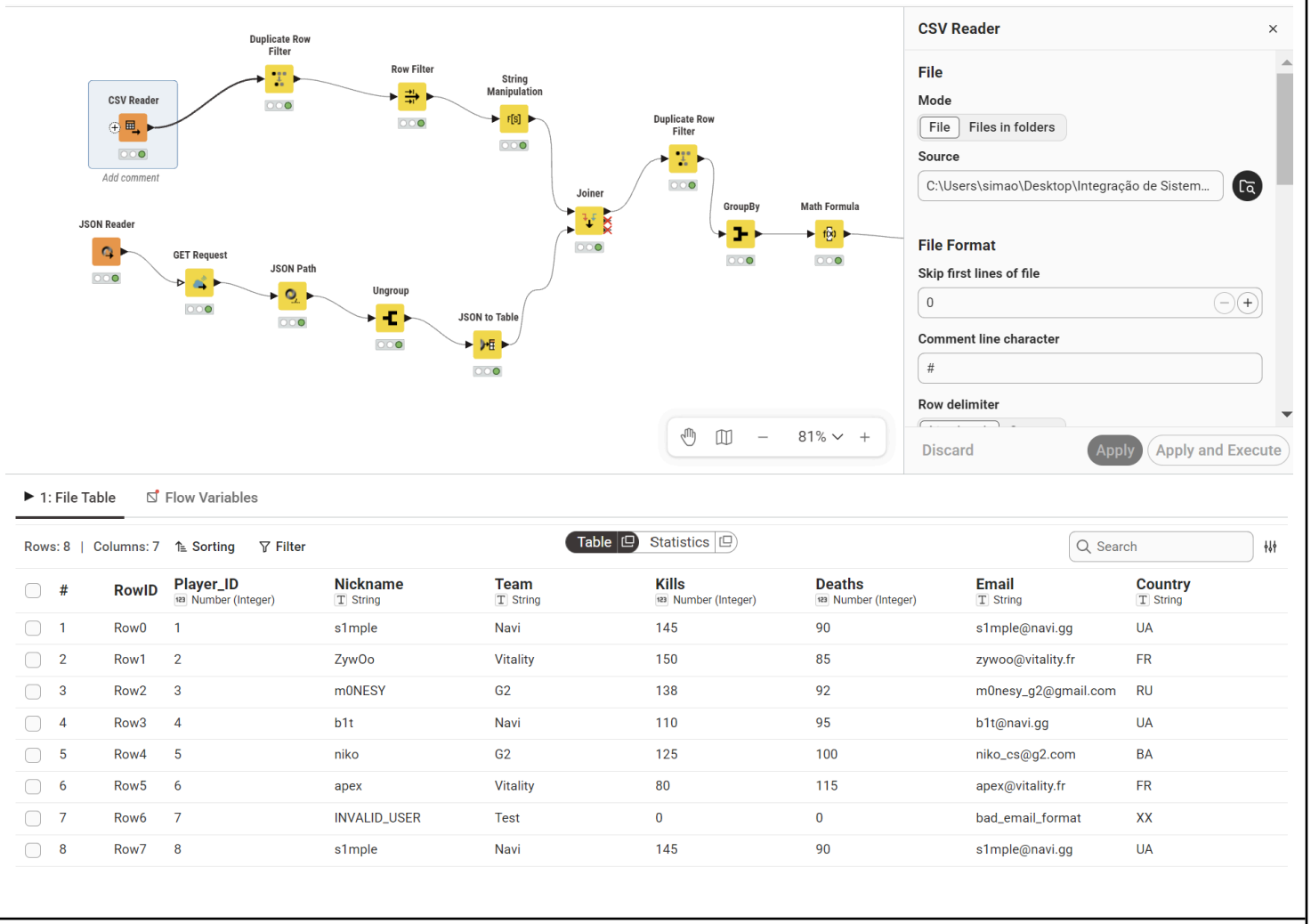
O primeiro ramo desenvolvido no KNIME Analytics Platform é responsável pela importação, higienização e normalização dos dados dos jogadores e equipas (provenientes do ficheiro Jogadores em formato CSV).

O fluxo implementado compreende as seguintes etapas sequenciais de tratamento de dados:

CSV Reader:

O nó CSV Reader efetua a leitura do repositório local, carregando a tabela inicial com 8 registos e 7 colunas (Player_ID, Nickname, Team, Kills, Deaths, Email e Country).

Nesta fase inicial, o conjunto de dados bruto continha inconsistências propositais para validação do sistema, tais como registos duplicados (ex: o jogador s1mple repetido no Row0 e Row7) e entradas inválidas (INVALID_USER com formato de e-mail incorreto).

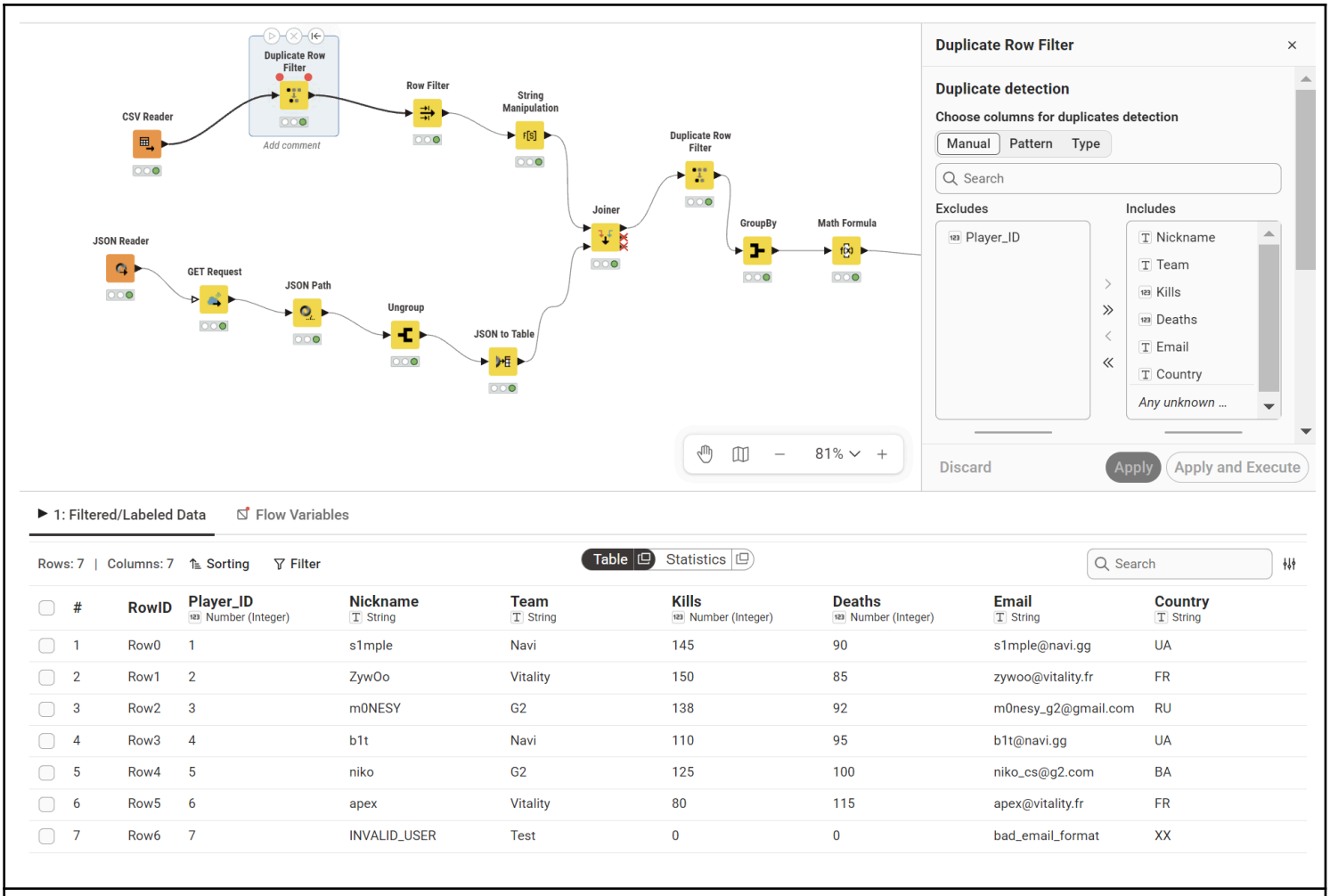


#	RowID	Player_ID (Number (Integer))	Nickname (String)	Team (String)	Kills (Number (Integer))	Deaths (Number (Integer))	Email (String)	Country (String)
1	Row0	1	s1mple	Navi	145	90	s1mple@navi.gg	UA
2	Row1	2	ZywOo	Vitality	150	85	zywoo@vitality.fr	FR
3	Row2	3	m0NESY	G2	138	92	m0nesy_g2@gmail.com	RU
4	Row3	4	b1t	Navi	110	95	b1t@navi.gg	UA
5	Row4	5	niko	G2	125	100	niko_cs@g2.com	BA
6	Row5	6	apex	Vitality	80	115	apex@vitality.fr	FR
7	Row6	7	INVALID_USER	Test	0	0	bad_email_format	XX
8	Row7	8	s1mple	Navi	145	90	s1mple@navi.gg	UA

Fig 3 - CSV Reader

Duplicate Row Filter:

Para assegurar a unicidade dos dados dos atletas e evitar distorções nas agregações estatísticas, utilizou-se o nó Duplicate Row Filter configurado com base nos identificadores e atributos dos jogadores, filtrando com sucesso o registo redundante e reduzindo a tabela para 7 linhas válidas.



#	RowID	Player_ID (Number (Integer))	Nickname (String)	Team (String)	Kills (Number (Integer))	Deaths (Number (Integer))	Email (String)	Country (String)
1	Row0	1	s1mple	Navi	145	90	s1mple@navi.gg	UA
2	Row1	2	ZywOo	Vitality	150	85	zywoo@vitality.fr	FR
3	Row2	3	m0NESY	G2	138	92	m0nesy_g2@gmail.com	RU
4	Row3	4	b1t	Navi	110	95	b1t@navi.gg	UA
5	Row4	5	niko	G2	125	100	niko_cs@g2.com	BA
6	Row5	6	apex	Vitality	80	115	apex@vitality.fr	FR
7	Row6	7	INVALID_USER	Test	0	0	bad_email_format	XX

Fig 4 - Duplicate Row Filter

Row Filter:

Através do nó Row Filter, aplicou-se uma regra de exclusão direcionada à coluna Email, eliminando registos que continham strings de erro (como bad_email_format associadas a utilizadores de teste/inválidos). Após esta filtragem, a tabela ficou limpa com um total de 6 registos consistentes correspondentes aos jogadores ativos das equipas NaVi, G2 e Vitality.

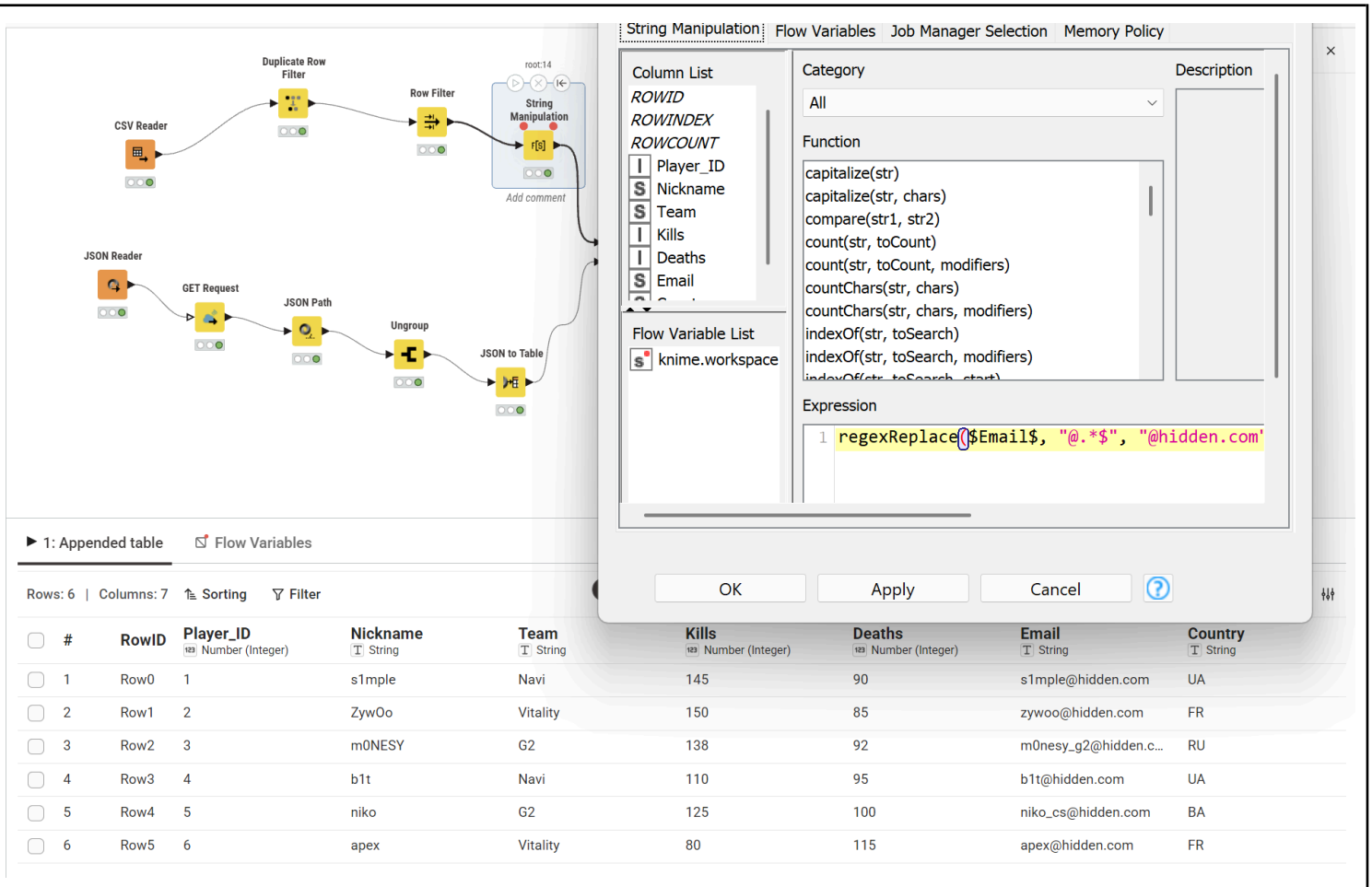


Fig 5 - Row Filter

String Manipulation:

Para cumprir os requisitos de anonimização e normalização de dados sensíveis ou textuais, recorreu-se ao nó String Manipulation.

Neste nó, implementou-se uma Expressão Regular (Regex) utilizando a função regexReplace($Email$, "@.*$", "@hidden.com"). Esta operação substitui o domínio de todos os e-mails dos jogadores por um domínio genérico (@hidden.com), garantindo a anonimização e validação de padrões de texto complexos dentro da pipeline.



#	RowID	Player_ID (Number (Integer))	Nickname (String)	Team (String)	Kills (Number (Integer))	Deaths (Number (Integer))	Email (String)	Country (String)
1	Row0	1	s1mple	Navi	145	90	s1mple@hidden.com	UA
2	Row1	2	ZywOo	Vitality	150	85	zywoo@hidden.com	FR
3	Row2	3	m0NESY	G2	138	92	m0nesy_g2@hidden.c...	RU
4	Row3	4	b1t	Navi	110	95	b1t@hidden.com	UA
5	Row4	5	niko	G2	125	100	niko_cs@hidden.com	BA
6	Row5	6	apex	Vitality	80	115	apex@hidden.com	FR

Fig 6 - String Manipulation

6 Ingestão de Dados JSON e Consumo de API REST (HTTP GET)

JSON Reader:

O nó JSON Reader importa o repositório local Partidas.json, contendo a estrutura de documentos com informações detalhadas sobre as partidas do torneio (identificadores de partidas, mapas e pontuações).

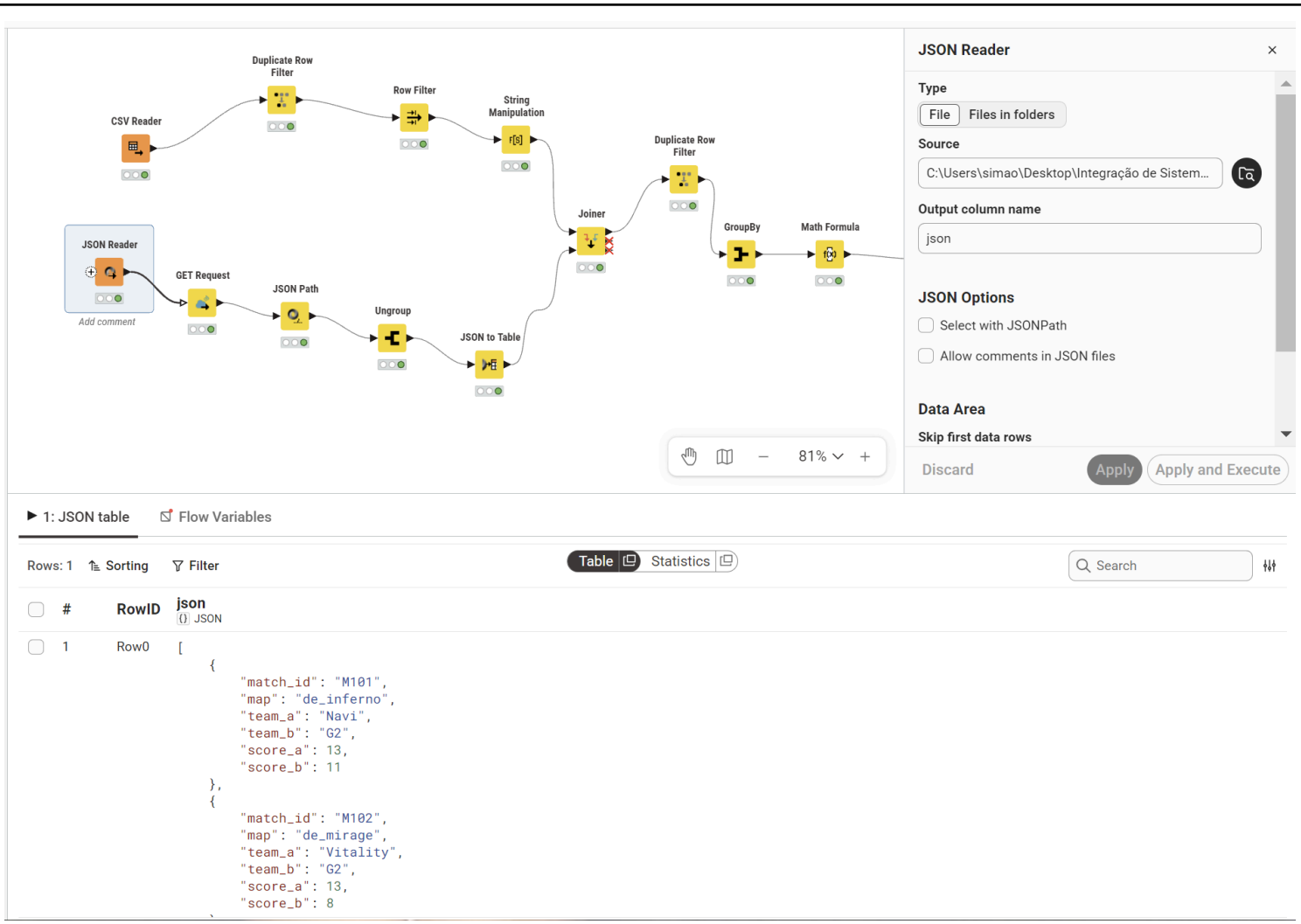


Fig 7 - JSON Reader

GET Request:

Para simular uma arquitetura orientada a serviços, o nó GET Request efetua um pedido HTTP GET ao endpoint disponibilizado localmente (http://127.0.0.1:1880/Partidas).

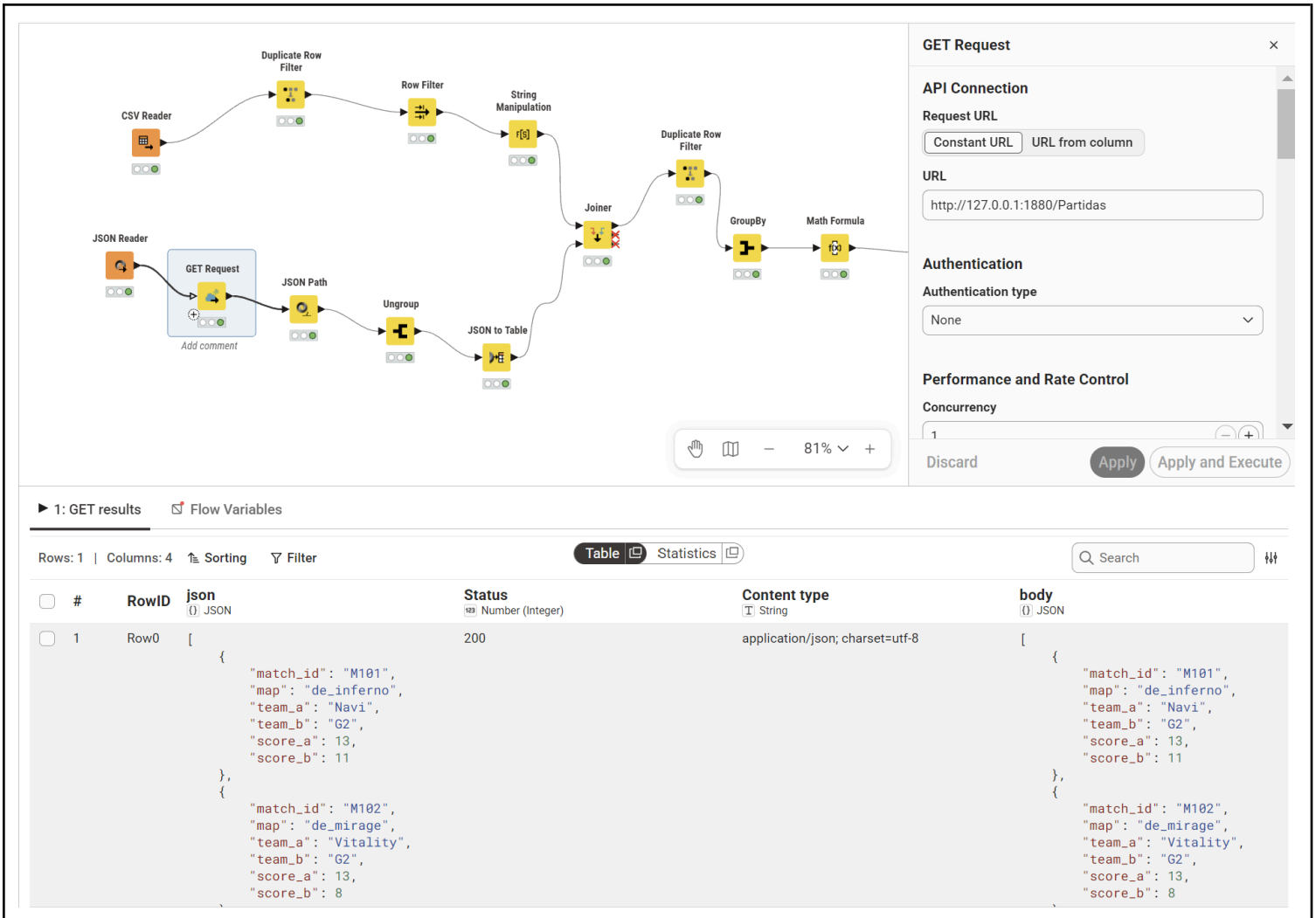


Fig 8 - GET Request

A resposta obtida retorna um código de estado 200 (OK) com o Content-Type application/json; charset=utf-8, preenchendo a coluna body com os dados atualizados das partidas em tempo real.

JSON Path e Ungroup:

Como o payload recebido possui uma estrutura em árvore/array, utilizou-se o nó JSON Path para mapear e extrair os atributos chave através de expressões direcionadas (ex: $.[*].match_id, $.[*].map, $.[*].team_a, $.[*].team_b, $.[*].score_a e $.[*].score_b).

De seguida, o nó Ungroup expande os arrays colecionados numa tabela relacional plana, transformando as listas de elementos em linhas individuais correspondentes a cada partida

jogada (como os confrontos nos mapas de_inferno, de_mirage e de_nuke entre NaVi, G2 e Vitality).

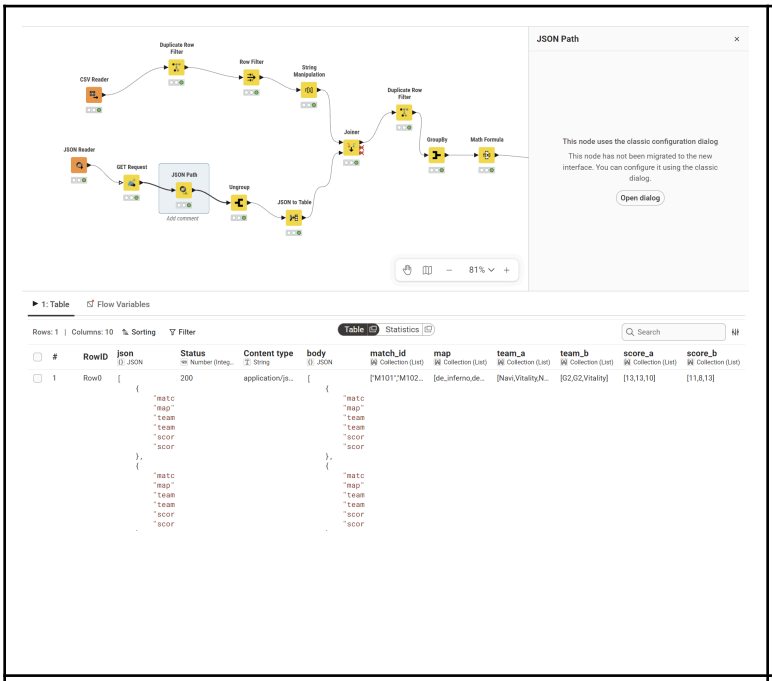


Fig 9 - JSON Path

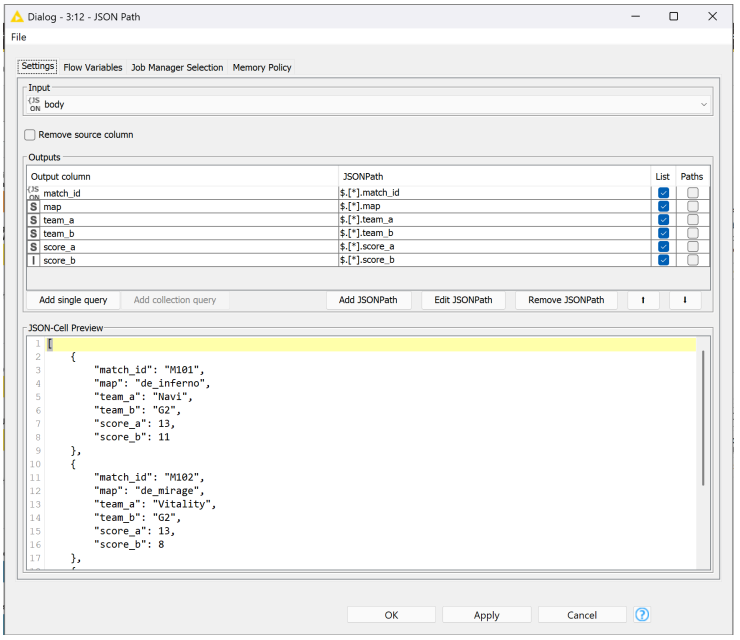


Fig 10 - JSON Path Config

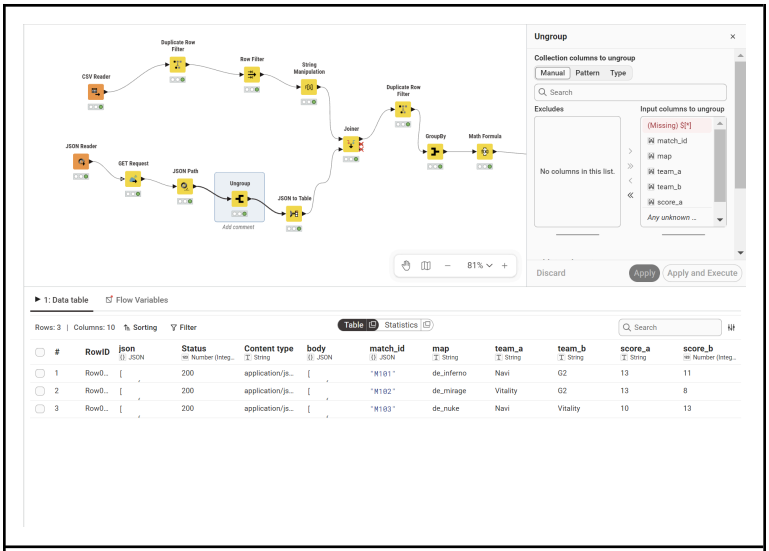


Fig 11 - Ungroup

JSON to Table

O resultado desta operação gera colunas distintas e tipificadas para cada parâmetro do torneio (como match_id, map, team_a, team_b, score_a e score_b), removendo a coluna de origem em JSON e disponibilizando os dados prontos para a etapa seguinte de cruzamento relacional (Joiner).

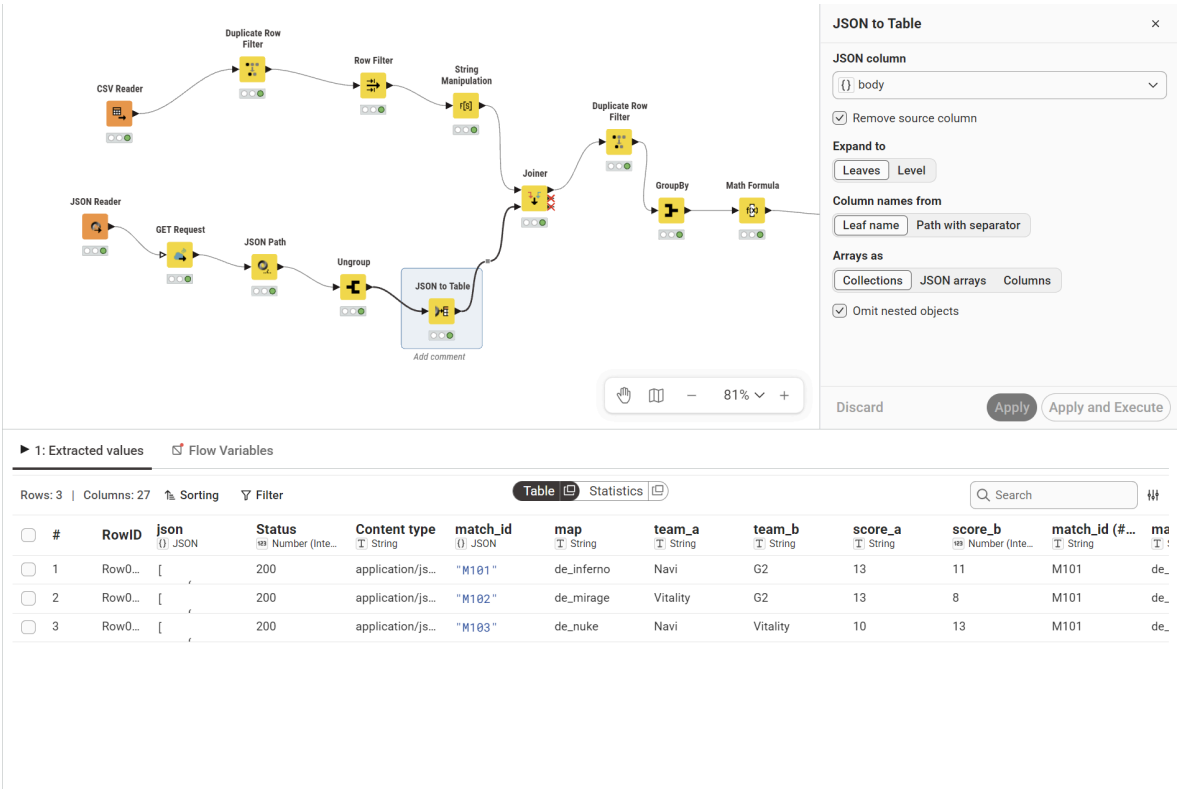


Fig 12 - JSON to Table

7 Cruzamento de Dados Relacionais

Joiner

Com os dois ramos processados — o tratamento tabular dos dados de jogadores via CSV e a ingestão das partidas dinâmicas via API REST/JSON —, procedeu-se à integração e unificação da informação através do nó Joiner.

Configuração dos Critérios de Correspondência (Matching Criteria):

- O nó estabelece a junção relacional cruzando a coluna Team (proveniente da tabela de jogadores à esquerda) com a coluna team_a/team_b.

- Esta operação permite associar o desempenho individual de cada atleta e equipa diretamente ao contexto das partidas disputadas no torneio.

Resultado do Cruzamento (Join result):

- O resultado da junção gera uma tabela consolidada e enriquecida (com 12 linhas e 34 colunas na vista resultante), onde cada registo cruza os atributos do jogador (como Nickname, Kills, Deaths e o e-mail anonimizado) com os metadados globais da respetiva partida (como o identificador do jogo, mapa disputado e pontuações).

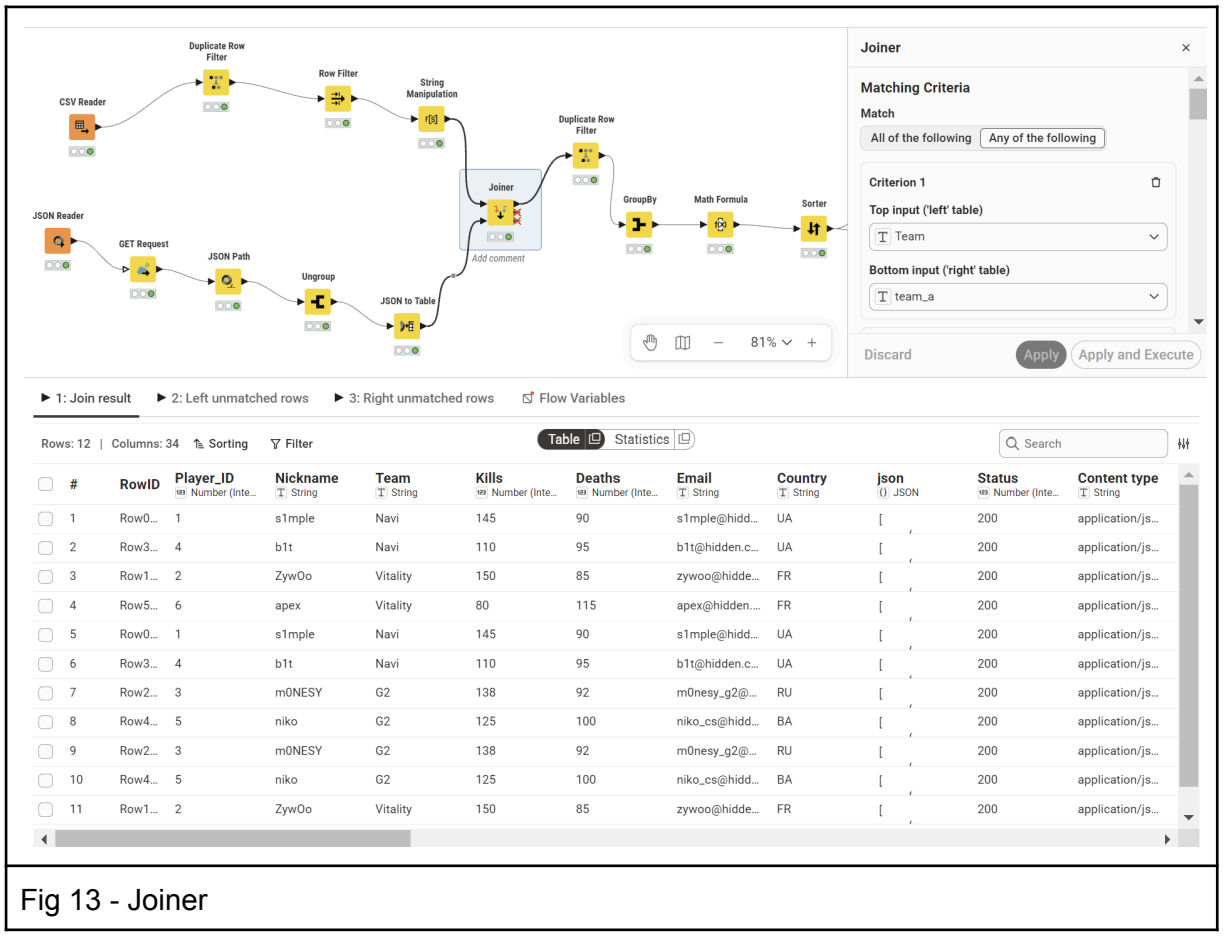


Fig 13 - Joiner

8 Desduplicação Pós-Junção

Duplicate Row Filter

Devido ao produto cartesiano gerado na junção entre a tabela de jogadores e as múltiplas partidas no nó Joiner, verificou-se a duplicação dos dados individuais dos atletas no fluxo resultante.

Para garantir a coerência das métricas estatísticas antes do cálculo dos indicadores agregados, aplicou-se um segundo nó Duplicate Row Filter:

Configuração da Deteção de Duplicados:

- A desduplicação foi configurada com foco na coluna Player_ID na secção de inclusão (Includes), isolando cada atleta de forma única.

Garantia de Integridade dos Dados:

- Com esta filtragem, reduziram-se os registos resultantes de 12 para 6 linhas únicas e consistentes, correspondentes exatamente aos atletas das equipas analisadas (NaVi, Vitality e G2: s1mple, b1t, ZywOo, apex, m0NESY e niko).
- Este passo elimina qualquer sobreposição de métricas individuais antes do agrupamento por equipa.

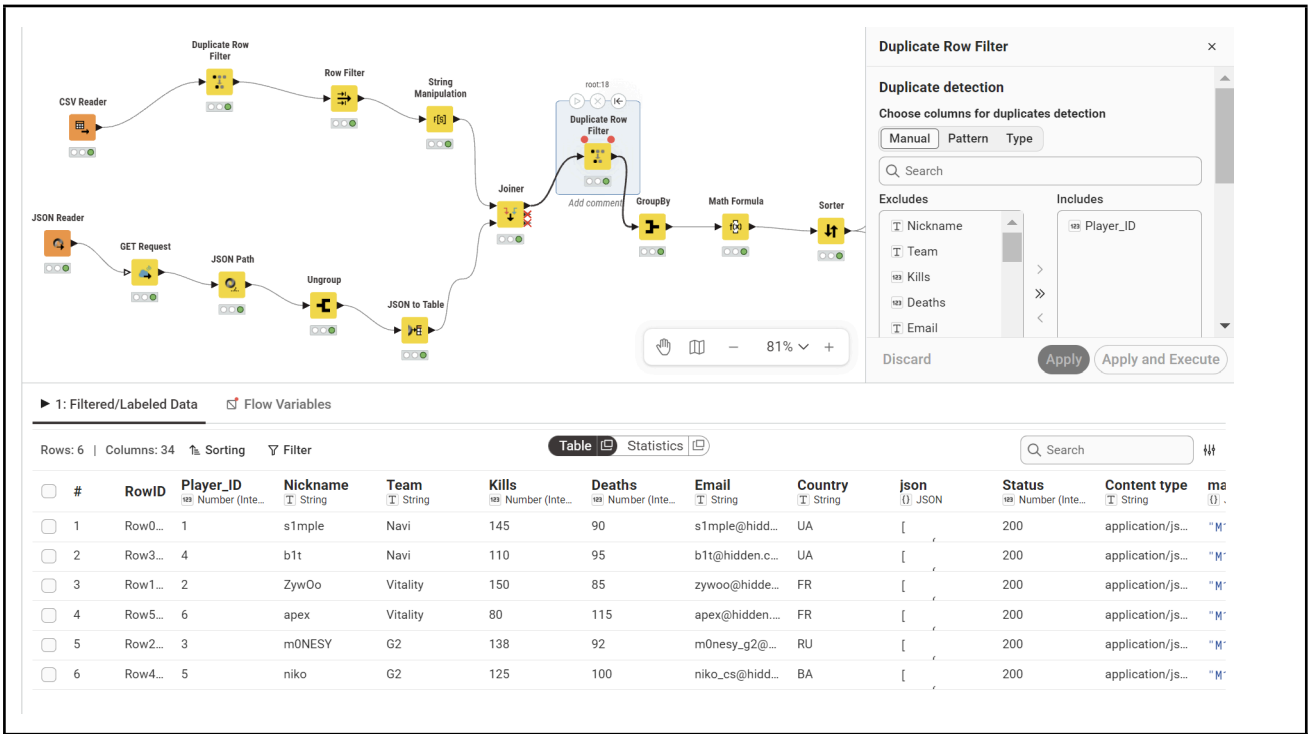


Fig 14 - Duplicate Row Filter

9 Agregação e Consolidação de Métricas por Equipa

GroupBy

Com a tabela limpa, cruzada e sem redundâncias, o passo seguinte consistiu em agregar as estatísticas individuais dos jogadores para o nível de equipa, permitindo calcular o desempenho global de cada organização no torneio (G2, NaVi e Vitality).

O nó GroupBy foi configurado para agrupar os registos com base na coluna Team.

Na agregação manual (Manual Aggregation), definiram-se as funções de cálculo para as métricas numéricas:

- Kills: aplicou-se a função de agregação Sum para totalizar as eliminações de todos os jogadores da respetiva equipa.
- Deaths: aplicou-se igualmente a função Sum para somar o total de mortes registadas por equipa.

Resultado da Tabela Agregada (Group table):

- A execução da operação resultou numa tabela resumida com exatamente 3 linhas (uma por cada equipa participante: G2, NaVi e Vitality) e as colunas calculadas Sum(Kills) e Sum(Deaths).

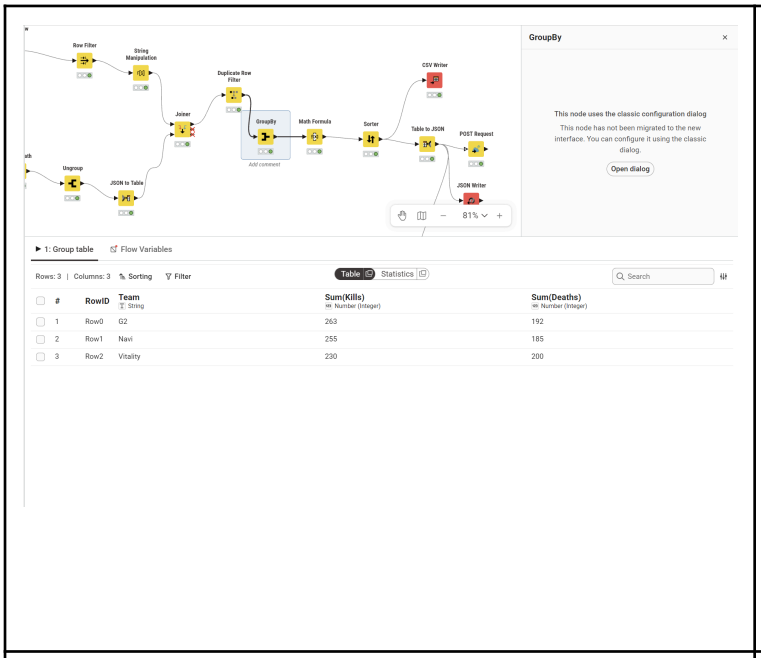


Fig 15 - GroupBy

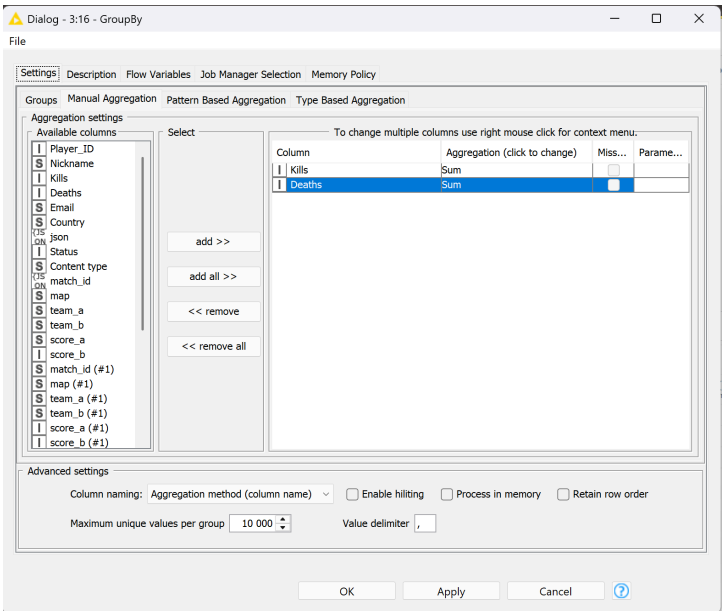


Fig 16 - GroupBy config

Cálculo do Indicador de Desempenho

Math Formula

Após a agregação dos totais de eliminações e mortes por equipa, tornou-se necessário calcular a métrica chave de desempenho nos eSports: o rácio de Kills/Deaths (K/D Ratio).

Expressão Matemática e Configuração:

Utilizou-se o nó Math Formula, onde se definiu a expressão matemática:

- K/D Ratio = Sum(Kills) / Sum(Deaths)

A operação foi configurada para criar uma nova coluna chamada K/D Ratio do tipo de dados numérico de vírgula flutuante (Float).

Métricas Resultantes:

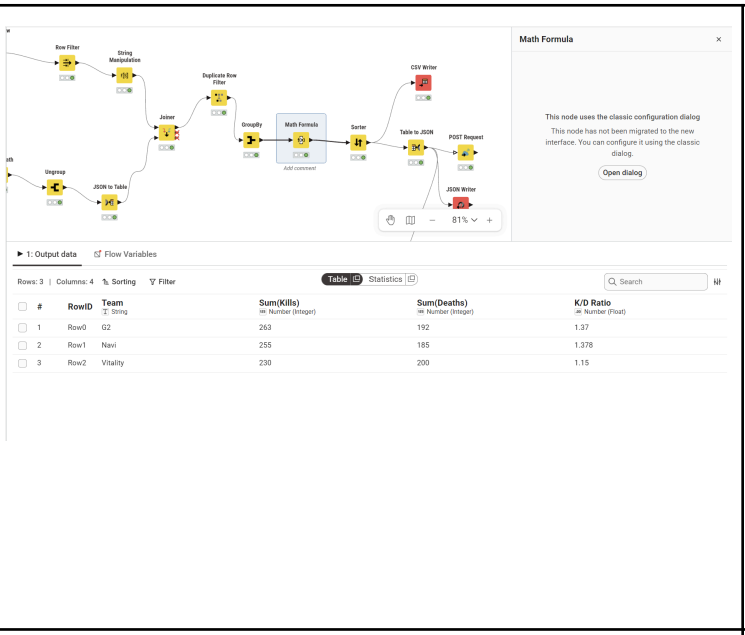


Fig 17 - Math Formula

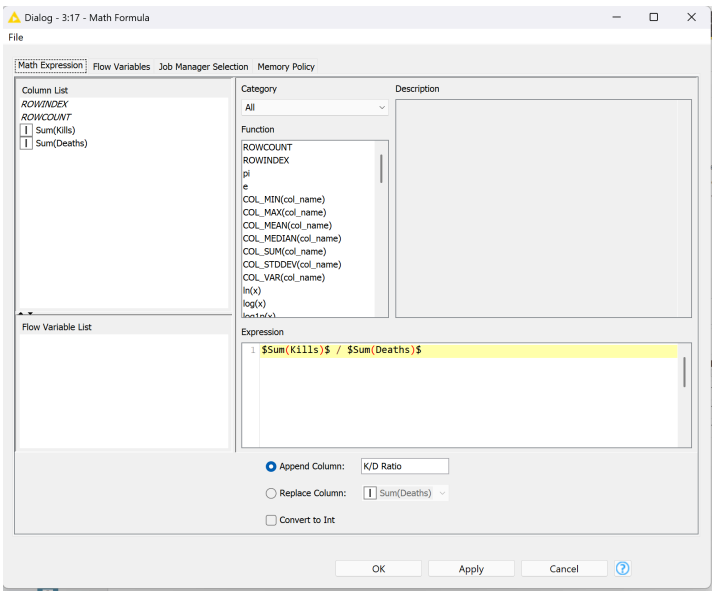


Fig 18 - Math Formula config

- NaVi: K/D Ratio 1.378 (255 Kills / 185 Deaths);
- G2: K/D Ratio 1.37 (263 Kills / 192 Deaths);
- Vitality: K/D Ratio 1.15 (230 Kills / 200 Deaths).

10 Ordenação de Desempenho por K/D Ratio

Sorter

Após o cálculo do indicador K/D Ratio para cada equipa, aplicou-se o nó Sorter com o objetivo de estruturar e hierarquizar os resultados de forma lógica.

Configuração da Ordenação:

- O critério de ordenação foi configurado com base na coluna K/D Ratio, selecionando o modo Decrescente (Descending).
- Desta forma, garantiu-se que a tabela final apresentasse as equipas ordenadas da melhor para a pior performance no torneio.

Tabela Ordenada Resultante:

Na tabela resultante (Sorted Table), a liderança do torneio ficou atribuída à NaVi com um rácio de 1.378, seguida de perto pela G2 em segundo lugar com 1.37, e pela Vitality na terceira posição com 1.15.

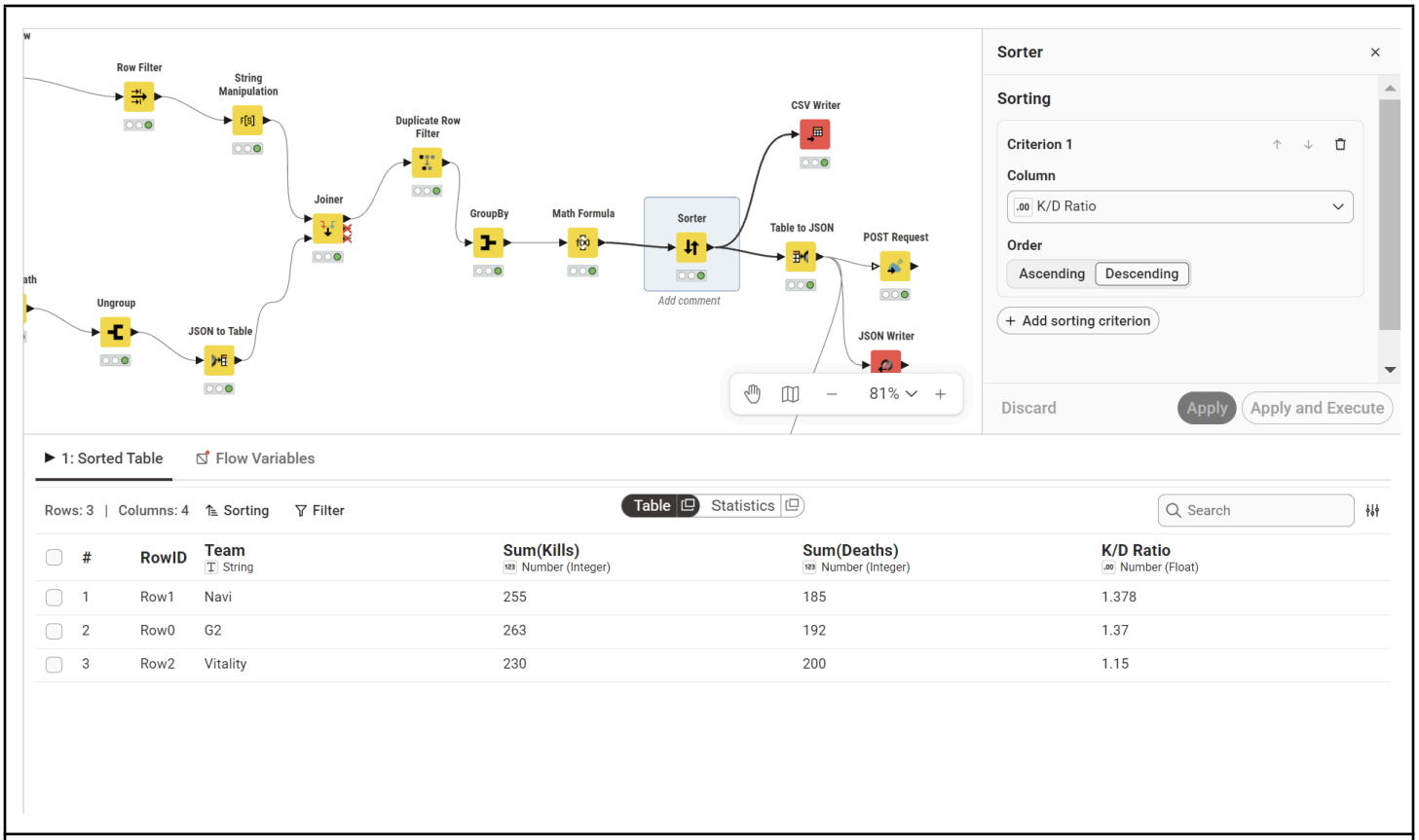


Fig 19 - Sorter

11 Serialização e Envio de Dados em Tempo Real

Table to JSON

Após o cálculo e a ordenação das métricas globais do torneio, o KNIME prepara e envia os resultados finais para a camada de visualização em tempo real (Node-RED):

Serialização dos Dados (Table to JSON):

- O nó Table to JSON é responsável por converter a estrutura relacional tabular (composta pelas colunas Team, Sum(Kills), Sum(Deaths) e K/D Ratio) num array de objetos em formato JSON.

Comunicação Inter-Sistemas via HTTP POST (POST Request):

- O nó POST Request estabelece a ponte de integração, submetendo o payload JSON diretamente para o endpoint REST configurado no Node-RED através do URL http://127.0.0.1:1880/SalvarDados

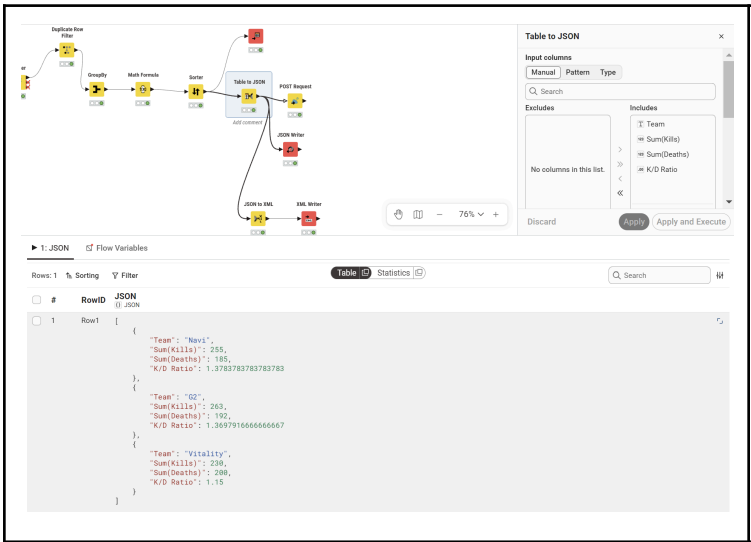


Fig 20 - Table to JSON

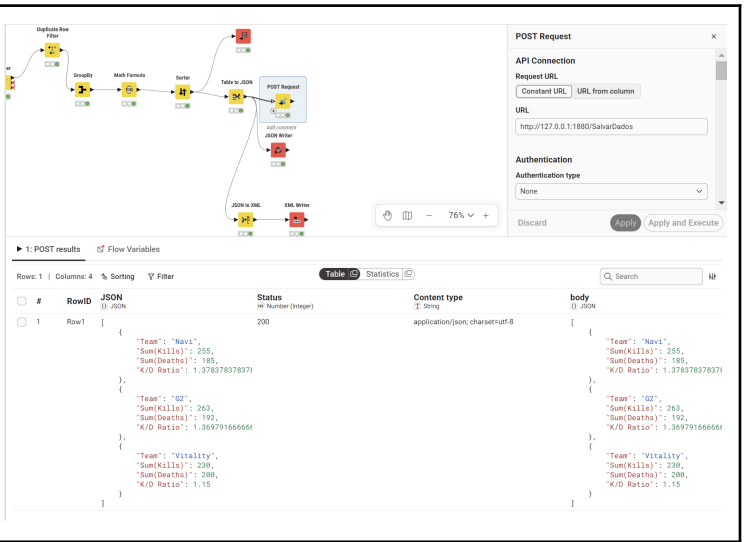


Fig 21 - POST Request

12 Persistência e Exportação Multiformato

(CSV, JSON e XML)

Para além do streaming em tempo real via HTTP POST para o Node-RED, a pipeline desenvolvida no KNIME garante a persistência dos resultados finais em múltiplos formatos de ficheiro no repositório local, assegurando a interoperabilidade com diferentes sistemas de suporte:

Exportação para CSV (CSV Writer):

- O nó CSV Writer exporta a tabela final de métricas diretamente para o sistema de ficheiros local.
- O ficheiro é configurado com vírgula (,) como delimitador de colunas, mantendo os cabeçalhos das colunas (Write column headers) e sobrescrevendo registos anteriores (Overwrite).

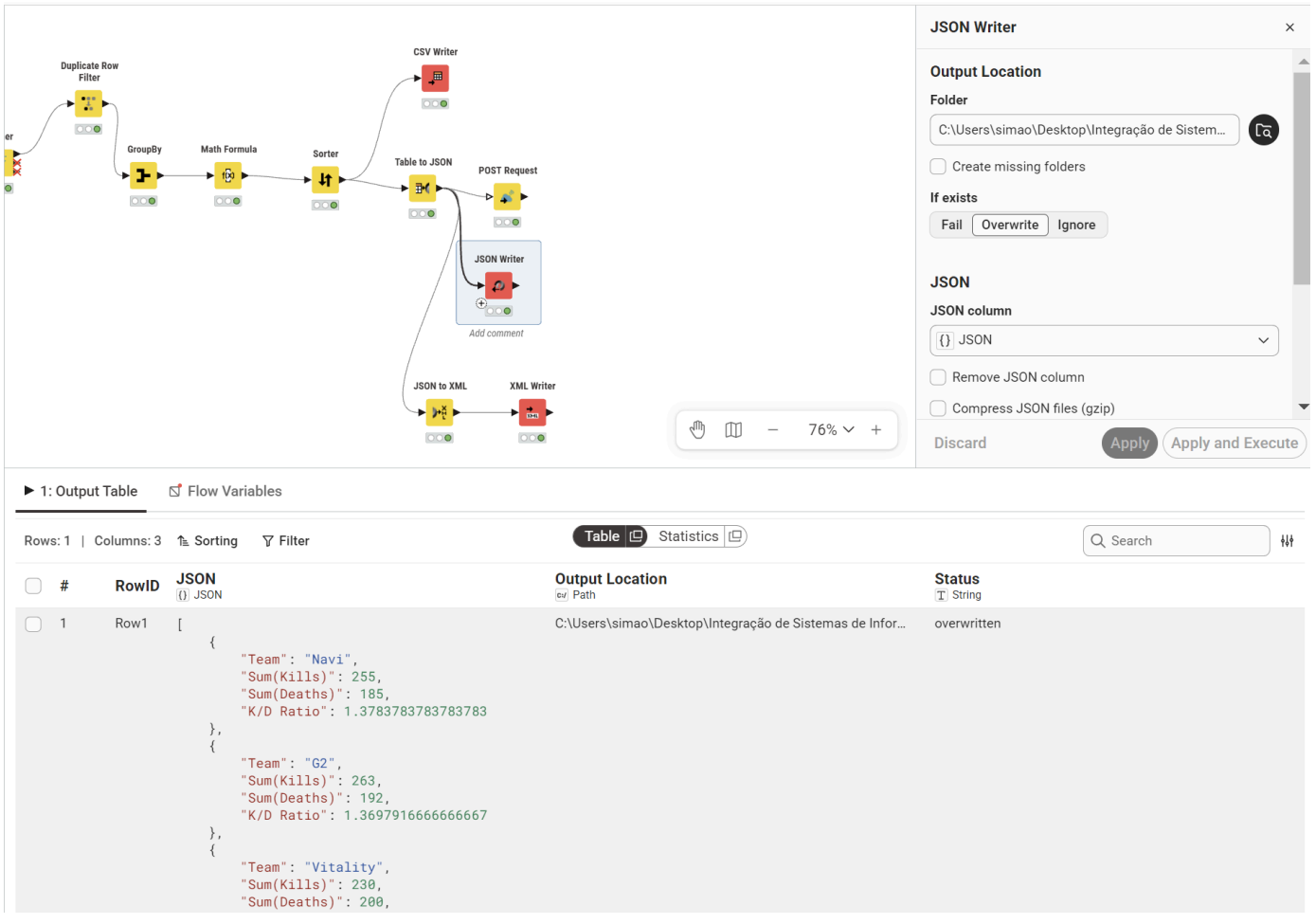


Fig 22 - CSV (CSV Writer)

Exportação para JSON (JSON Writer):

- A partir da estrutura gerada no nó Table to JSON, o nó JSON Writer grava o payload num ficheiro .json local.

- Como se observa na validação do output, o nó confirma o estado de execução como overwritten, armazenando o array estruturado com os dados das equipas (Navi, G2 e Vitality), as respetivas somas de Kills/Deaths e o K/D Ratio.

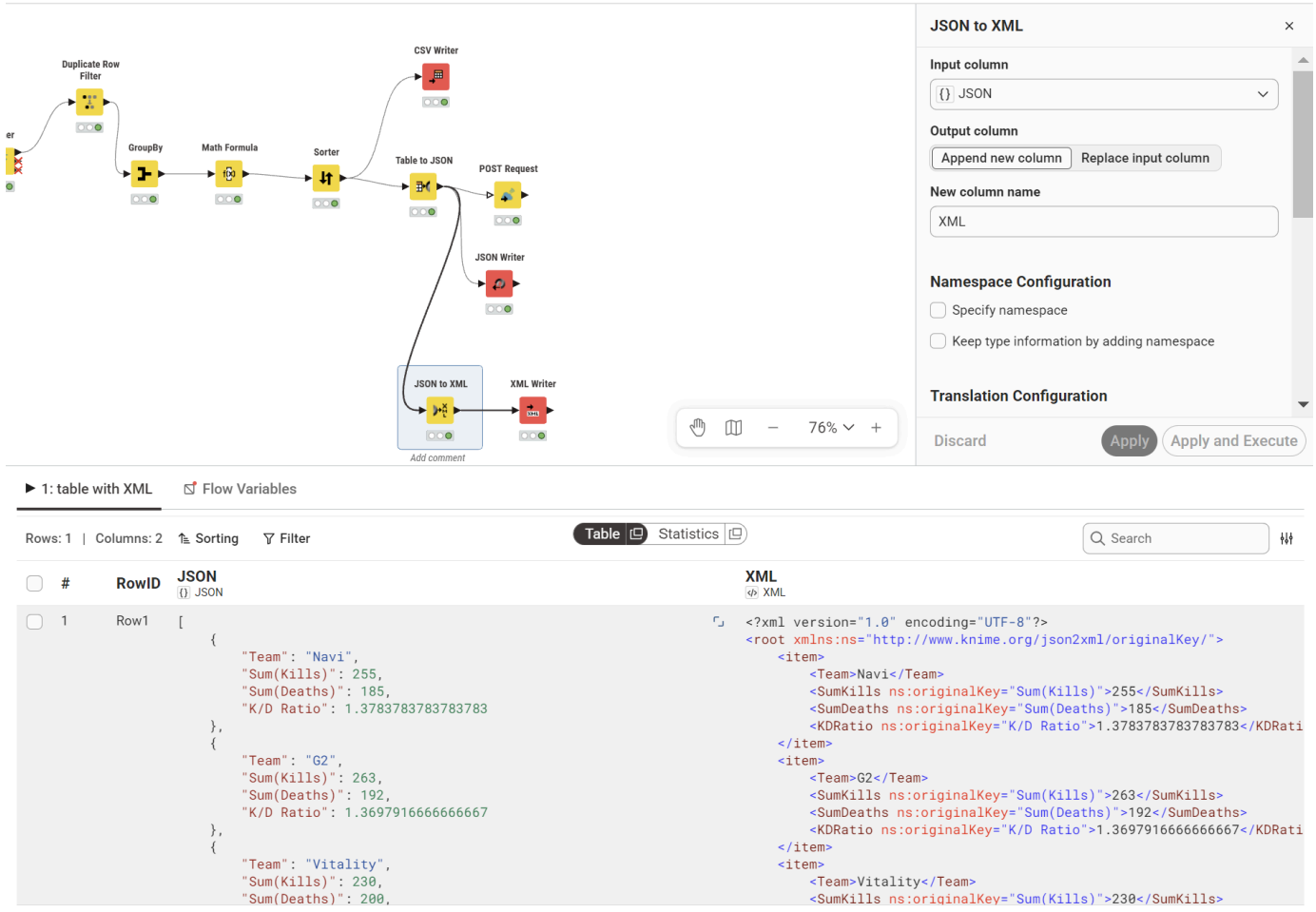


Fig 23 - JSON (JSON Writer)

Conversão e Exportação para XML (JSON to XML e XML Writer):

- Para responder a cenários de integração com sistemas legados ou orientados a XML, utilizou-se o nó JSON to XML, que converte a árvore JSON numa estrutura com declaração <?xml version="1.0" encoding="UTF-8"?>.

- Em seguida, o nó XML Writer escreve estes dados no repositório local, completando a persistência no formato XML.

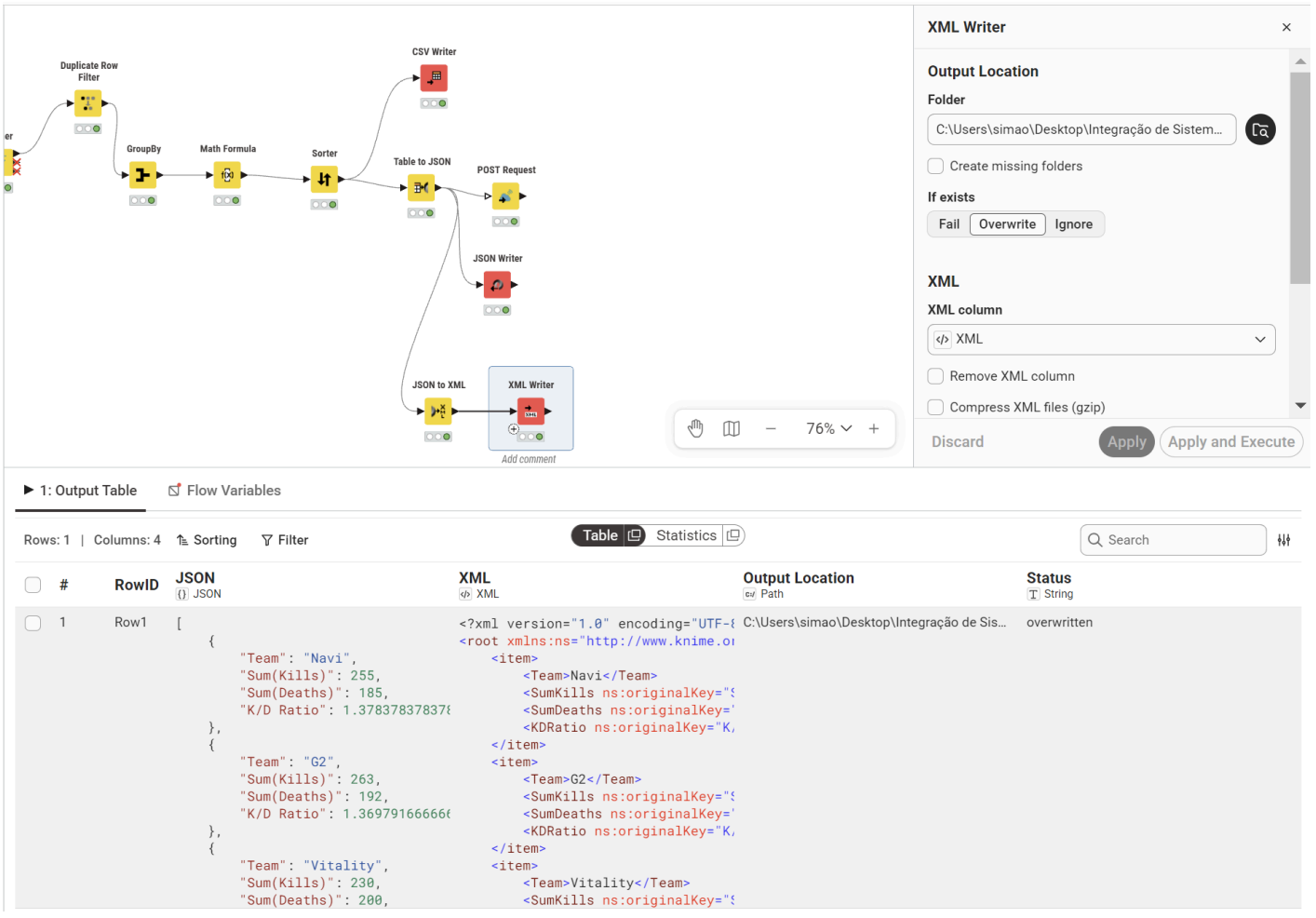


Fig 24 - XML (JSON to XML e XML Writer)

13 Implementação do Servidor REST e Visualização de Partidas

/Partidas

Para permitir que a arquitetura disponibilize dados das partidas em tempo real para o KNIME e, simultaneamente, os apresente na interface gráfica do Node-RED Dashboard, foi construído um fluxo dedicado com endpoints RESTful HTTP:

Recepção de Pedidos HTTP GET (http in):

- O nó [get] /Partidas é configurado para escutar requisições do tipo GET na rota /Partidas.
- Este nó atua como ponto de entrada para clientes externos (incluindo o nó GET Request do KNIME) que pretendem obter o estado atualizado do torneio.

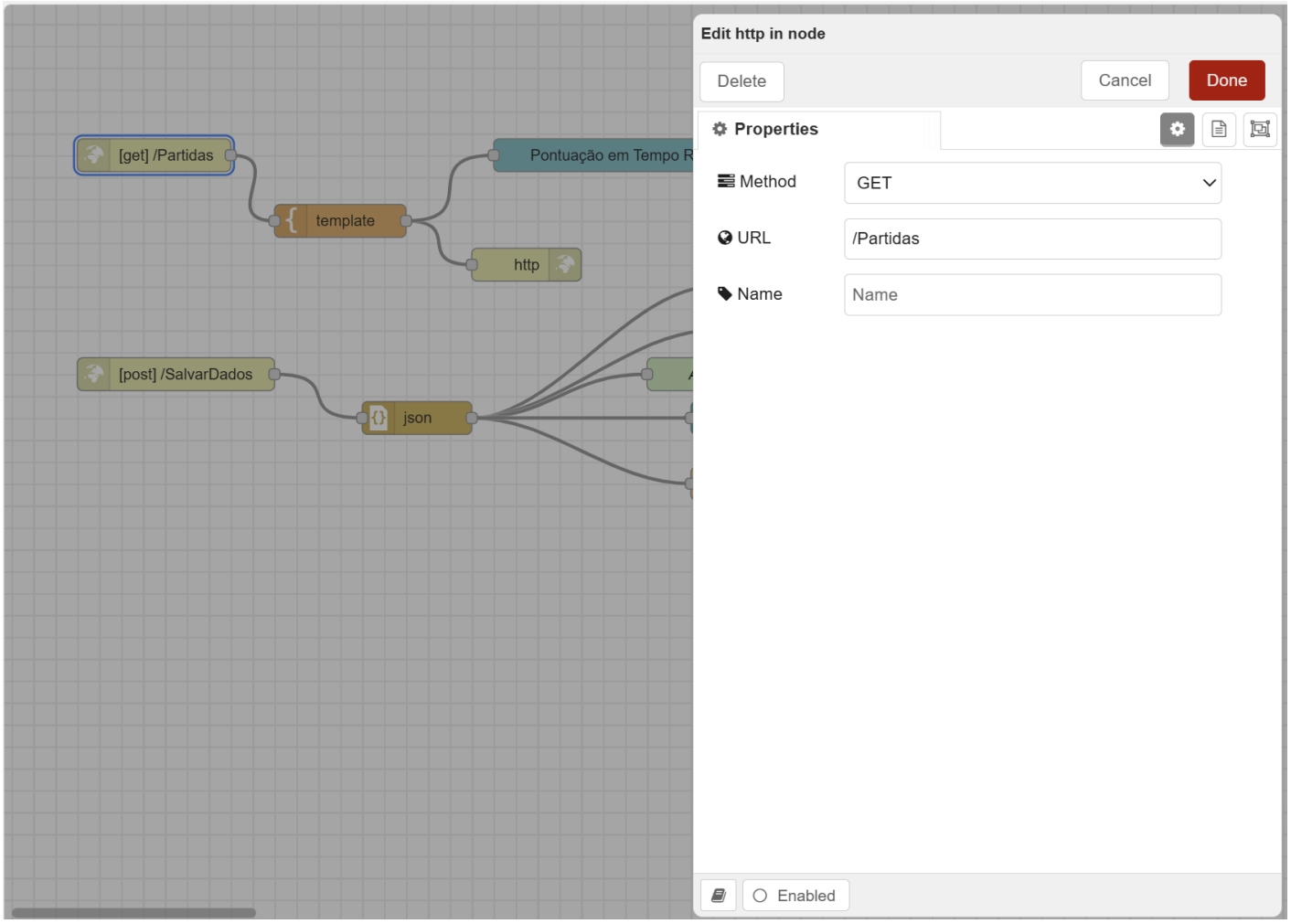


Fig 25 - HTTP GET (http in)

Definição e Estruturação do Payload (template):

- Interligado ao nó HTTP GET, o nó template é utilizado para definir a resposta em formato JSON.

- A propriedade msg.payload é preenchida com um array de objetos contendo os metadados das partidas disputadas (identificador da partida match_id, mapa map, equipas em confronto team_a/team_b e pontuações score_a/score_b). O nó está configurado para devolver a mensagem formatada como Parsed JSON.

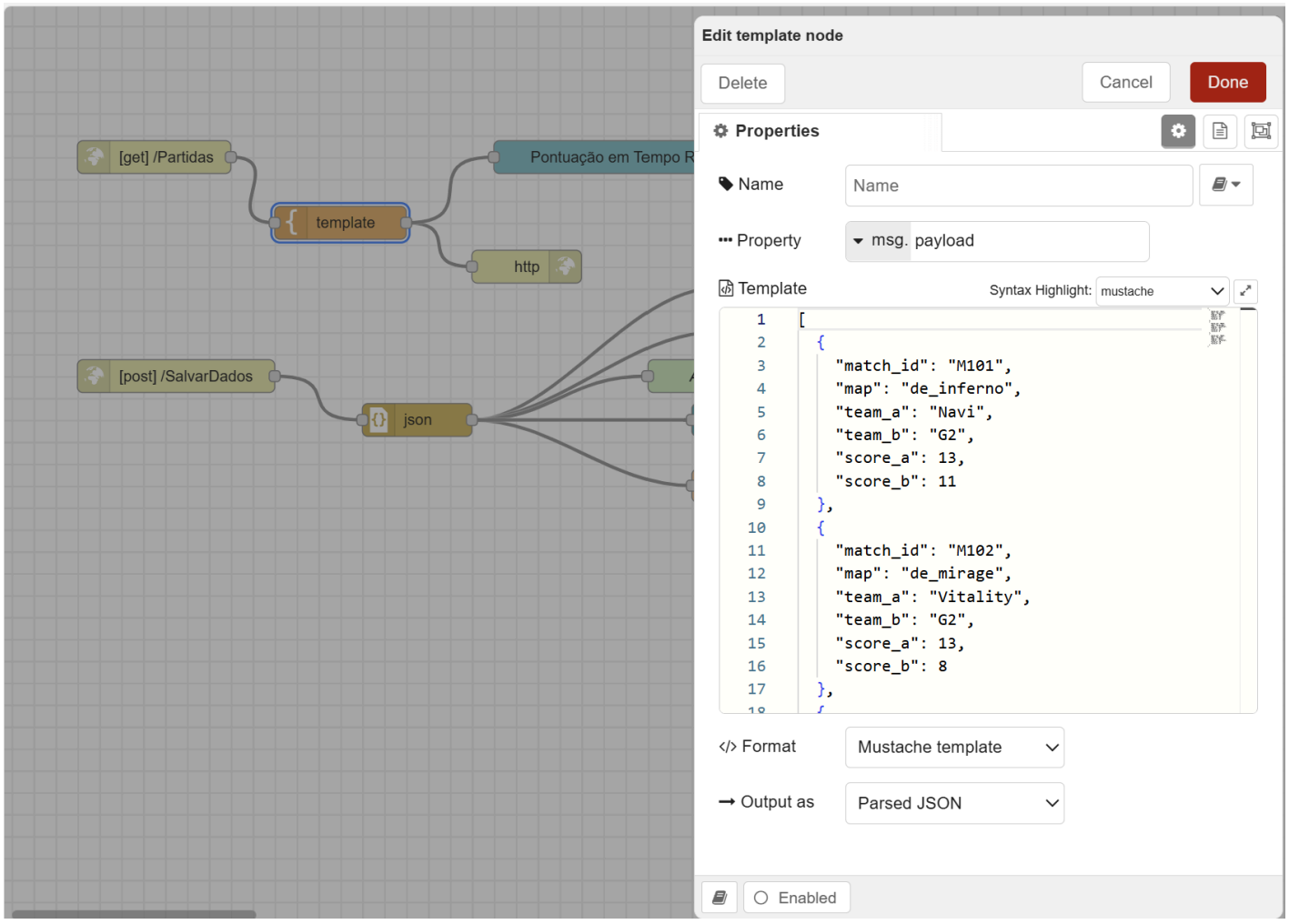


Fig 26 - Template

Resposta HTTP ao Cliente (http response):

- O nó http (http response) encerra o ciclo do pedido, devolvendo a mensagem contendo o payload JSON ao cliente com o código de estado retornado por msg.statusCode (HTTP 200 OK).

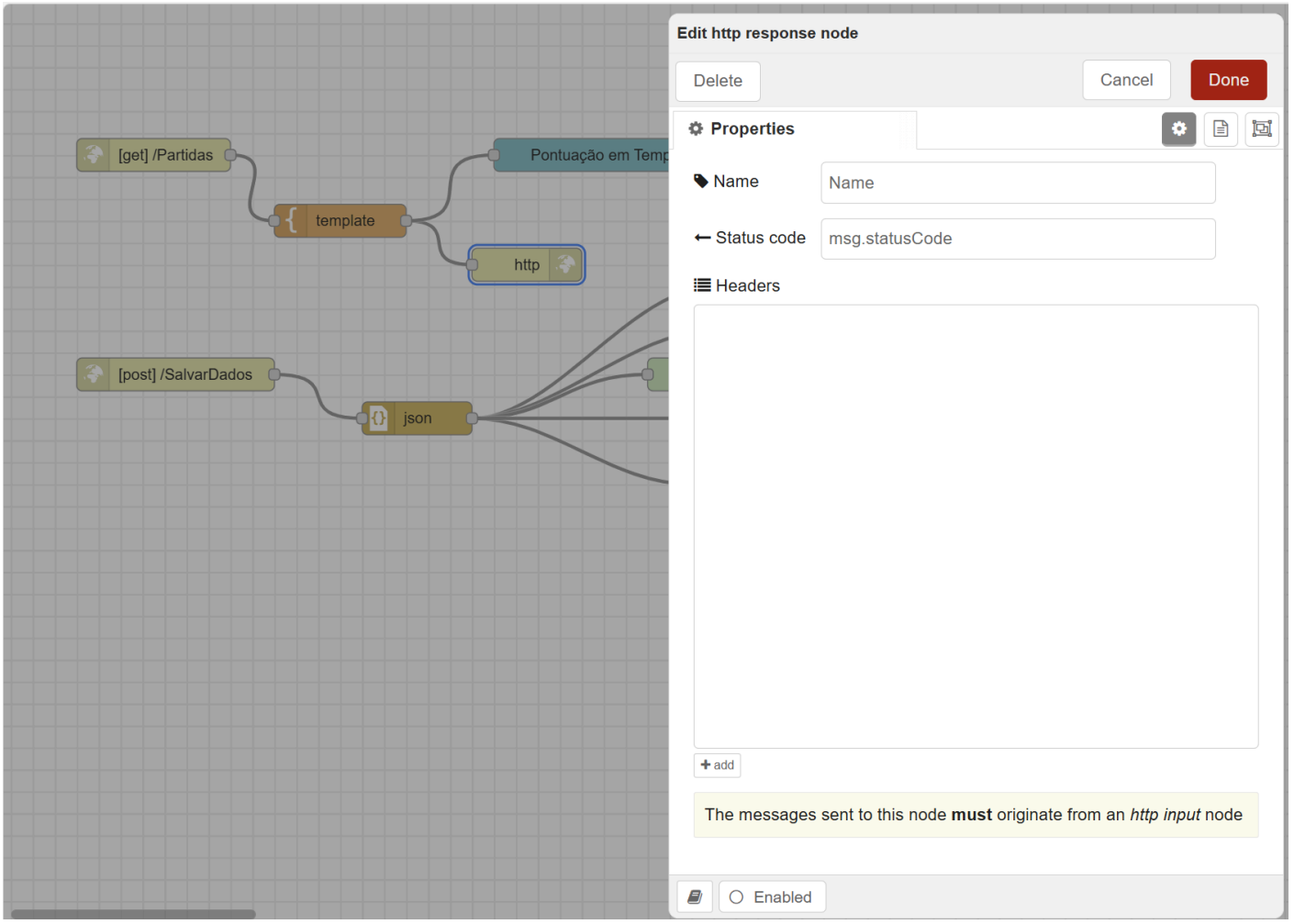


Fig 27 - http response

Visualização Gráfica em Tempo Real (chart - Pontuação em Tempo Real):

- Em paralelo, o payload gerado no nó template alimenta o nó de visualização Pontuação em Tempo Real (tipo Chart / Line chart).

- Este nó pertence ao grupo do dashboard [Monitorização CS2] e é responsável por plotar a evolução temporal dos pontos das equipas ao longo do tempo, permitindo o acompanhamento dinâmico dos marcadores do torneio.

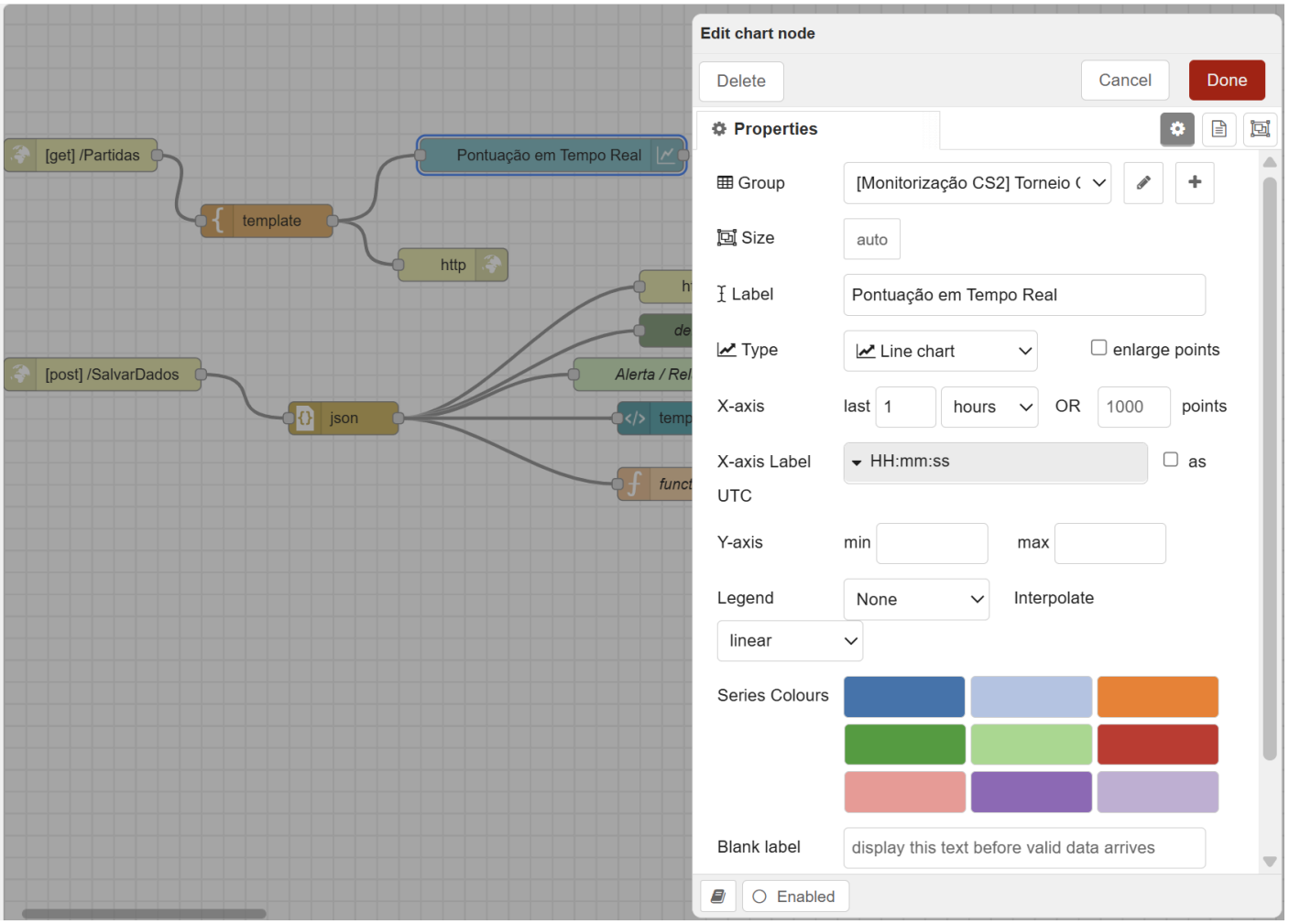


Fig 28 - Pontuação em Tempo Real

14 Endpoint de Ingestão de Dados e Deserialização

/SalvarDados

Para receber as métricas agregadas e calculadas no KNIME (como o K/D Ratio, total de Kills e Deaths por equipa) em tempo real, foi criado o endpoint REST de ingestão no Node-RED:

Recepção do Pedido HTTP POST (http in):

- O nó [post] /SalvarDados é configurado para escutar requisições do método POST no URL /SalvarDados.
- Este nó atua como a porta de entrada para o payload enviado pelo nó POST Request do KNIME após a conclusão do processamento de dados.

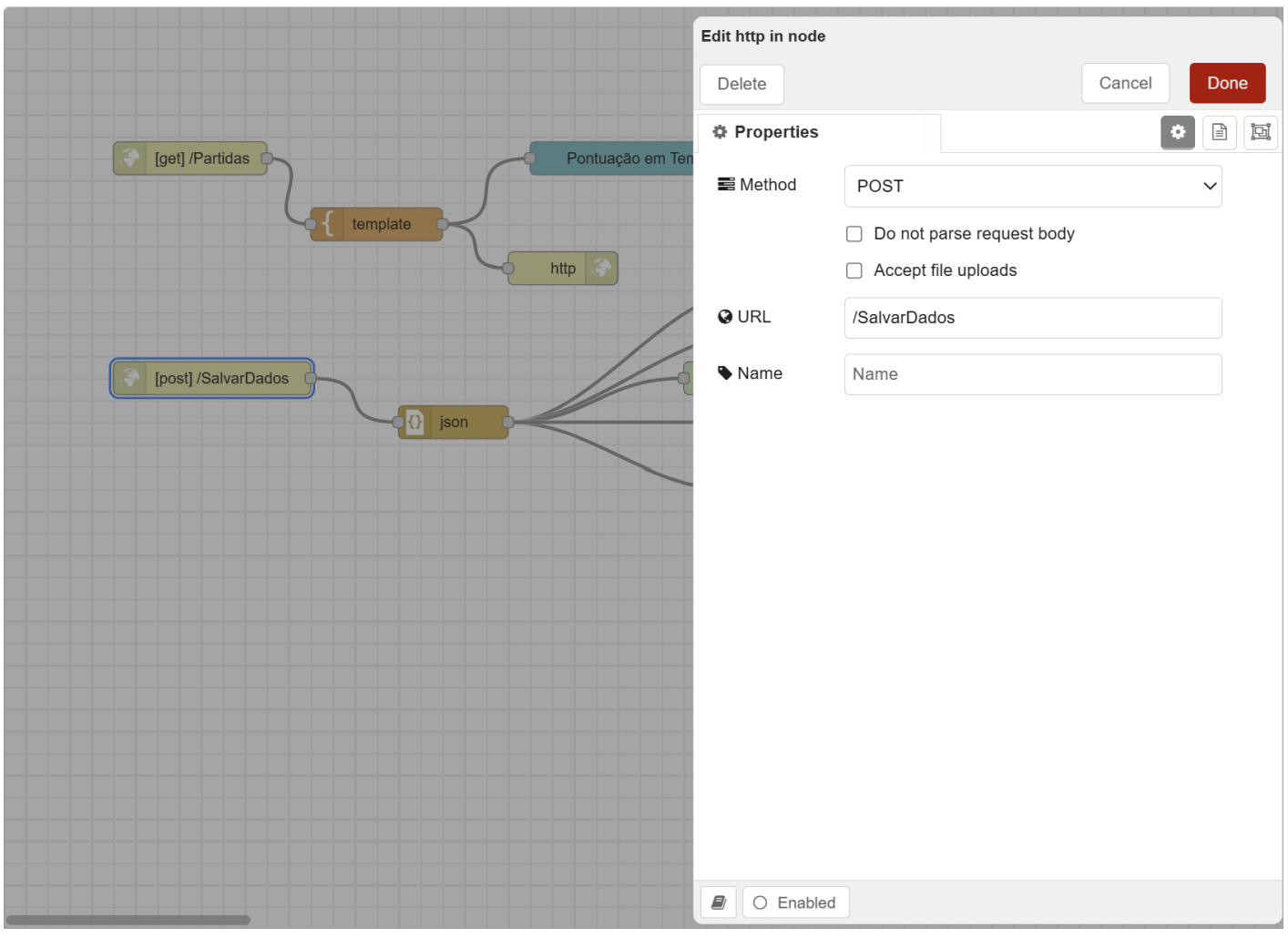


Fig 29 - HTTP POST

Deserialização e Conversão do Payload (json):

- O fluxo passa imediatamente pelo nó de transformação json.
- Configurado com a ação "Always convert to JavaScript Object" na propriedade msg.payload, este nó converte a string JSON recebida no corpo do pedido num objeto/array de objetos nativo do JavaScript. Esta conversão é fundamental para permitir que os nós subsequentes do Node-RED acedam diretamente às propriedades das equipas (Team, Sum(Kills), Sum(Deaths), K/D Ratio).

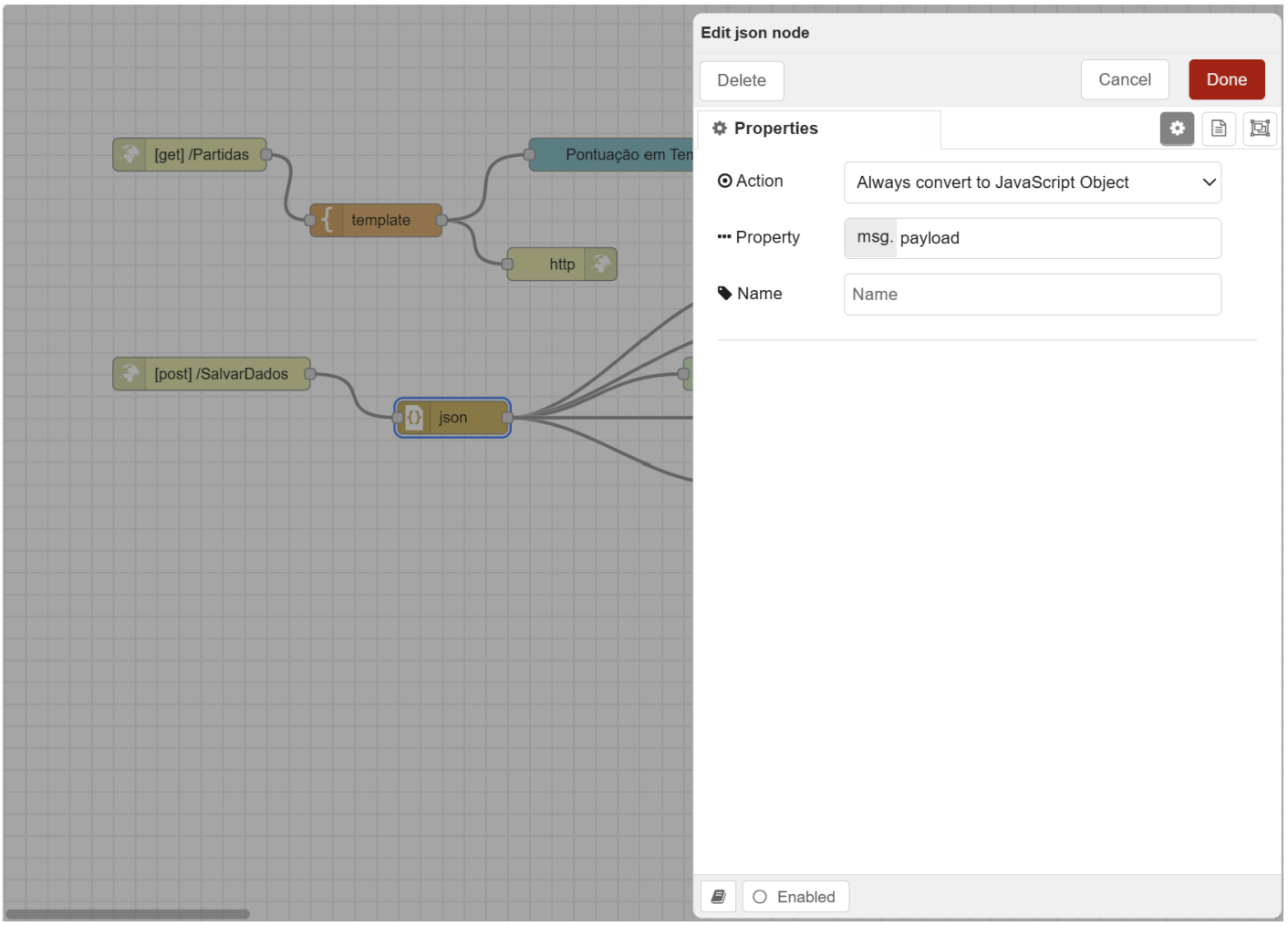


Fig 30 - Conversão json

Confirmação da Resposta HTTP (http response):

- O nó http (http response) fecha a transação HTTP devolvendo a resposta ao KNIME. Garante o envio do código de estado de sucesso (HTTP 200 OK), confirmando que a mensagem foi recebida e processada pela plataforma com êxito.

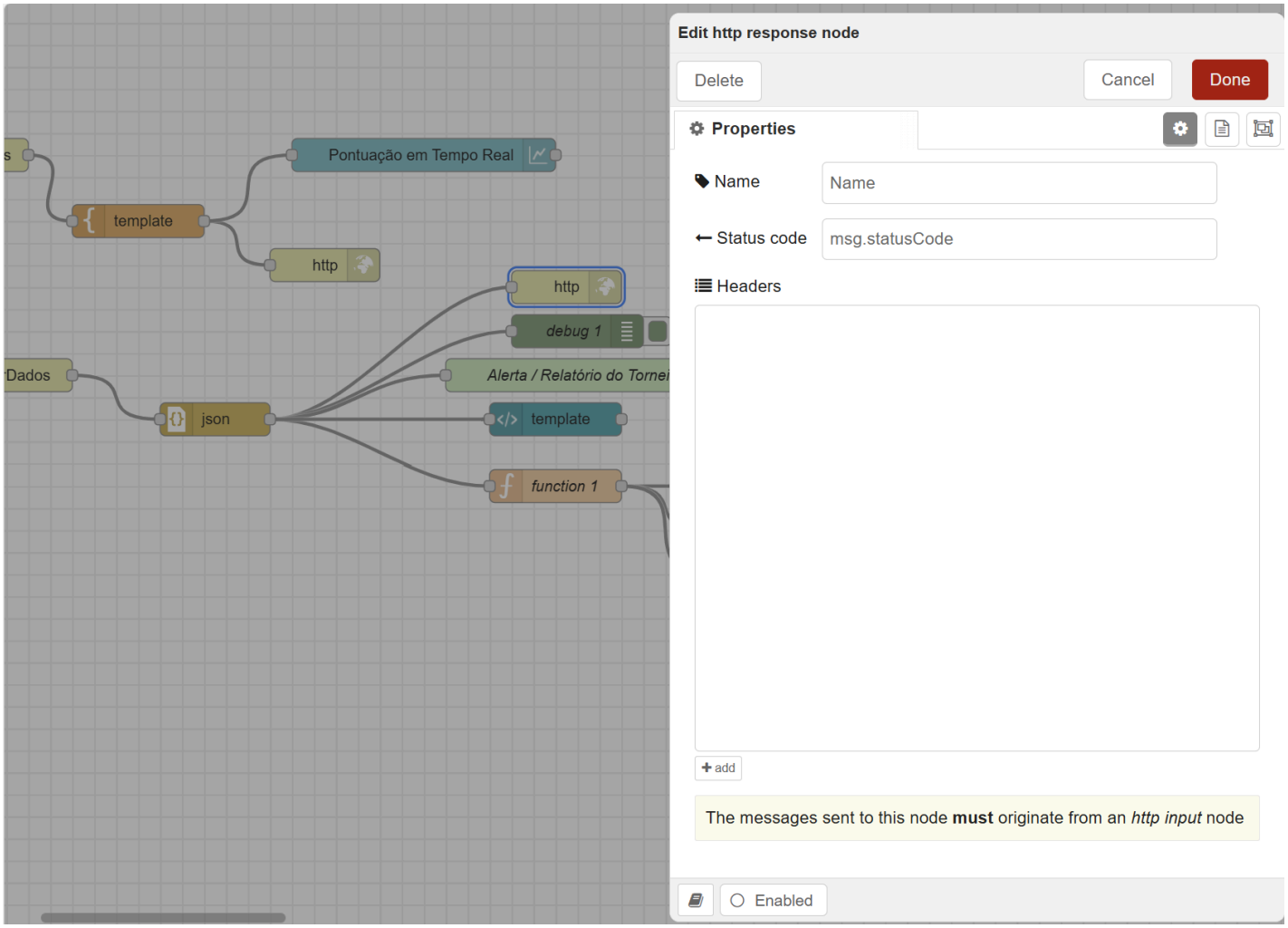


Fig 31 - http response

Após a conversão do payload recebido pelo endpoint /SalvarDados, a informação é encaminhada em paralelo para nós de validação e comunicação de alertas:

Monitorização e Depuração (debug 1):

- O nó debug 1 está ligado à saída do nó de conversão JSON para permitir a inspeção em tempo real das mensagens que circulam no fluxo.
- Configurado para direcionar o conteúdo de msg.payload para a janela de debug (debug window), este nó facilita a verificação do formato dos dados recebidos do KNIME e o troubleshooting do sistema durante a execução.

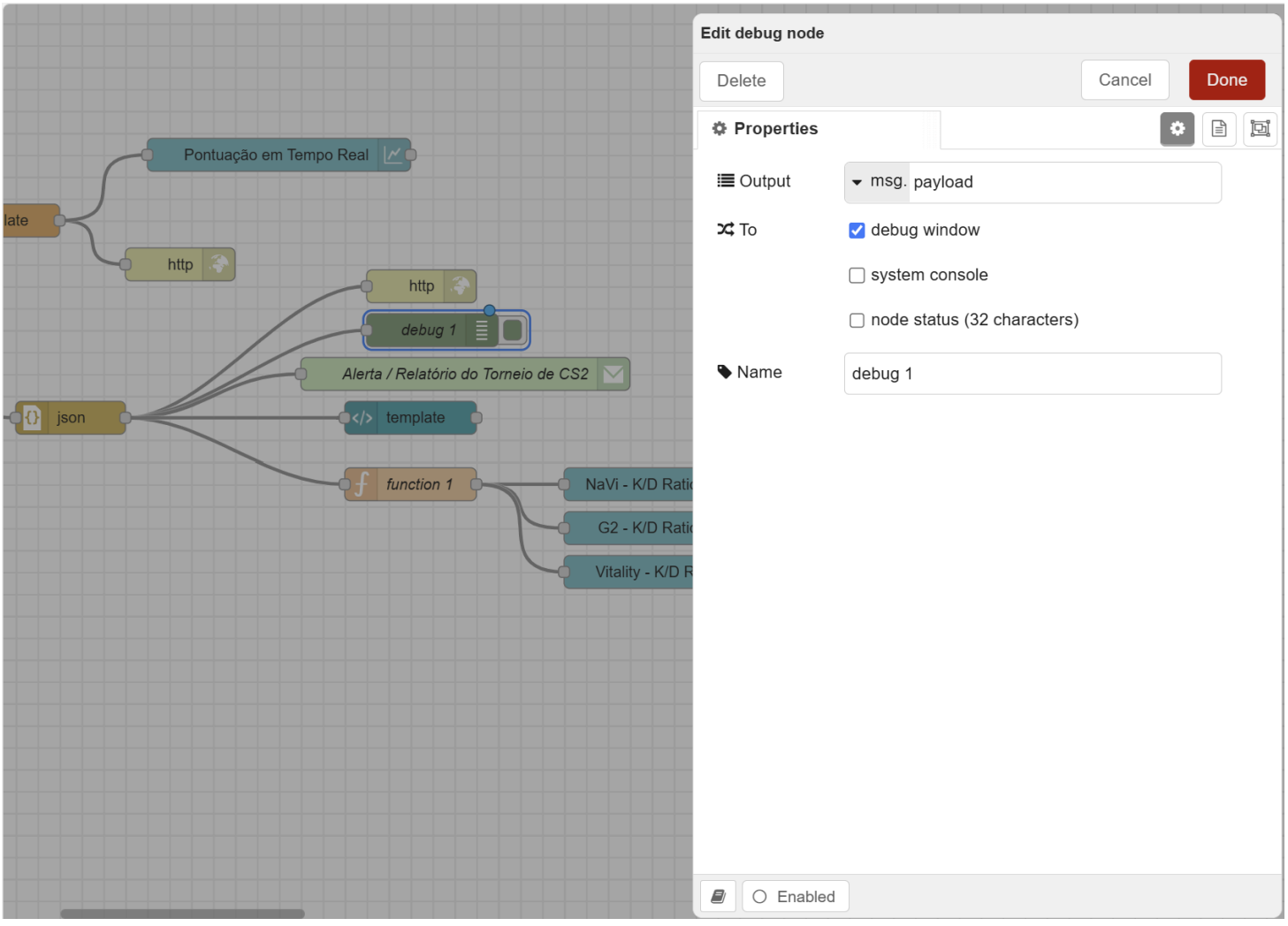


Fig 32 - Debug 1

Envio Automático de Relatórios e Alertas por E-mail (email):O nó Alerta / Relatório do Torneio de CS2 implementa um mecanismo de notificação automática via protocolo SMTP.

Envio Automático de Relatórios e Alertas por E-mail (email):

O nó Alerta / Relatório do Torneio de CS2 implementa um mecanismo de notificação automática via protocolo SMTP.

- Configuração da Ligação:Servidor SMTP: smtp.gmail.com na porta 465 com ligação segura ativa (Use secure connection / SSL/TLS).
- Autenticação: Configurada com credenciais dedicadas via protocolo seguro (Basic).
- Destinatário: O relatório é enviado automaticamente para o endereço de e-mail do administrador/gestor do sistema.

Sempre que o KNIME submete um novo lote de resultados processados, este nó dispara instantaneamente um e-mail com as métricas do torneio, garantindo um acompanhamento passivo e imediato dos eventos sem necessidade de consulta direta ao Dashboard.

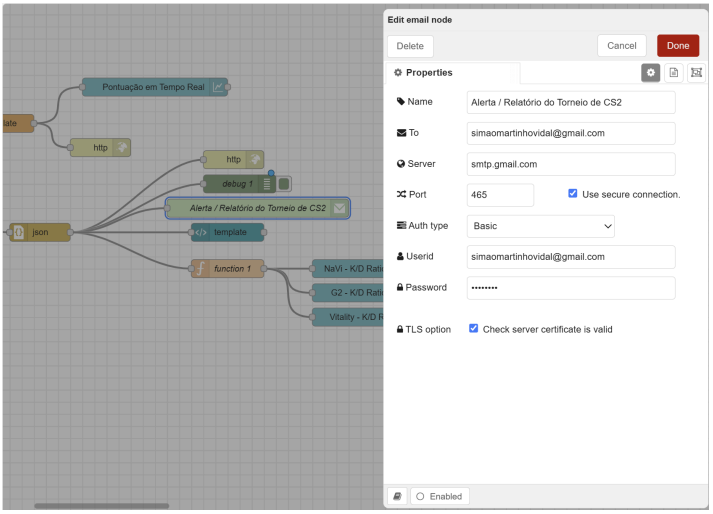


Fig 33 - Alerta / Relatório do Torneio de CS2

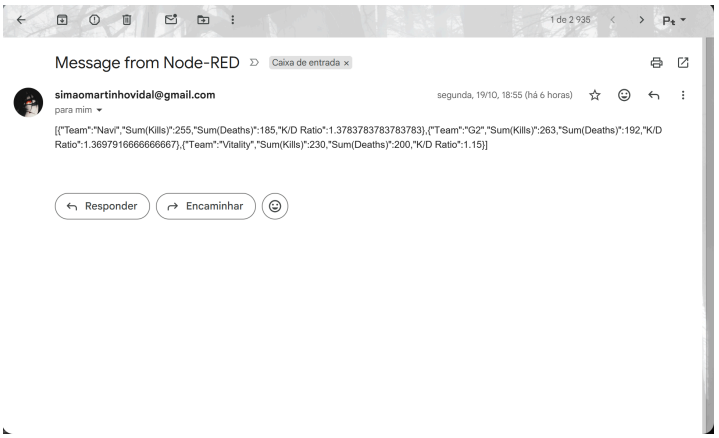


Fig 34 - Relatório

Encaminhamento Lógico e Visualização Dinâmica no Dashboard:

Para apresentar os dados processados na interface do utilizador, o Node-RED divide a informação recebida do KNIME em dois componentes visuais principais no Node-RED Dashboard:

Construção de Tabela Personalizada via HTML/AngularJS (template):

- O nó template de visualização (configurado como Widget in group) renderiza uma tabela dinamizada em HTML/CSS para o Dashboard.

- Utilizando a diretiva ng-repeat="item in msg.payload", o nó percorre iterativamente os objetos do array recebido do KNIME, gerando linhas dinâmicas para apresentar as colunas: Equipa, Kills, Deaths e K/D Ratio com estilo estruturado.

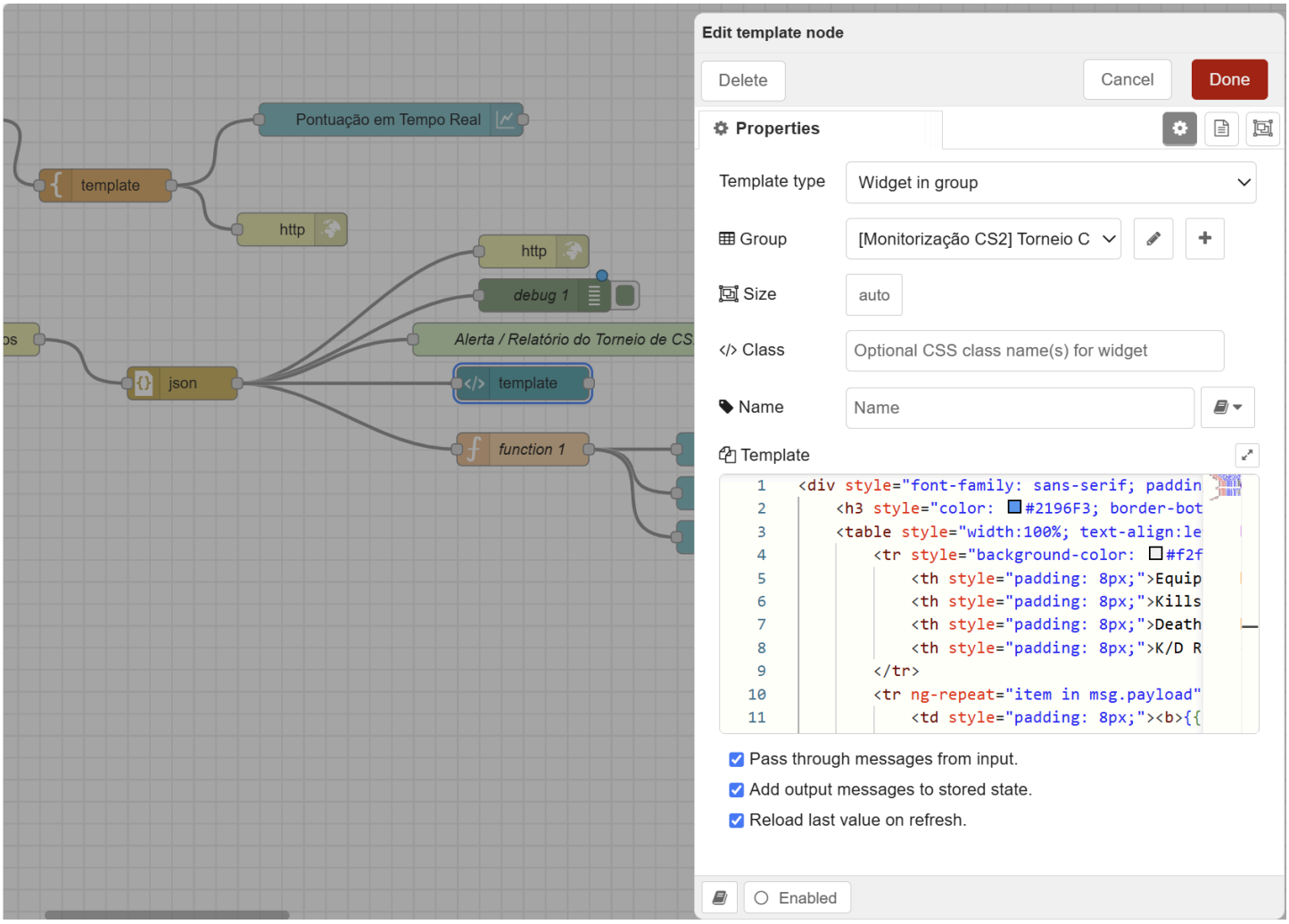


Fig 35 - Template

Encaminhamento Lógico e Visualização Dinâmica por Blocos Funcionais

Para garantir uma separação rigorosa e fiável dos indicadores visuais no Node-RED Dashboard, a arquitetura de processamento em JavaScript foi refatorizada em três nós de função independentes e paralelos:

Abordagem de Extração Dedicada por Bloco Funcional:

- Em vez de recorrer a saídas múltiplas a partir de uma única função, o payload JSON recebido do KNIME é bifurcado em paralelo para três nós de função distintos (function 1, function 2 e function 3).

Cada nó implementa um algoritmo de varredura isolado sobre o array de dados, filtrando iterativamente pelo nome exato da equipa ("Team") e extraindo unicamente a respetiva métrica de desempenho de interesse ("K/D Ratio"):

- function 1 (NaVi): Identifica o registo correspondente à NaVi, convertendo e atribuindo o rácio correspondente a msg.payload.

- function 2 (G2): Filtra os dados referentes à equipa G2, isolando o respetivo indicador numérico.

- function 3 (Vitality): Captura isoladamente o rácio da equipa Vitality para envio direto ao manómetro final.

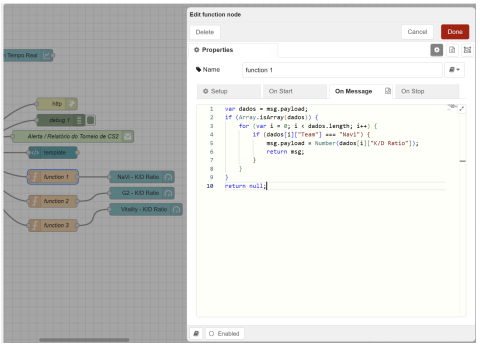


Fig 36 - function 1 (NaVi)

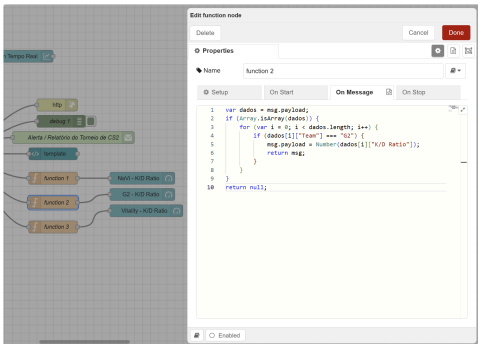


Fig 37 - function 2 (G2)

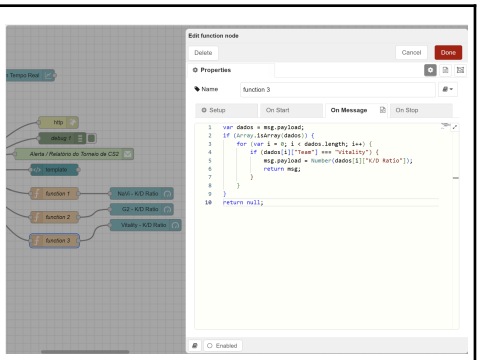


Fig 38 - function 3 (Vitality)

Manómetros de Desempenho em Tempo Real (gauge):

Foram configurados 3 nós do tipo Gauge interligados às três saídas do nó function 1:

- NaVi - K/D Ratio: Recebe o valor de naviKd da primeira saída.
- G2 - K/D Ratio: Recebe o valor de g2Kd da segunda saída.
- Vitality - K/D Ratio: Recebe o valor de vitalityKd da terceira saída.
- Todos os manómetros estão parametrizados na escala contínua de 0 a 2 (min: 0, max: 2), com gradiente de cores (Colour gradient) e formatação numérica com duas casas decimais ({{value | number:2}}), permitindo uma leitura visual imediata do desempenho de cada equipa no Dashboard.

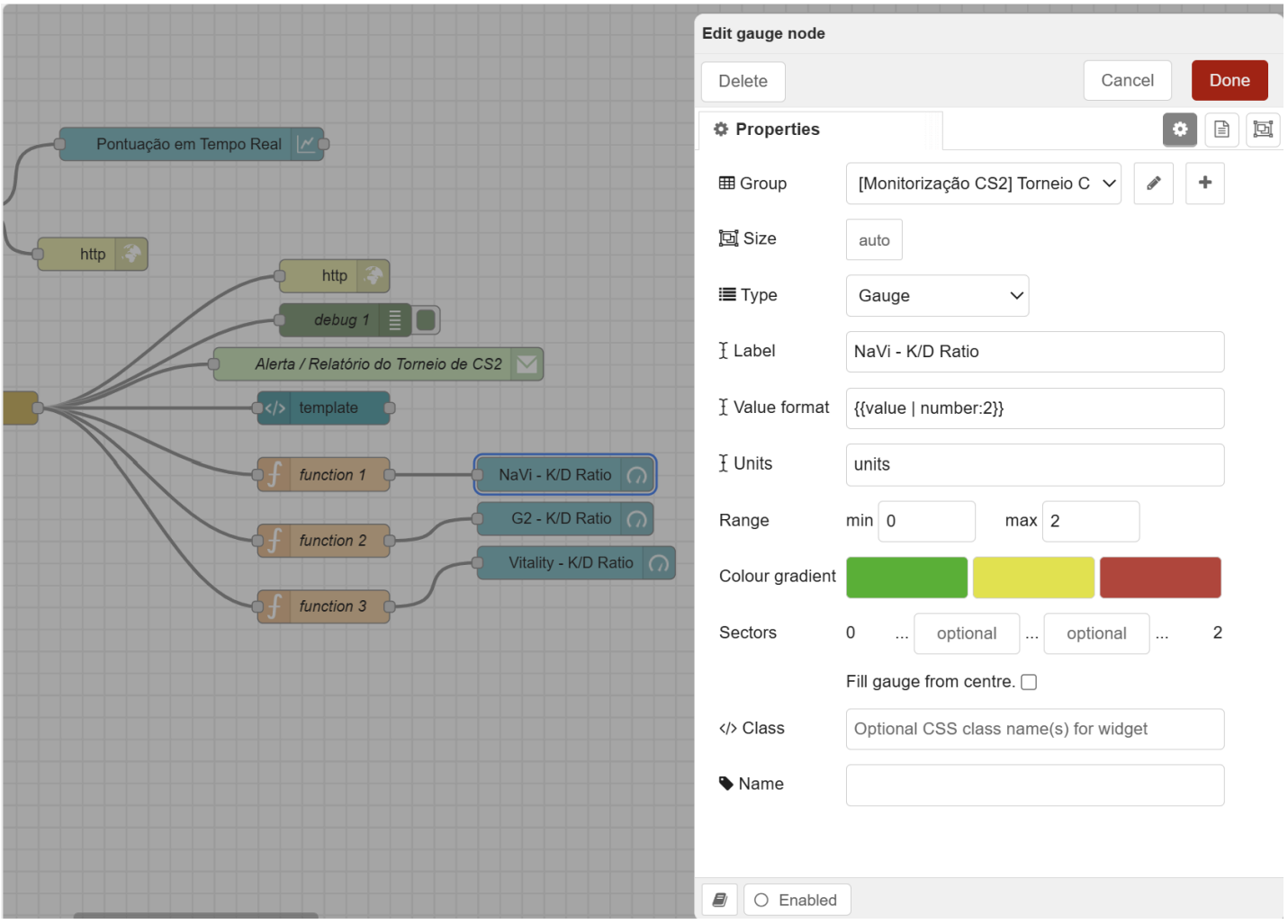


Fig 39 - Manómetro naviKd

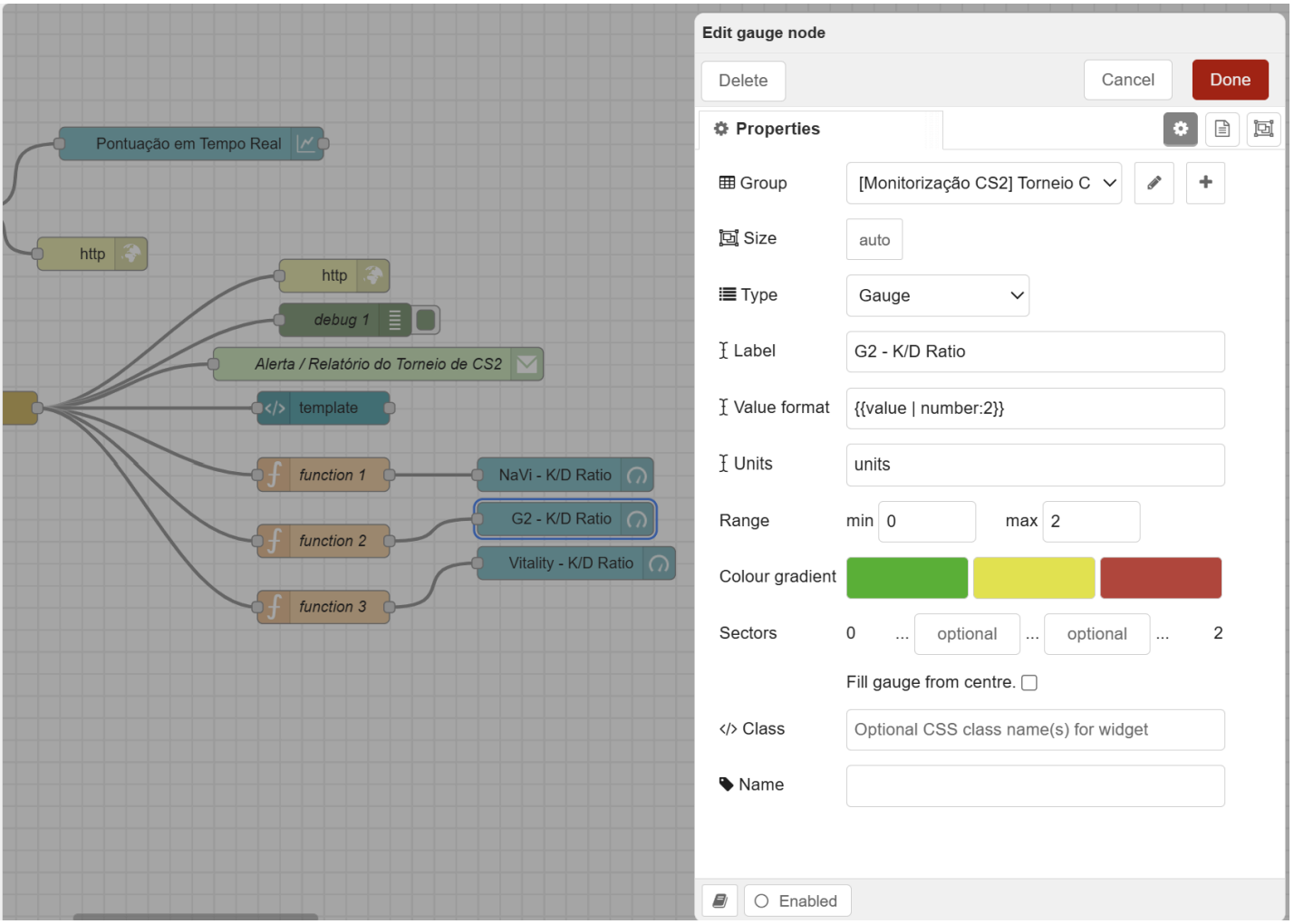


Fig 40 - Manómetro g2Kd

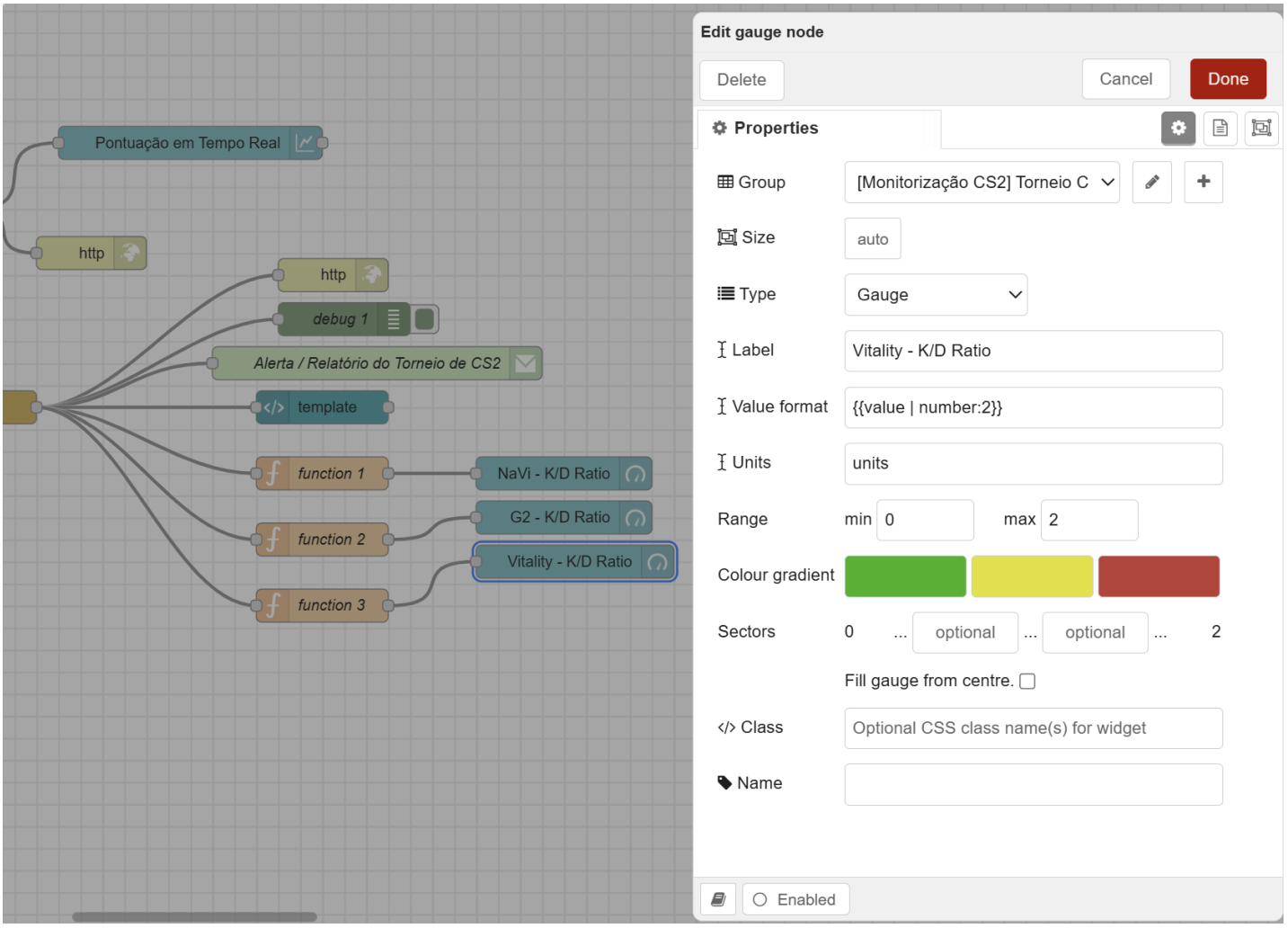


Fig 41 - Manómetro vitalityKd

15 - Interface Gráfica e Visualização Final

Node-RED Dashboard

A camada de apresentação visual desenvolvida no Node-RED Dashboard consolida a arquitetura de integração de sistemas, exibindo em tempo real os resultados calculados e processados pela pipeline do KNIME:

Estruturação do Painel de Controlo:

O dashboard está organizado sob o título global Monitorização CS2 (grupo Torneio CS2), integrando múltiplos widgets interativos que refletem instantaneamente o estado do torneio.

Indicadores de Desempenho em Tempo Real (Gauges):

- O painel apresenta três manómetros independentes dedicados a cada organização (NaVi, G2 e Vitality), dimensionados numa escala de 0 a 2.
- Como se verifica nas capturas da interface, os manómetros traduzem visualmente o K/D Ratio apurado (ex: 1.38 para a NaVi, 1.37 para a G2, e 1.15 para a Vitality), permitindo uma avaliação imediata da performance competitiva das equipas.

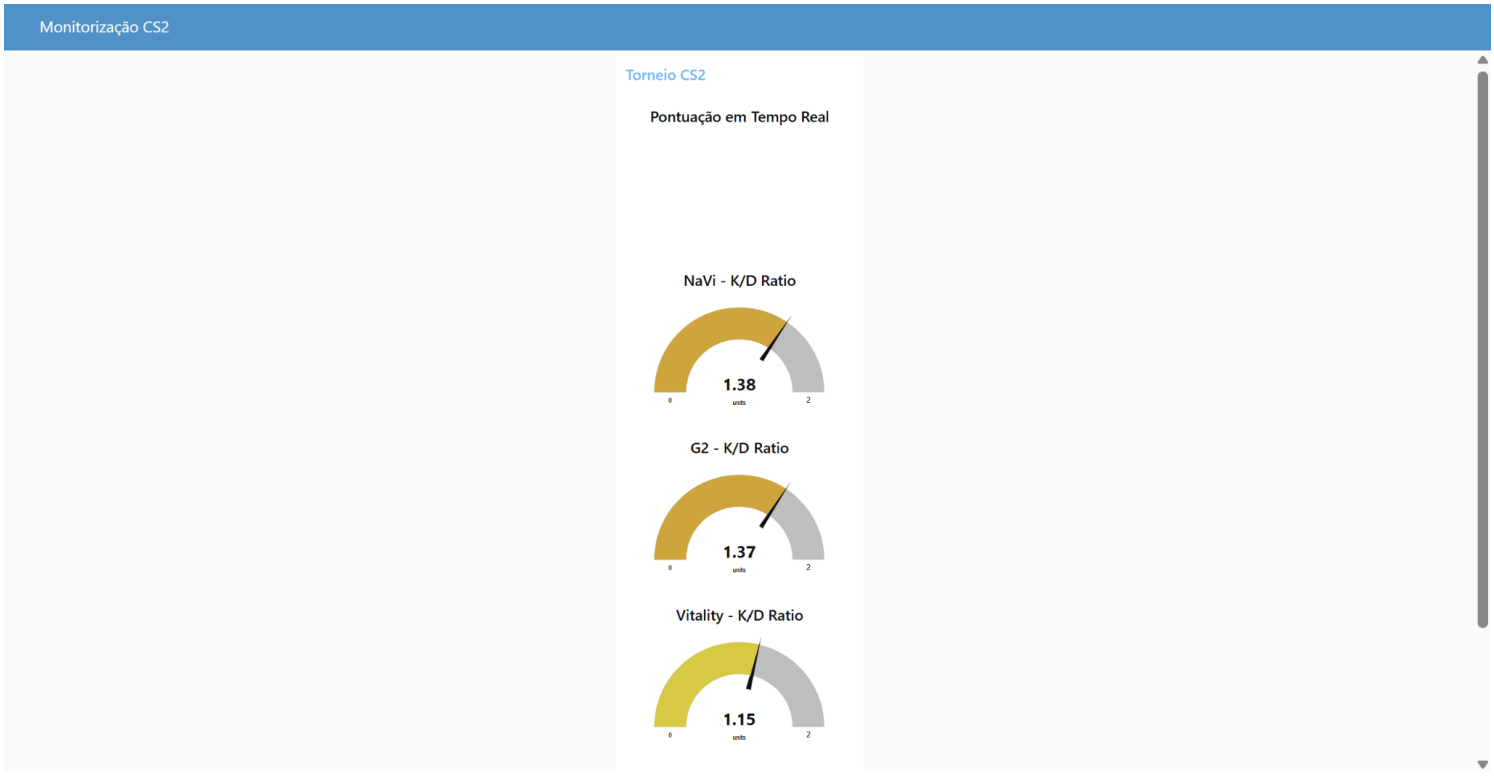


Fig 42 - Manómetros K/D Ratio

Resumo Dinâmico:

Na secção inferior do dashboard (Resumo do Torneio), a tabela dinâmica gerada pelo nó template renderiza de forma limpa e estruturada o detalhe completo dos dados recolhidos e processados:

- NaVi: 255 Kills, 185 Deaths (Rácio: 1.38).
- G2: 263 Kills, 192 Deaths (Rácio: 1.37).
- Vitality: 230 Kills, 200 Deaths (Rácio: 1.15).

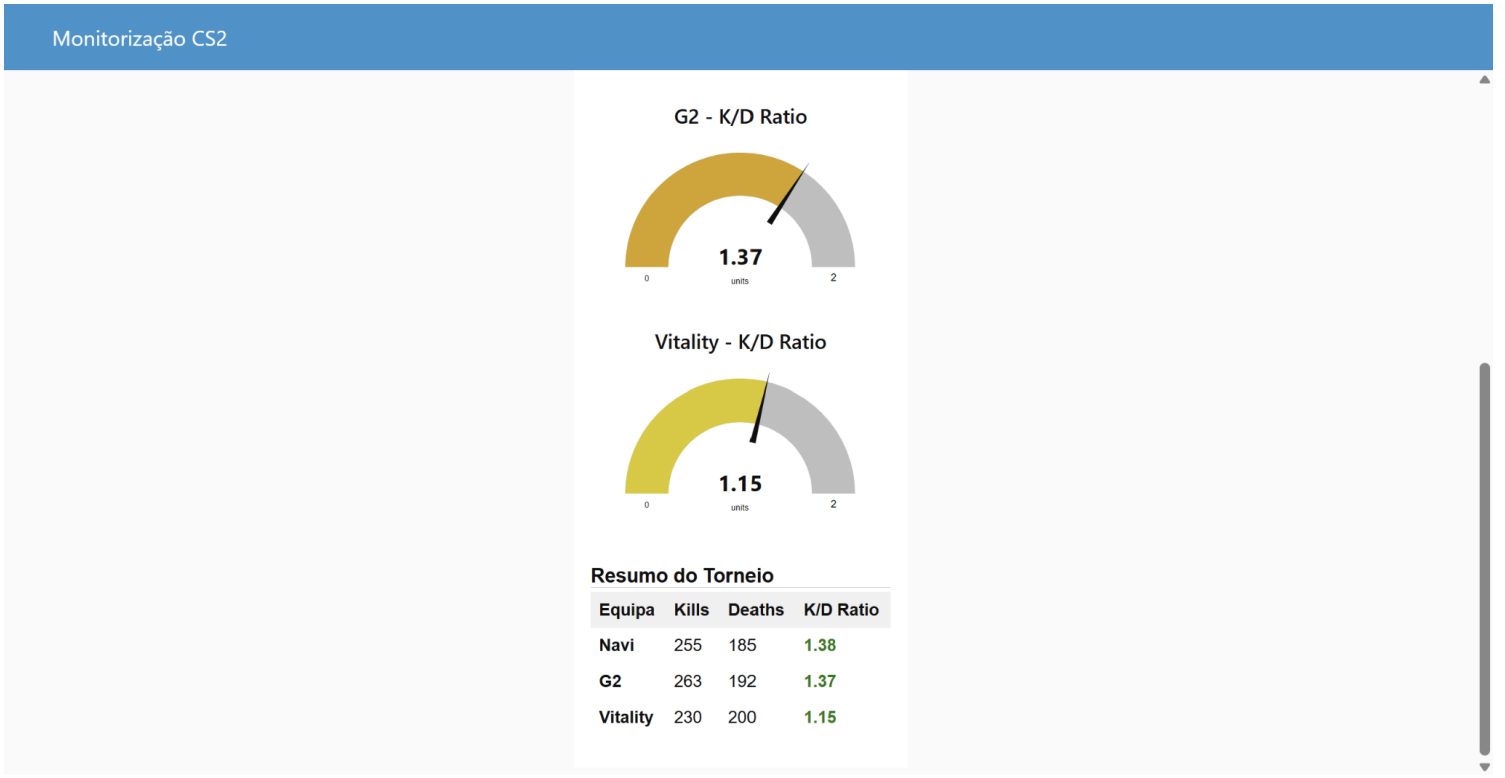


Fig 43 - Tabela dinâmica

Parte II - Implementação no Pentaho Data Integration (Kettle)

16 Visão Geral do Fluxo no Pentaho Kettle

Para dar resposta aos objetivos de integração e orquestração propostos neste trabalho prático, foi desenhada e implementada uma *pipeline* completa de ETL (Extract, Transform, Load) utilizando a ferramenta Pentaho Data Integration (Kettle).

O fluxo desenvolvido, visível na figura abaixo, estrutura-se em três grandes fases operacionais, garantindo o processamento de dados desde a sua extração bruta até à sua disponibilização em tempo real num *dashboard* interativo:

1. **Fase 1 (Extração e Limpeza):**

   O fluxo divide-se inicialmente em dois ramos paralelos. O ramo da esquerda é responsável por ingerir e limpar os dados estáticos dos jogadores em formato CSV, aplicando validação via Expressões Regulares (Regex). O ramo da direita extrai dinamicamente os dados estruturados das partidas (JSON) via API REST.

2. **Fase 2 (Cruzamento e Transformação de Negócio):**

   Os dois fluxos convergem num nó de junção relacional (`Merge join`). Seguidamente, aplicam-se operações de agregação e cálculos matemáticos para apurar o desempenho de cada organização (K/D Ratio).

3. Fase 3 (Persistência e Exportação):

Na etapa final, a informação consolidada é ramificada para assegurar a sua persistência local em múltiplos formatos (XML, JSON e CSV), sendo simultaneamente enviada através de requisições HTTP POST para o Node-RED.

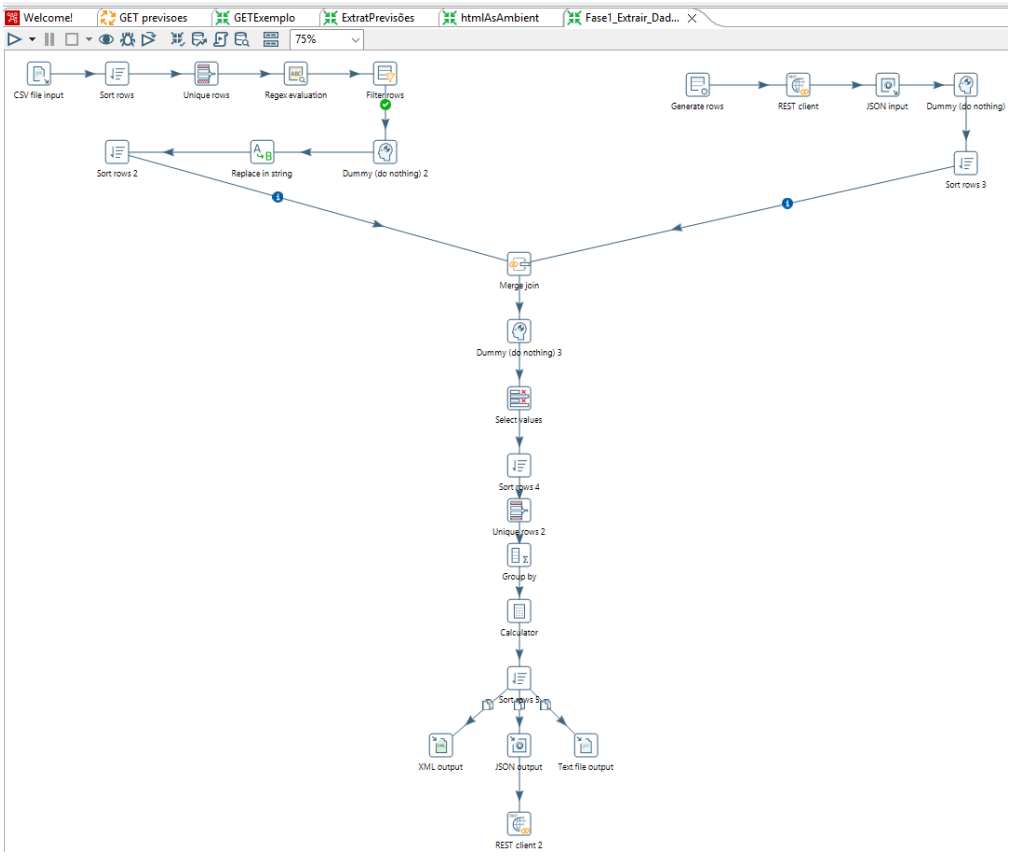


Fig 44 - Visão Geral do Fluxo no Pentaho Kettle

17 Processamento e Limpeza de Dados Estruturados (CSV)

O primeiro ramo desenvolvido no Pentaho Data Integration (Kettle) é responsável pela importação e normalização dos dados dos jogadores e equipas (provenientes do ficheiro local em formato CSV). O fluxo implementado compreende as seguintes etapas sequenciais de tratamento de dados:

CSV file input:

O passo `CSV file input` efetua a leitura do repositório local, carregando a tabela inicial com os atributos estruturados (como ID, Nickname, Team, Kills, Deaths e Email). Nesta fase inicial, o conjunto de dados bruto continha inconsistências propositais para validação do sistema, tais como registos duplicados e entradas inválidas (com formatos de e-mail incorretos).

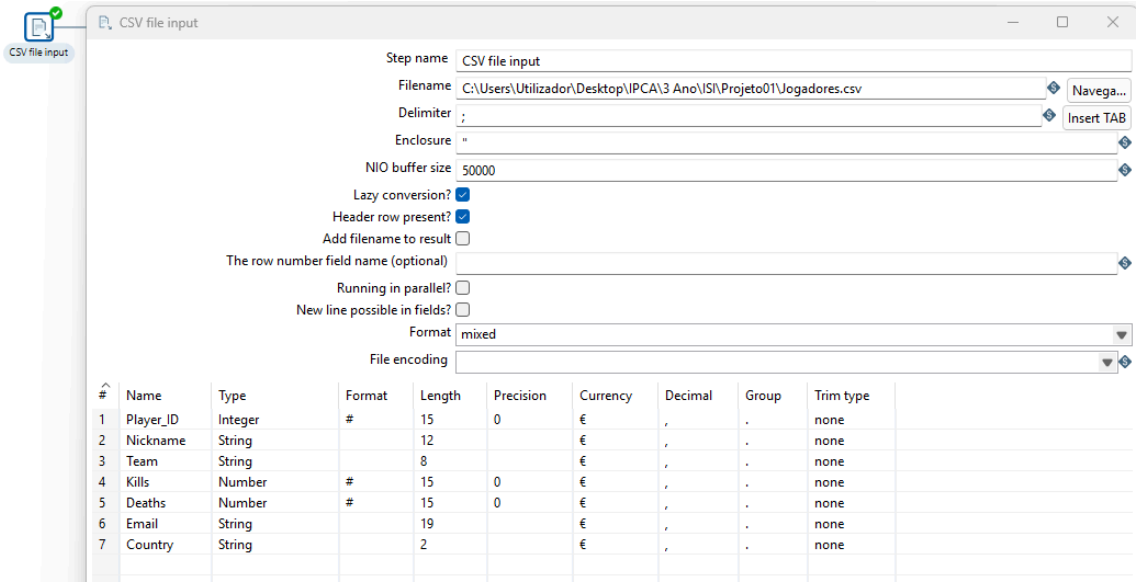


Fig 45 - CSV file input

Sort rows e Unique rows:

Para assegurar a unicidade dos dados dos atletas e evitar distorções nas agregações estatísticas, utilizou-se a combinação dos passos `Sort rows` (para agrupar e ordenar os dados) seguido do `Unique rows`. Esta operação filtrou com sucesso os registos redundantes, mantendo apenas entradas únicas.

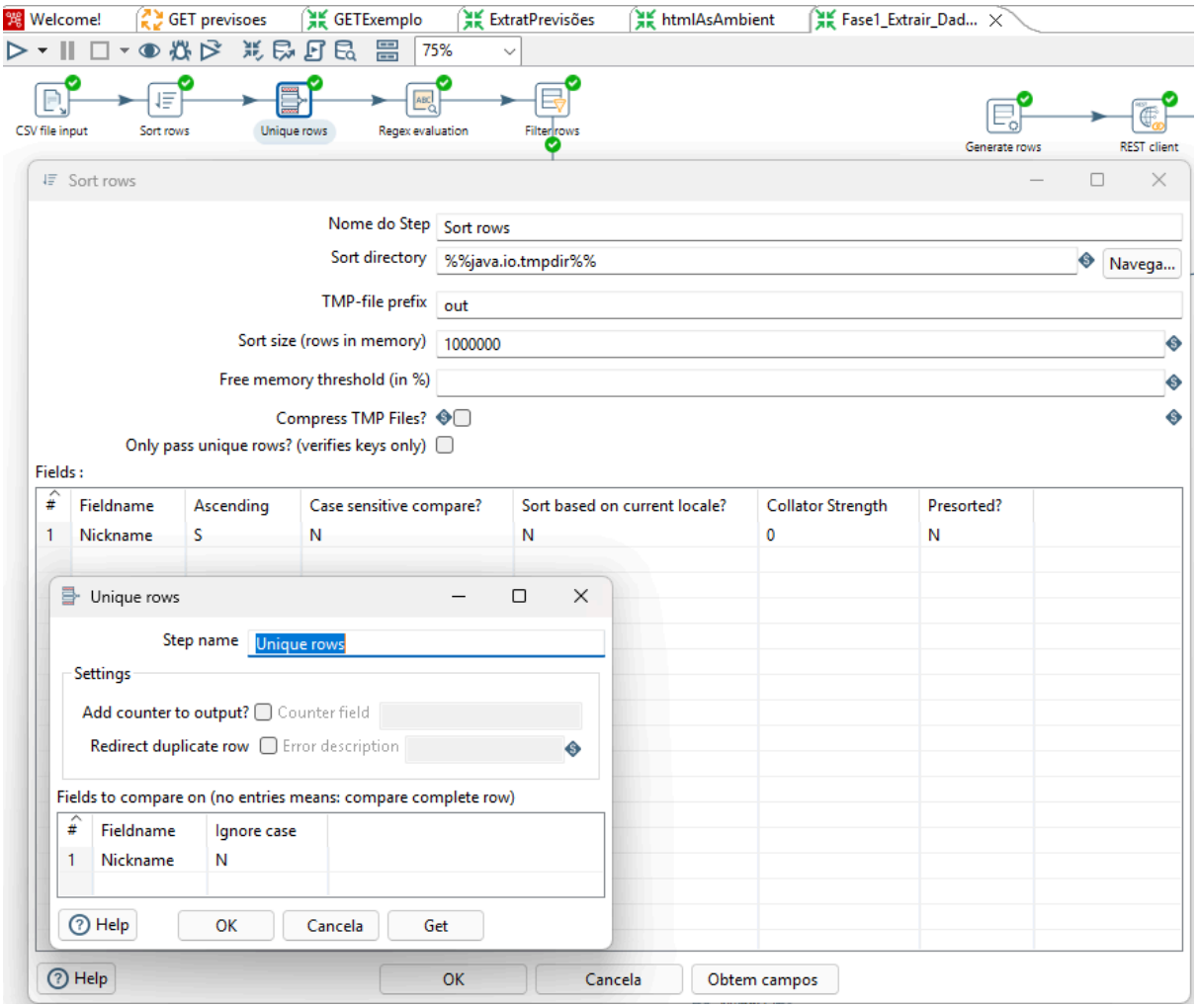


Fig 46 - Sort rows e Unique rows

`Regex evaluation`:

Para analisar a formatação da coluna "Email" através de Expressões Regulares. O resultado desta avaliação foi posteriormente utilizado pelo passo `Filter rows`.

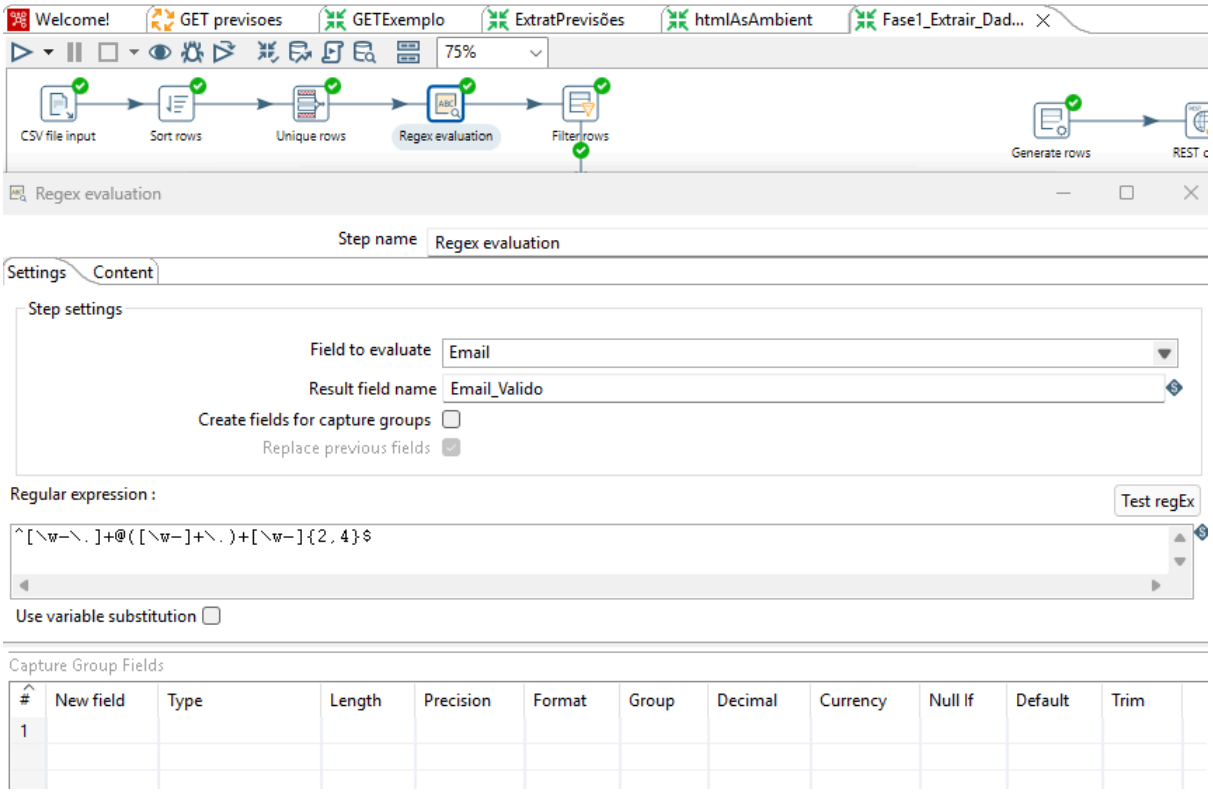


Fig 47 - Regex evaluation

Filter rows:

Através do passo `Filter rows`, aplicou-se uma regra de exclusão direcionada à coluna Email, eliminando registos que continham anomalias ou falhas de validação. Após esta filtragem, a tabela prosseguiu apenas com registos consistentes correspondentes aos jogadores ativos das equipas.

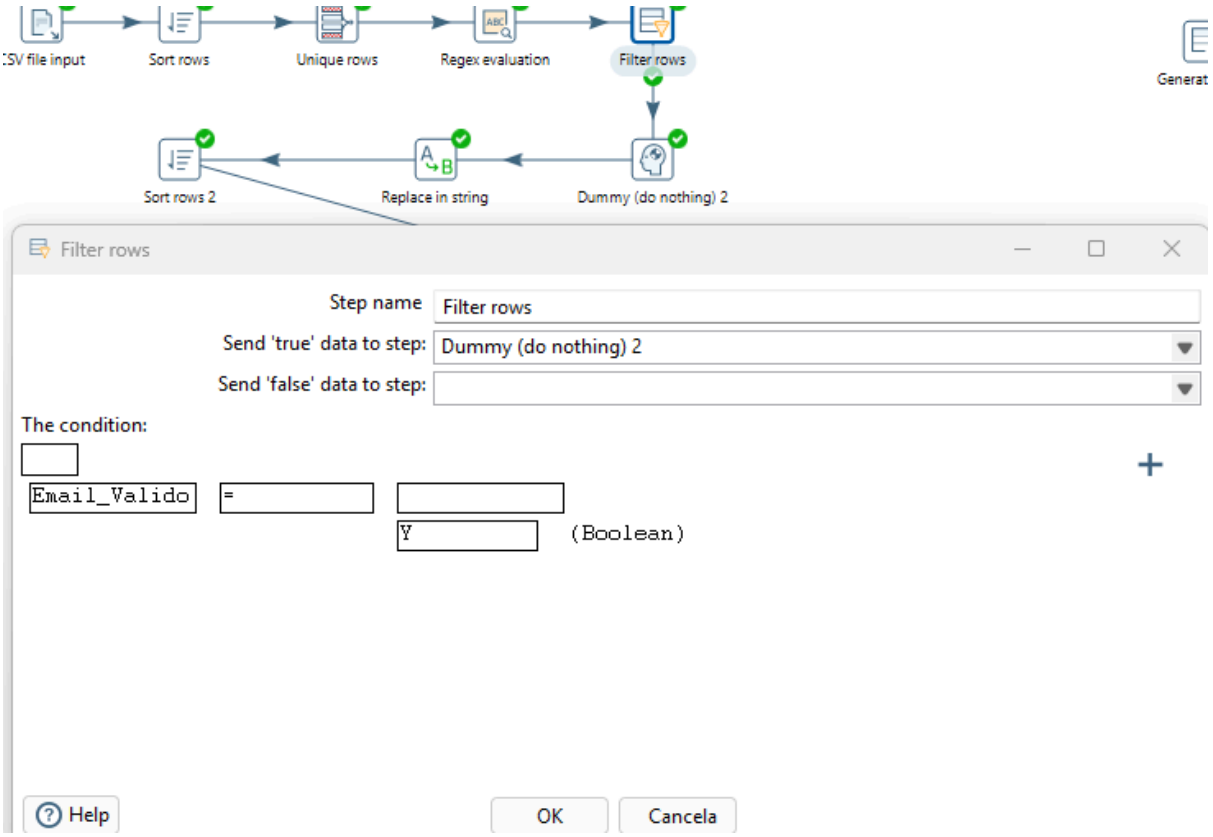
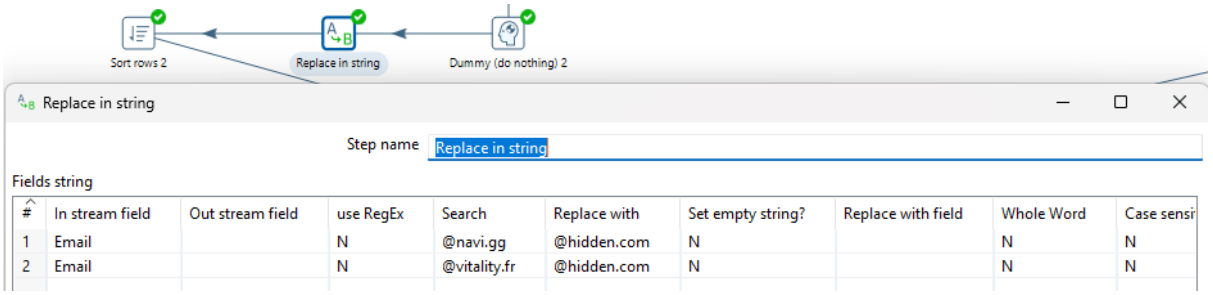


Fig 48 - Filter rows

Replace in string:

Para cumprir os requisitos de anonimização de dados sensíveis, recorreu-se ao passo `Replace in string`. Neste nó, substituiu-se o domínio de todos os e-mails dos jogadores por um domínio genérico (`@hidden.com`), garantindo a anonimização em massa e preservando a estrutura da informação.



#	In stream field	Out stream field	use RegEx	Search	Replace with	Set empty string?	Replace with field	Whole Word	Case sensi
1	Email		N	@navi.gg	@hidden.com	N		N	N
2	Email		N	@vitality.fr	@hidden.com	N		N	N

Fig 49 - Replace in string

18 Ingestão de Dados JSON e Consumo de API REST (HTTP GET)

O segundo ramo do fluxo (à direita) é dedicado à importação de dados semi-estruturados relativos às partidas do torneio, simulando o consumo de uma arquitetura orientada a serviços:

Generate rows e REST client:

Para iniciar o processo de recolha de dados externos, utilizou-se o nó `Generate rows` para instigar a execução, seguido do nó REST `client`. Este passo está configurado para efetuar um pedido HTTP GET à API, consumindo o *payload* dinâmico com os dados das partidas disputadas.

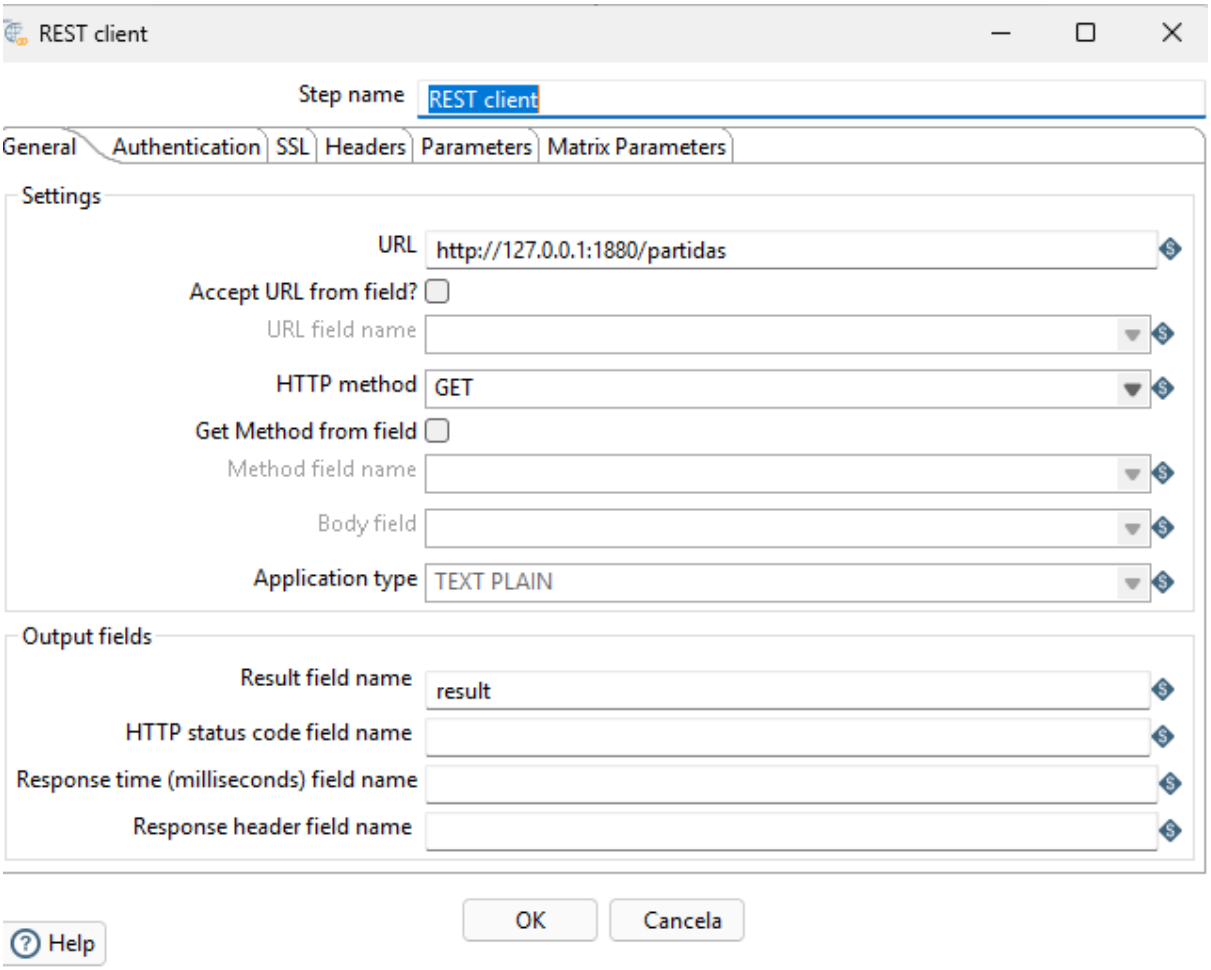


Fig 50 - REST client (GET)

JSON input:

O *payload* recebido é passado ao nó JSON `input`. Este passo atua simultaneamente como leitor e mapeador (substituindo a necessidade de múltiplos nós como *JSON Path* ou *Ungroup* de outras plataformas). Aqui, extraíram-se os atributos chave através de expressões JSONPath (`match_id`, `map`, `team_a`, `team_b`, `score_a` e `score_b`), transformando as listas em linhas individuais prontas para a união relacional.

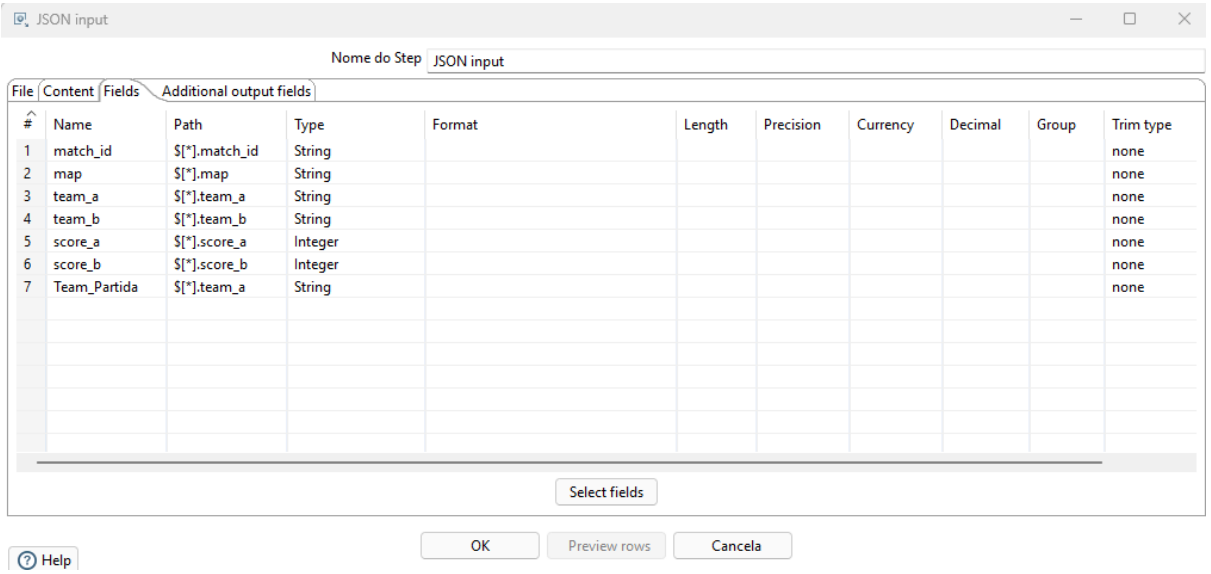


Fig 51 - JSON input

19 Cruzamento de Dados Relacionais

Sort rows e Merge join:

Com as duas fontes de dados tratadas (CSV à esquerda e JSON à direita), procedeu-se à integração através do nó `Merge join`. No Pentaho Kettle, esta operação exige que os dados de entrada estejam previamente ordenados, motivo pelo qual se utilizaram os passos `Sort rows 2` e `Sort rows 3`. O `Merge join` foi configurado para cruzar a coluna de equipa dos jogadores (`Team`) com a equipa das partidas, fundindo o desempenho individual com o contexto global do torneio.

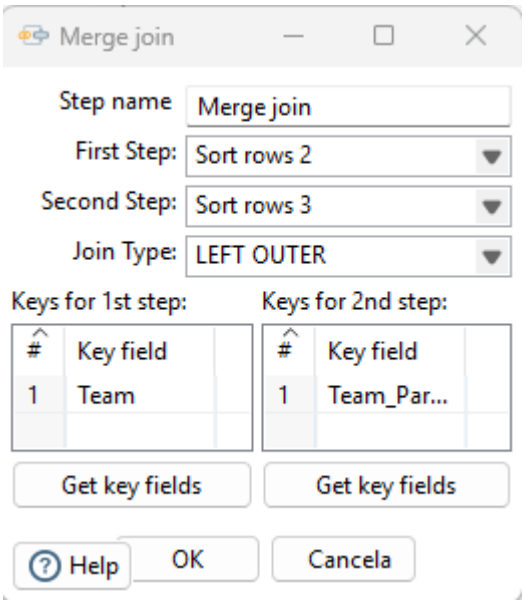


Fig 52 - Merge join

20 Limpeza Pós-Junção e Desduplicação

Select values:

Após o cruzamento de dados no nó `Merge join` (e passagem por um nó `Dummy` de controlo), a tabela resultante continha o produto da união das duas origens (CSV e JSON), o que muitas vezes gera colunas redundantes ou desnecessárias para a análise final. Utilizou-se o passo `Select values` para limpar a *stream* de dados, removendo os campos que já não eram precisos e selecionando apenas as colunas essenciais (como o nome da Equipa, as Kills e as Deaths) para otimizar o processamento seguinte.

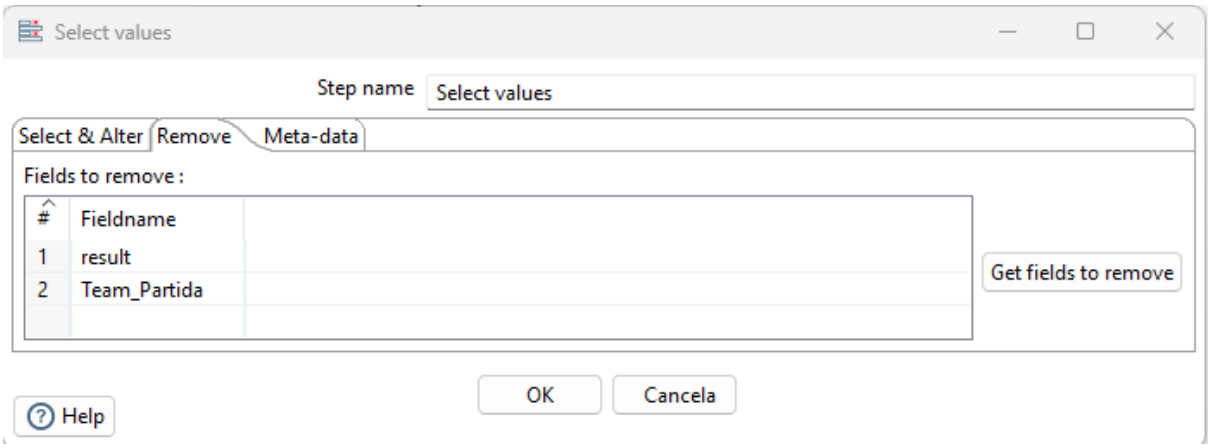


Fig 53 - Select values

Sort rows 4 e Unique rows 2:

Devido ao produto inerente ao cruzamento entre jogadores e múltiplas partidas, ocorreu a duplicação estatística dos registos. Para corrigir isto e preparar os dados para os cálculos globais, utilizou-se o nó `Sort rows 4` seguido do `Unique rows 2`. Este passo isolou cada atleta de forma única, eliminando a sobreposição de métricas antes do cálculo por equipa.

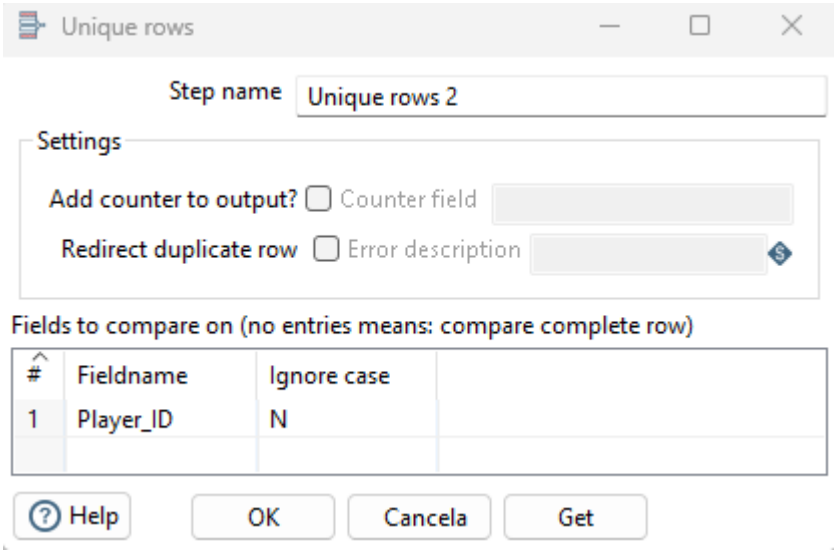
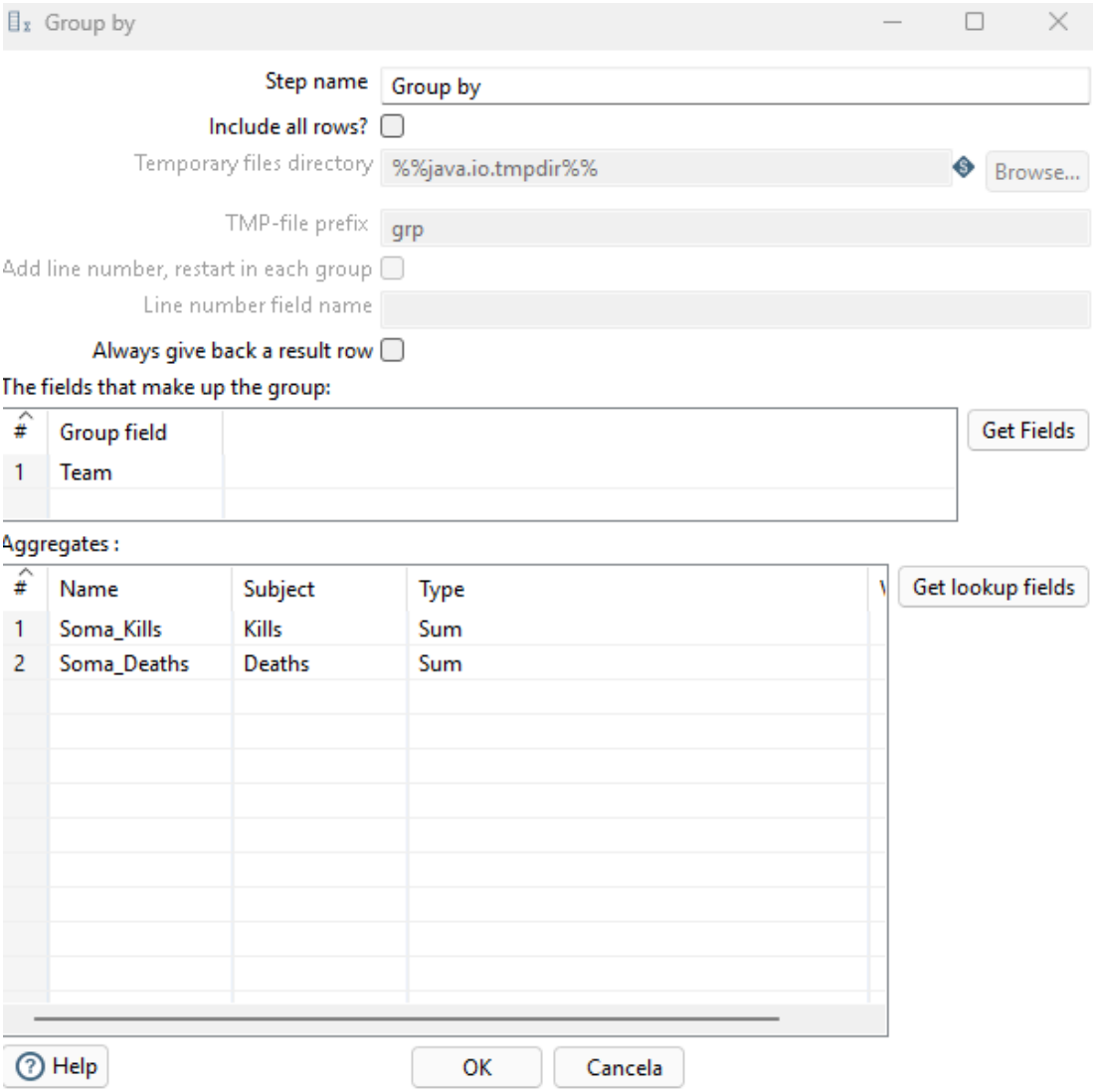


Fig 54 - Unique rows 2

21 Agregação e Consolidação de Métricas por Equipa

Group by e Calculator:

No passo `Group by`, agrupou-se a tabela pela coluna `Team`, definindo-se a soma para as colunas *Kills* e *Deaths* (`Soma_Kills` e `Soma_Deaths`). Logo a seguir, utilizou-se o nó `Calculator` para formular o indicador de desempenho, aplicando o cálculo `A / B` (Soma_Kills a dividir por Soma_Deaths), gerando a métrica fundamental de eSports: o KD_Ratio.



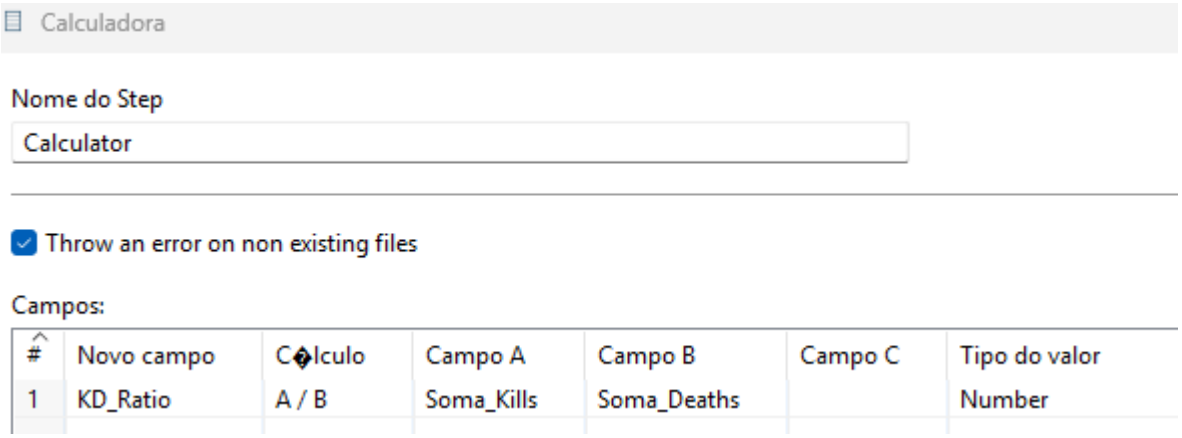


Fig 55 -Group by e Calculator e Cálculo de KD_Ratio

22 Ordenação de Desempenho por K/D Ratio

Sort rows 5:

O nó `Sort rows 5` organizou a tabela de forma estritamente decrescente com base no rácio. O resultado comprova a liderança da equipa NaVi (rácio de 1.38), seguida da G2 (1.37) e da Vitality (1.15).

Rows of step: Sort rows 5 (3 rows)

#	Team	Soma_Kills	Soma_Deaths	KD_Ratio
1	Navi	255,0	185,0	1,3783783784
2	G2	263,0	192,0	1,3697916667
3	Vitality	230,0	200,0	1,15

Fig 56 - Sort rows 5

23 Persistência e Exportação Multiformato (CSV, JSON e XML)

Após o ordenamento final, o fluxo do Pentaho Kettle divide-se em múltiplas ramificações paralelas (utilizando a distribuição de linhas) para garantir a persistência dos resultados finais em diferentes formatos no repositório local:

Text file output (Exportação para CSV):

Este passo exporta a tabela final de métricas diretamente para o sistema de ficheiros local num ficheiro `.csv`. O ficheiro é configurado com um delimitador específico, mantendo os cabeçalhos das colunas (Team, Soma_Kills, Soma_Deaths, KD_Ratio).

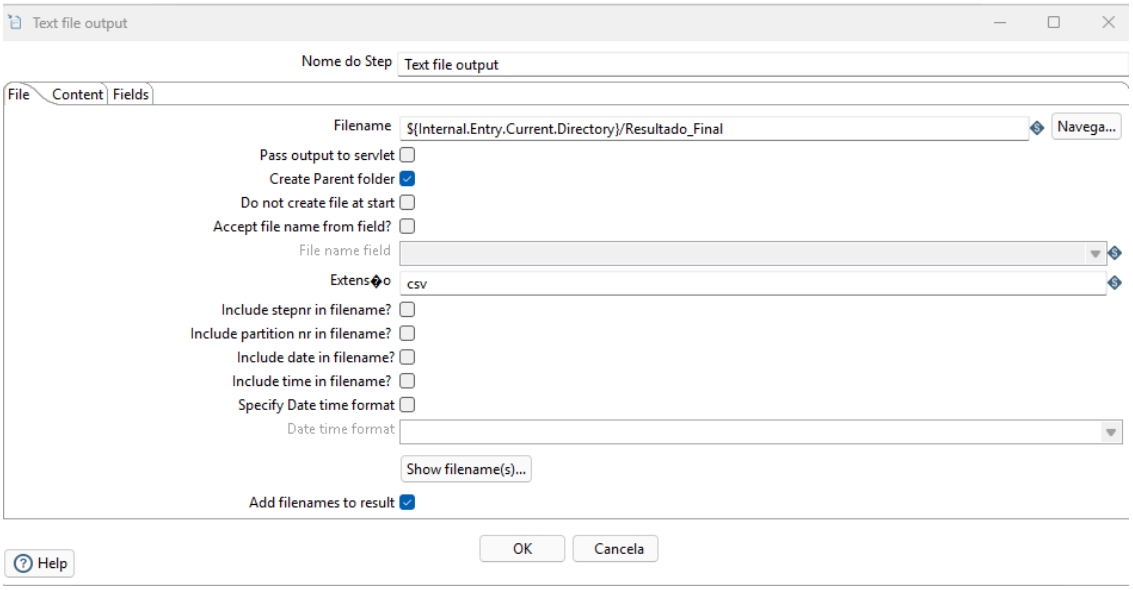


Fig 57 - Exportação Text file output

XML output:

Para responder a cenários de integração com sistemas orientados a XML, utilizou-se este passo que converte a estrutura relacional tabular numa árvore de marcação XML estruturada e guarda o ficheiro localmente.

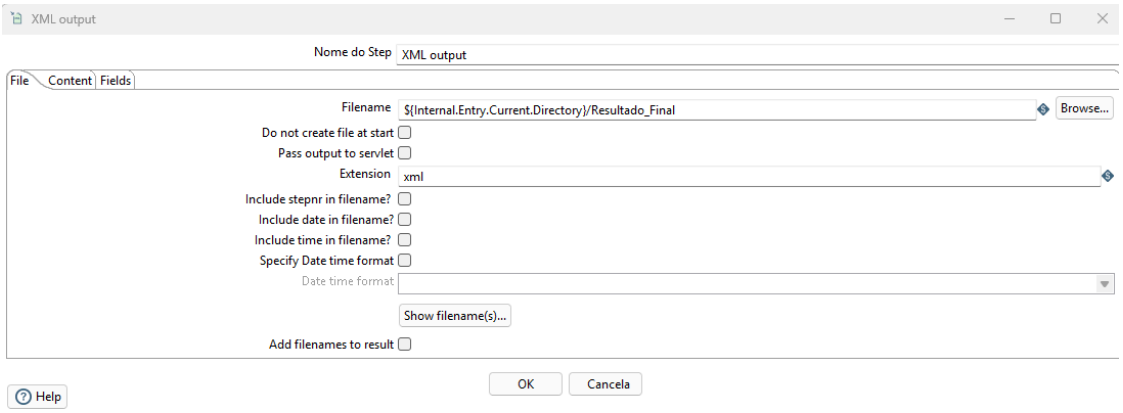


Fig. 58 - XML Output

JSON output:

Este passo é crucial na arquitetura, possuindo uma dupla finalidade. Primeiro, serializa a tabela relacional num JSON e grava o ficheiro `.json` no disco. Segundo, gera um novo campo (uma *string* JSON) que é passada diretamente para o passo seguinte da *pipeline*, permitindo o envio dos dados.

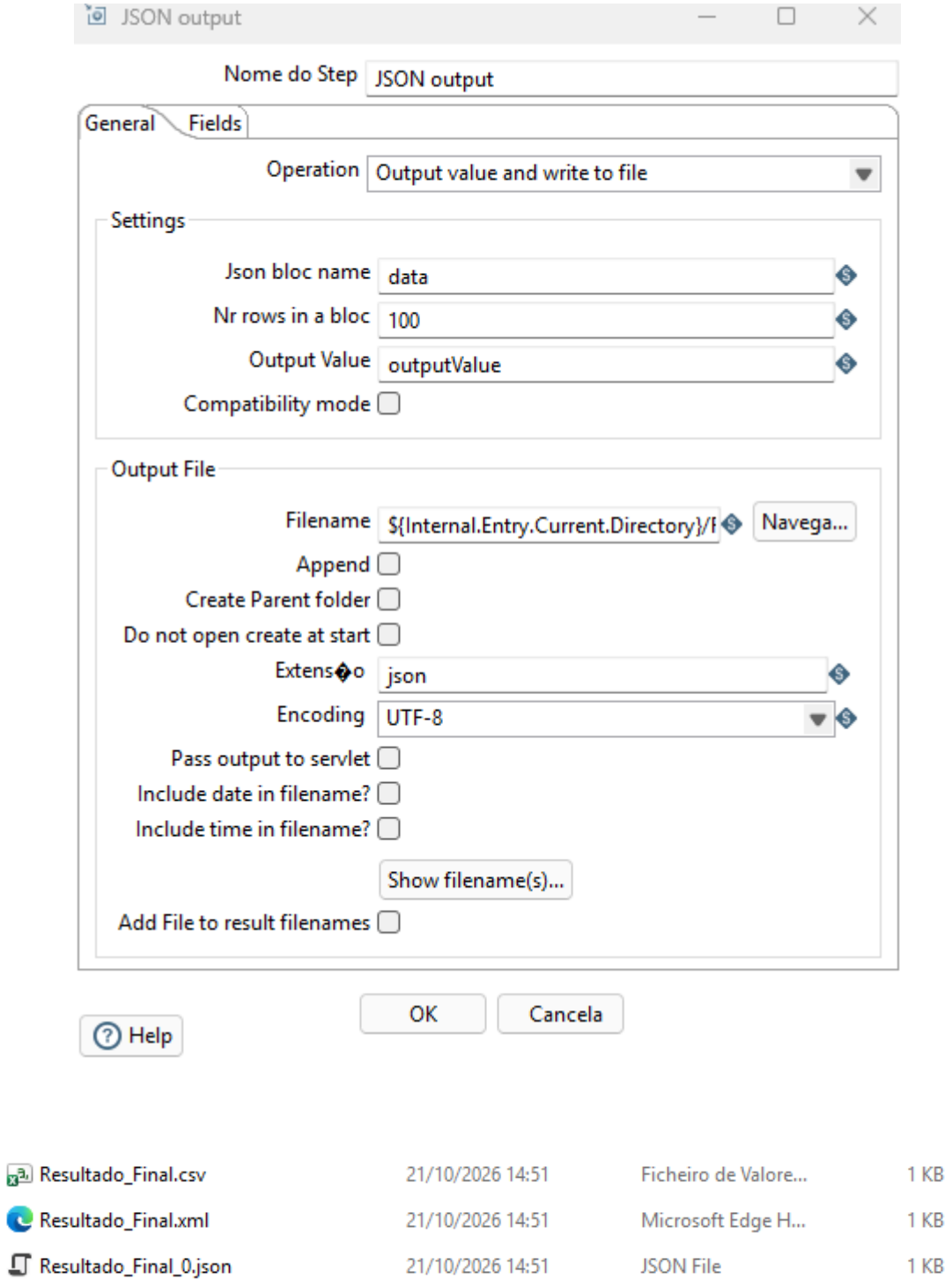
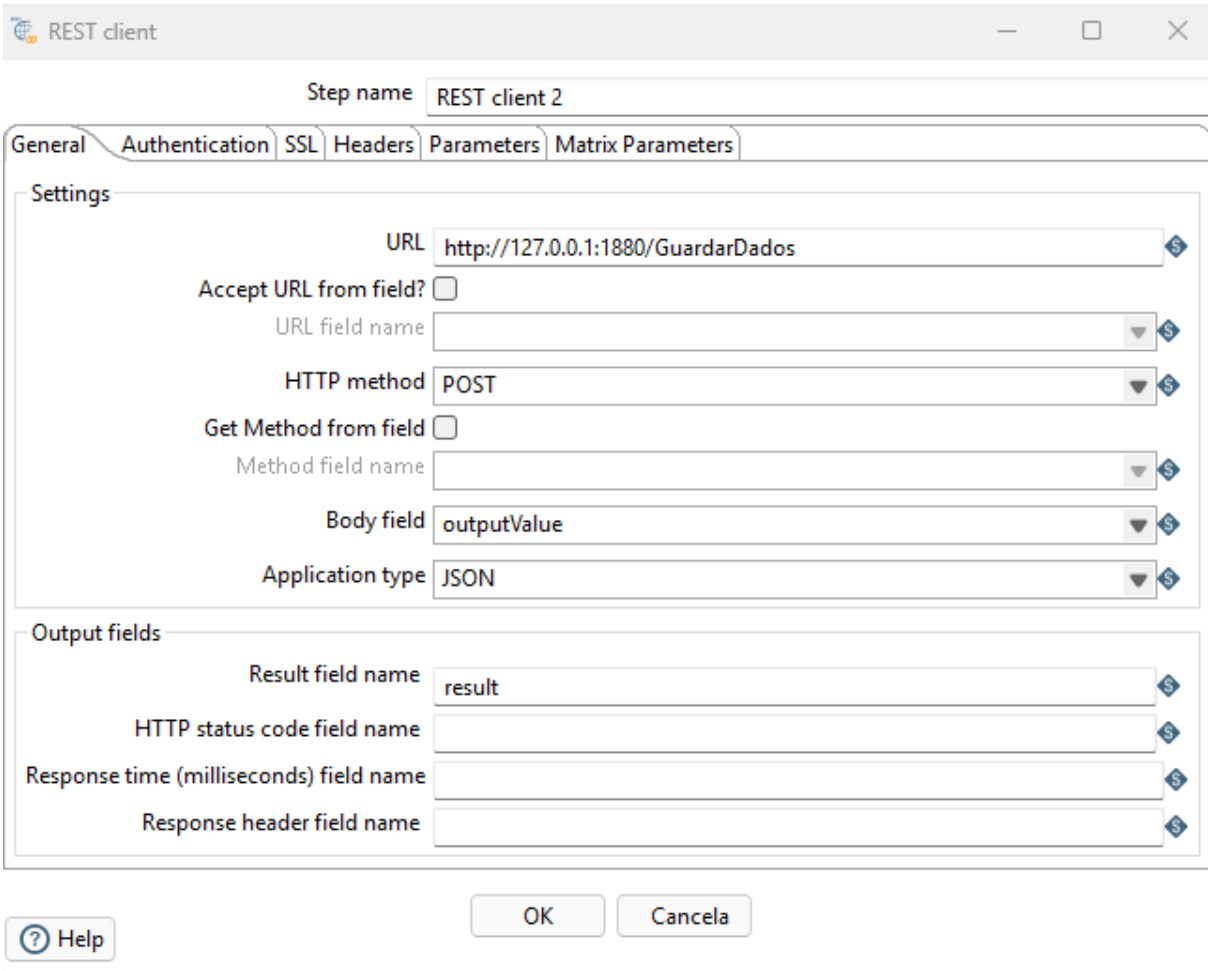


Fig 59 - Exportação JSON output

24 Serialização e Envio de Dados em Tempo Real

REST client 2:

Após a serialização, o nó REST `client 2` estabelece a ponte de integração em tempo real com a camada de visualização (Node-RED). Configurado para utilizar o método HTTP POST, este passo submete o *payload* JSON (criado pelo passo anterior) diretamente para o *endpoint* REST configurado na aplicação cliente, através do URL `[http://127.0.0.1:1880/GuardarDados](http://127.0.0.1:1880/GuardarDados)`.



:Fig 60- REST client 2 (POST)

25 Endpoint de Ingestão de Dados e Deserialização

Para receber as métricas agregadas e calculadas pelo Pentaho Kettle (como o Rácio, Kills e Deaths por equipa) em tempo real, foi criado um *endpoint* REST de ingestão na plataforma Node-RED:

Recepção do Pedido HTTP POST (http in)

O nó `[post] /GuardarDados` atua como a porta de entrada da aplicação. Está configurado para escutar requisições do método POST, recebendo e interpretando diretamente o *payload* JSON enviado pelo `REST client` do Kettle.

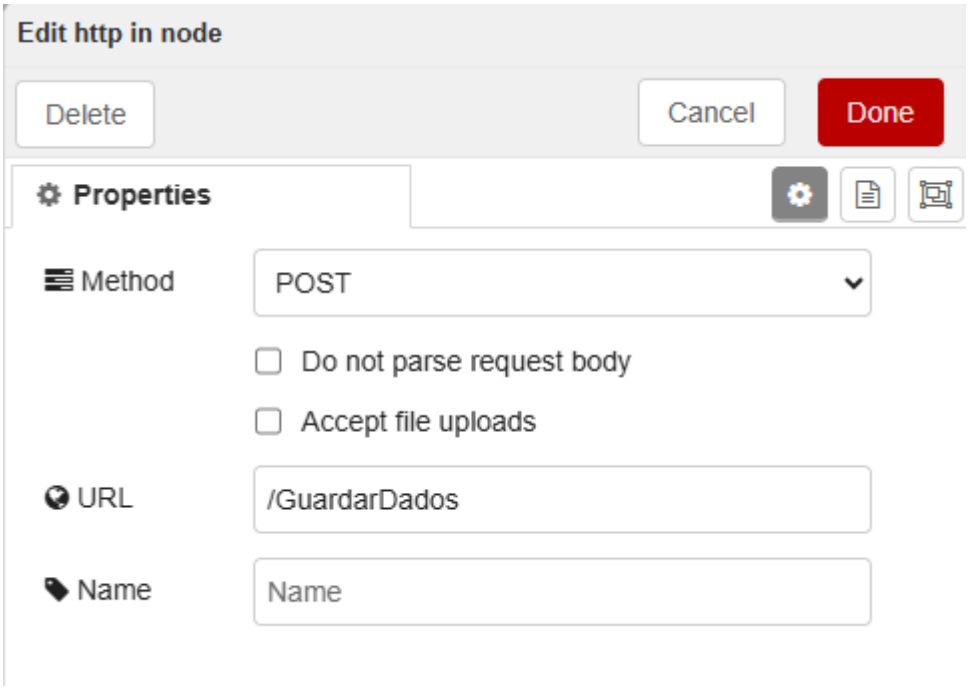


Fig 61 - http in (Endpoint /GuardarDados)

Confirmação da Resposta HTTP (http):

Para fechar a transação HTTP, o nó `http` devolve a resposta ao Pentaho Kettle, confirmando que os dados foram recebidos pela plataforma com êxito.

26 Encaminhamento Lógico, Alertas e Dashboards

A partir do momento em que os dados entram no sistema, o fluxo divide-se em paralelo para garantir a notificação, formatação e visualização gráfica:

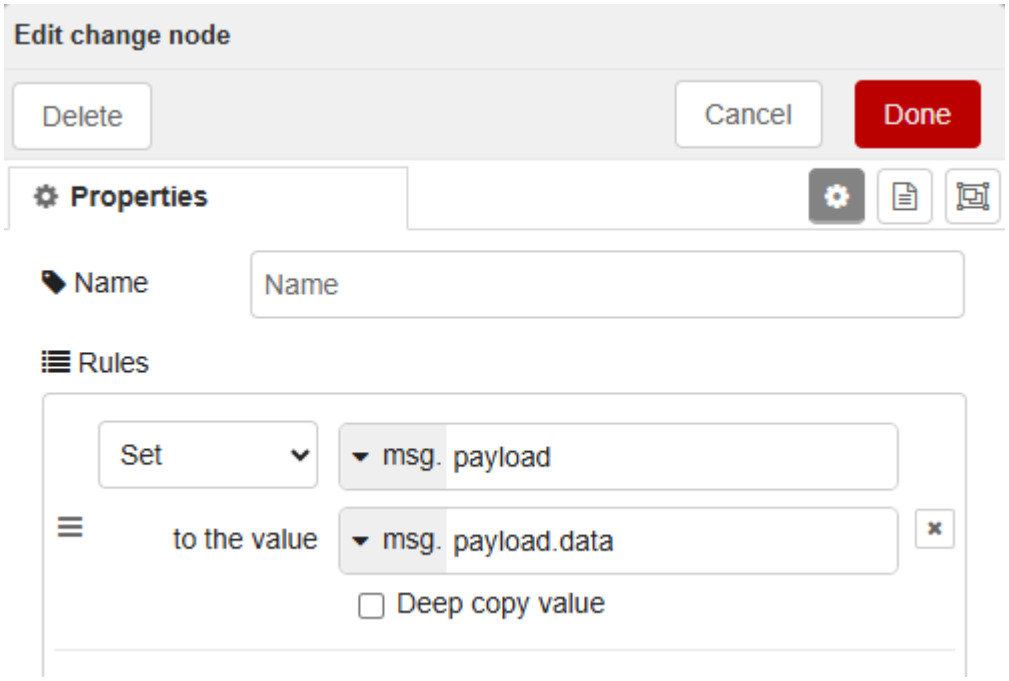


Fig 62 - change node

Tabela Dinâmica (change e table):

Para estruturar o resumo do torneio, o *payload* passa primeiro pelo nó `change` (`set msg.payload`). Este nó extrai o *array* de dados específico enviado pelo Kettle (geralmente alojado em `msg.payload.data`) e passa-o de forma limpa para o nó `table`, que renderiza automaticamente as linhas e colunas no *dashboard* sem necessidade de código HTML complexo.

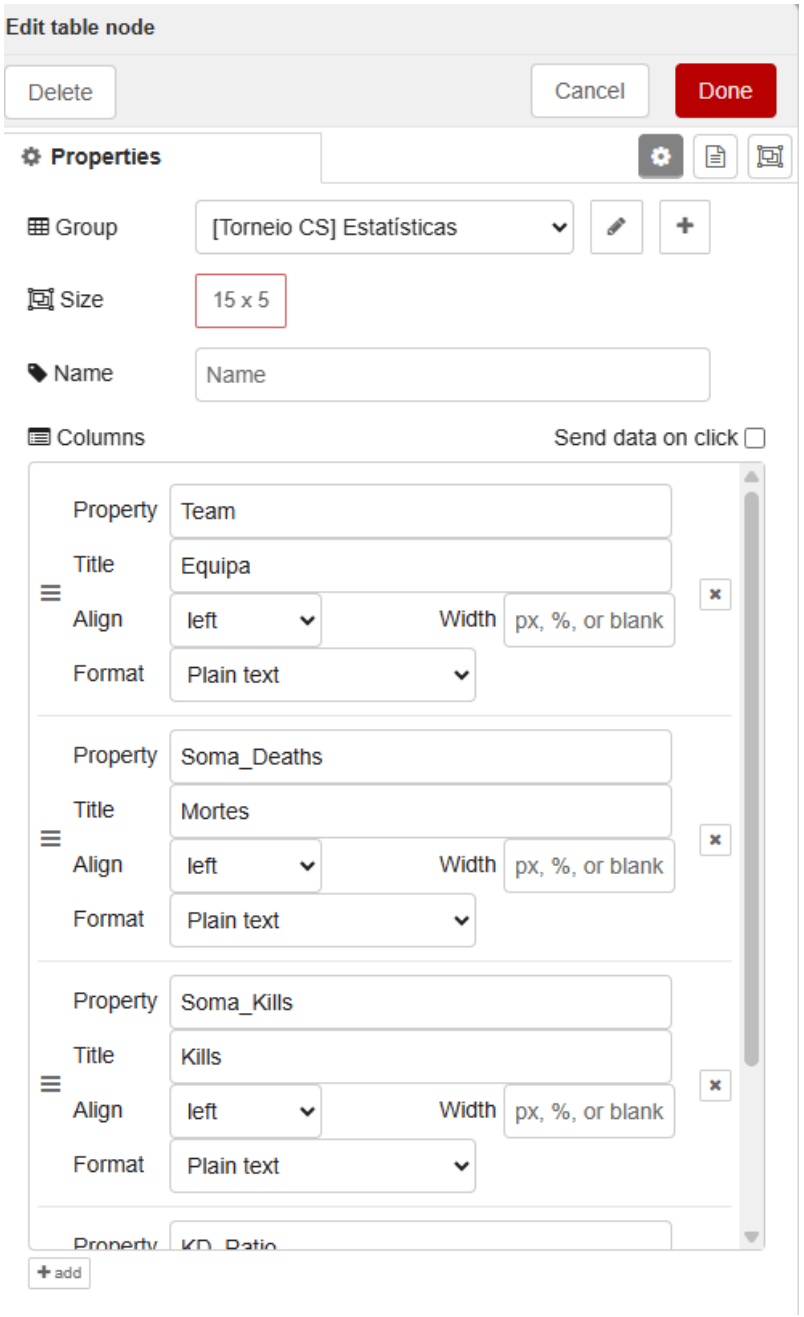


Fig 63 - table node

Envio Automático de Alertas por E-mail:

O nó `Alerta / Relatório do Torneio de CS` implementa o mecanismo de notificação via protocolo SMTP. Configurado com credenciais de aplicação, este nó dispara um e-mail com as métricas do torneio assim que o Kettle envia novos resultados

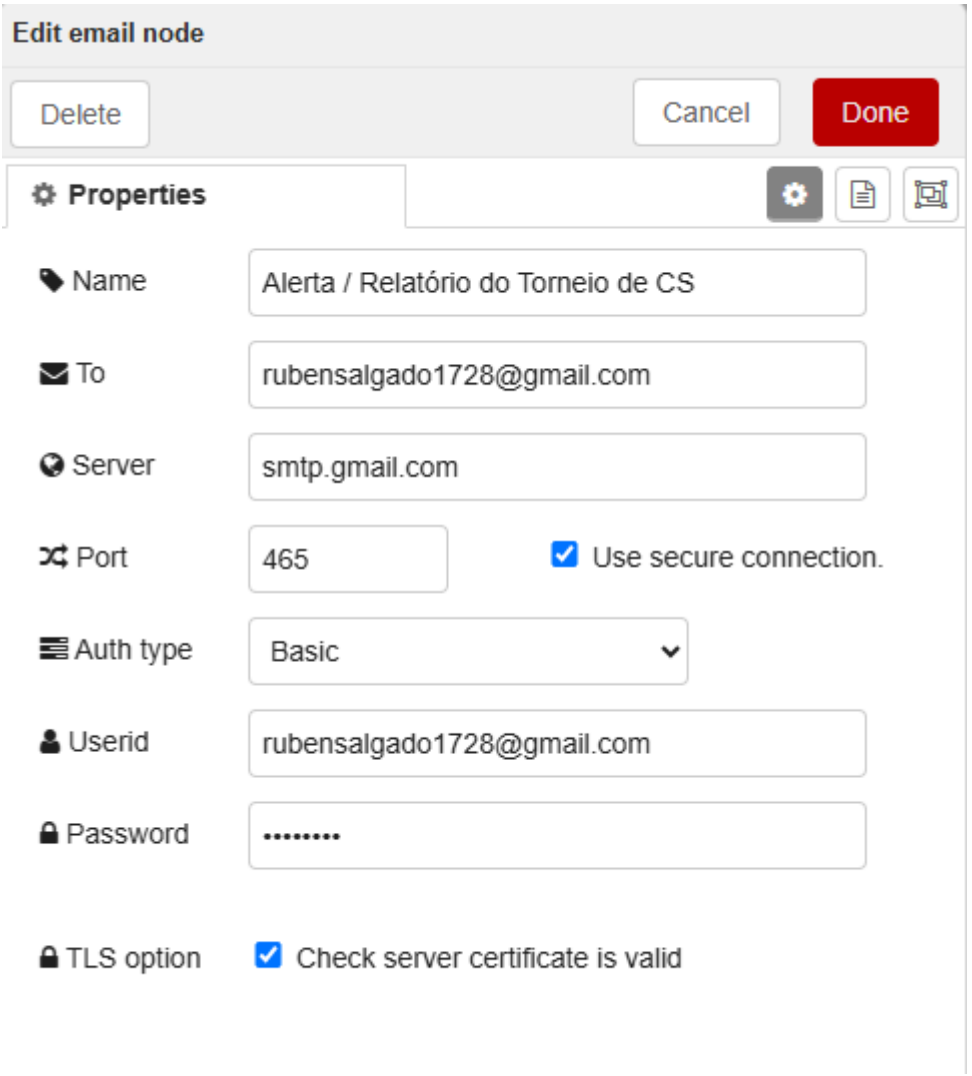


Fig 64 - email node (Alertas SMTP)

Separação Lógica por Equipas (Functions):

Para alimentar os indicadores visuais individuais, a informação é bifurcada para três nós de função independentes (`NaVi`, `G2` e `Vitality`). Cada nó executa um algoritmo em JavaScript que filtra os dados pela respetiva equipa, extraindo unicamente a métrica do Rácio.

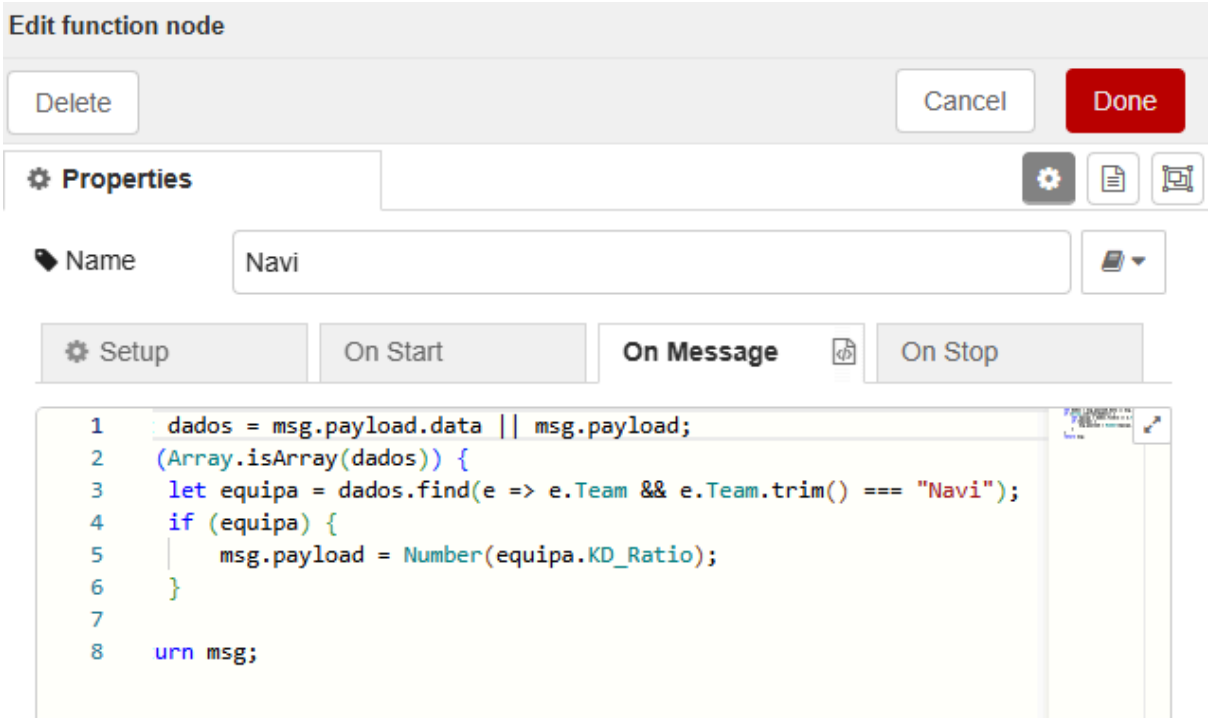


Fig 65 - function node

Manómetros de Desempenho (Gauges):

Interligados às funções, foram configurados os três nós do tipo Gauge (`NaVi - K/D Ratio`, `G2 - K/D Ratio` e `Vitality - K/D Ratio`). Parametrizados na escala adequada, traduzem visualmente o desempenho das equipas no *dashboard*.

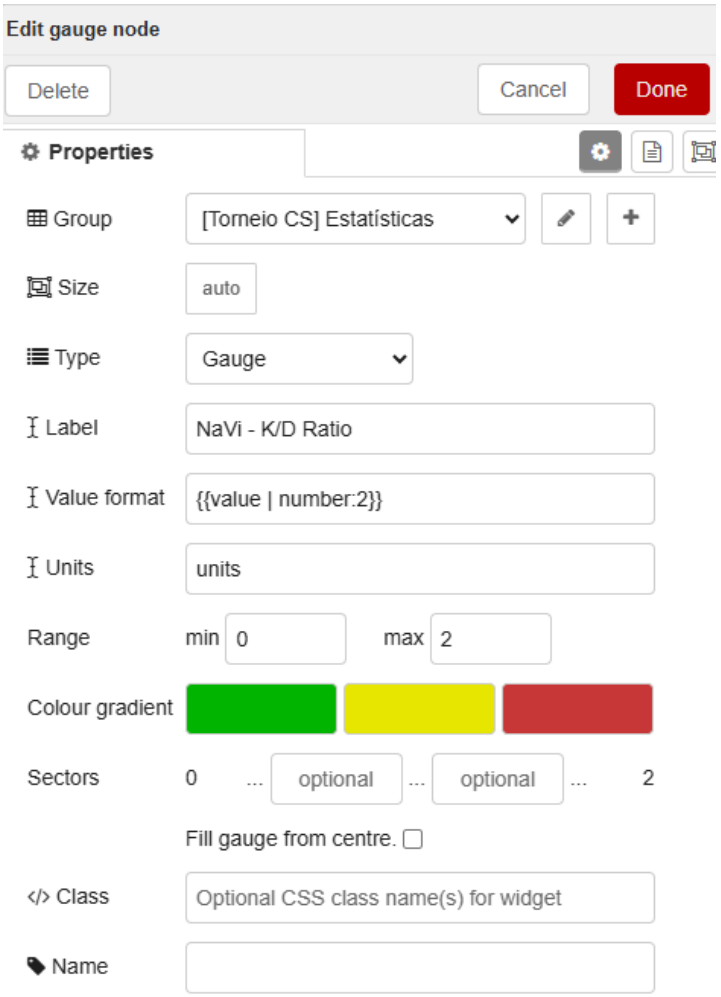
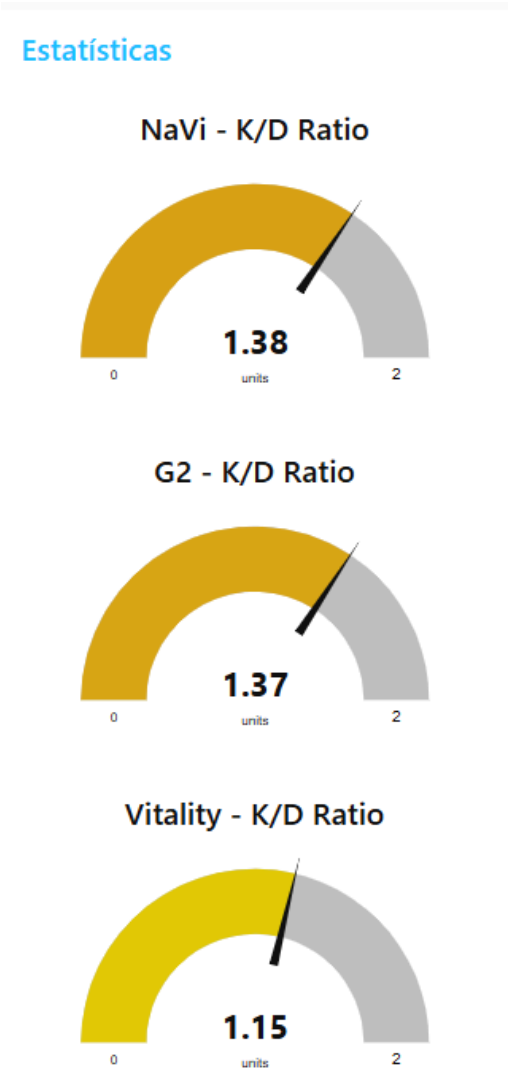


Fig 66 - gauge node

27 Interface Gráfica e Visualização Final

Node-RED Dashboard:

A camada de apresentação visual consolida a arquitetura. Acessível através do *browser*, o *dashboard* exibe em tempo real os resultados processados pela *pipeline* do Kettle, apresentando os três manómetros independentes e a tabela de resumo, refletindo instantaneamente a performance das equipas



Equipa	Mor...	Kills	K/D
Navi	185	255	1.378378...
G2	192	263	1.369791...
Vitality	200	230	1.15

Fig 67 - Dashboard Node-RED

28 Vídeo com demonstração (QR Code) - Simão Vidal



Caso nao deia apresento o link para a pasta:

https://drive.google.com/drive/folders/1sjAtQIdBVRtlOlL40XDllN8fFPWNrfd5?usp=sharing

29 Video com Demonstração (QRCode) - Rúben Salgado



Link da drive com o video:
https://drive.google.com/file/d/1G8mCgfqIIa0vhR3KEYZ0Rk0zt4WIzOG_/view?usp=drive_link

30 Conclusão e Trabalhos futuros

Conclusão

O desenvolvimento deste projeto no âmbito de ISI permitiu consolidar com sucesso a integração entre o KNIME Analytics Platform e o Node-RED. Através de uma pipeline end-to-end focada em eSports (Torneio de CS2), demonstrou-se a eficácia do KNIME na ingestão multiformato, anonimização de dados por Regex e cálculo de métricas (K/D Ratio). A camada do Node-RED garantiu a receção via REST POST, o envio de alertas automáticos por e-mail e a visualização dinâmica em tempo real através do Dashboard, validando a arquitetura proposta.
Em paralelo, a implementação desenvolvida no Pentaho Data Integration (Kettle) provou ser igualmente robusta e eficaz. O Kettle demonstrou grande capacidade e flexibilidade na extração e limpeza de dados estruturados (CSV) e semi-estruturados (JSON), validação de regras de negócio através de Expressões Regulares (Regex) e cruzamento de dados relacionais. A integração direta com o Node-RED via requisições HTTP POST através do nó REST Client funcionou de forma fluida, provando que é perfeitamente possível atingir os mesmos objetivos de interoperabilidade e *streaming* de dados utilizando diferentes ferramentas de ETL, enriquecendo assim a análise comparativa deste trabalho de grupo.

Trabalhos futuros

Como direções para evoluções futuras, destacam-se:

- Persistência em Base de Dados: Integração com sistemas relacionais ou NoSQL (ex: PostgreSQL ou MongoDB) para armazenamento histórico.

- APIs em Tempo Real: Ligação a APIs oficiais de eSports para recolha de dados ao vivo.

- Alertas Condicionais Avançados: Disparo de notificações inteligentes baseadas em limiares críticos de desempenho.

- Expansão do Dashboard: Inclusão de novos widgets para análise detalhada por mapas e jogadores.

31 Anexos

Exportação (CSV, JSON e XML)

https://drive.google.com/drive/folders/1DzCPPC5LRJT6PRgLUpckeLGEgzc_nsZo?usp=sharing